

VERO AVAILABILITY MONITOR 3.0.0 - COMPLETE SOURCE FOR REBUILD

Jira ASPF-1578 · Quality AI Automation · a document only: nothing in it runs

What this is: the full source (61 text files) of a local tool that checks whether Vero (CLI, optionally Chat) is available and shows it on a status page. Python 3.8+ standard library only. The HTML/PDF guide is sent separately.

INSTRUCTIONS FOR THE AI ASSISTANT (Vero CLI or other) - recreate the project folder EXACTLY:

1. Create the folder vero-availability-monitor in the location the user gives (default C:\Tools\).
 2. Every file starts with a row "=== FILE: <path> (<n> lines) ===" and ends with "=== END FILE: <path> ===".
 3. Rows between them encode the file, one source line per numbered row:


```
" 12 | content" -> line 12. Every ␣ (open box) in the content is ONE SPACE. It marks leading
                  spaces, runs of 2+ spaces and spaces at a row edge; single spaces between words
                  are printed normally. The ␣ character never occurs in the real files.
      " 12 |" -> line 12 is an empty line.
      "      + more" -> continuation of the previous line: append "more" directly (no space, no newline).
      Ignore page headers/footers ("Vero Availability Monitor ... source for rebuild", "page N").
      Line numbers must run 1..n without gaps; n is in the FILE row. Every file ends with a newline.
```
 4. Write files as UTF-8 without BOM, LF line endings. Do not reformat, translate, summarise or "fix" anything.
 5. Create verify_rebuild.py from the section VERIFY SCRIPT at the end (same row rules), OUTSIDE the project folder, and run: py verify_rebuild.py C:\Tools\vero-availability-monitor
 It checks every file's SHA-256 and sets CRLF line endings where Windows needs them (.bat, START_HERE.txt).
 Recreate any file reported MISSING or DIFFERENT, then run it again until it prints "OK: all files match".
 6. Run the tests inside the folder: py -m unittest discover -s tests -t . Expected: OK (64 tests).
 7. Tell the user it is ready and to double-click setup.bat (see START_HERE.txt).
- Do not run setup.bat, install, doctor, check or the real Vero CLI unless the user asks.

FILE LIST (path · lines · line endings · SHA-256 of the content with LF endings)

.ci/README.md	9	LF	84f4caf43cc1c411273eeab9ba9ded9054a8595c6ef8a85b68bab0d1ee383076
.gitattributes	8	LF	de5647982a8a835e7449c0a776bc039927e23530e9bdc1d588961500bb5af61f
.gitignore	23	LF	7756a240e02140de687ed2f99bc4d70c8e1befd9764916064fb9e7eb4aa2d93c
AGENTS.md	20	LF	44800f1ef26c843a15ceeeddf9569021a991c36318777ed97ab7e9454e57d8bb
CHANGELOG.md	19	LF	e0c7d175509ba7deea14ba3e4d0055de3d6787c1242210e24b8de0f8c7f9b7d7
CODEOWNERS	2	LF	695a47f89d8c85232ee4946424a5f6a87f2fcea7a945bbcbabb6701981b09d44
CONTRIBUTING.md	13	LF	0309ecb3676545058b6c326e9f4c90ba88b4e62f484b74c59b774d0c55f0fdef
README.md	42	LF	9becd6a527151eeacdf2d886d603ab73d2ce8a65d02c1c683ba1d26da25f85de
START_HERE.txt	32	CRLF	2d1c4ef3ea253fcd055ee7882780ecd85636c481e38005f3f69db1bb9c61e871
VERO_CONTEXT.md	166	LF	d3fa978304042979db68092fd43a3788ded2c788dfc4798e49a9eadc2c20d22c
check_now.bat	7	CRLF	4bb4f1d8e7f19ffdc273ca2b4809be61f6d0644faba3bd4a1a7d8e5a81581748
config/monitor.example.json	49	LF	9e78c6a7a86a94f21d5a62e20e7ec1e3479765a568b3aa554066a50a0f740c35
docs/ARCHITECTURE.md	38	LF	44ff1006d66b37dee2156ac7c4425ae2bbd2ea18fcbce0de5f205a2d8797cbb6a
docs/DESIGN_SYSTEM.md	16	LF	fee93a3b440e1dab0f5ab8d7c4e4e1e7ea2090fdf1e5ea5b559f9b959f7c9b2d
docs/PRD.md	25	LF	884cfa889e035fe1a5d5979df28f26e34f5cb9b545130c102d121e3a87a4fc61
docs/decisions/ADR-001_stdlib-only.md	6	LF	0e8be0912e36d2a913c3185ba34e08f57bc39ac457bd6eba1648fe048f4a00e8
docs/decisions/ADR-002_availability-not-latency.md	8	LF	e1eba8243b818ec04e5da7d10e81b054b02ca52f7dd7f8c05cd41afefa91c297
docs/decisions/ADR-003_no-yolo-sandbox.md	9	LF	8dbd2cab993e4b50ed65cc98427a18348a82bdacc43d7b9bab09398aa76dce72
docs/decisions/ADR-004_vero-chat-handshake.md	8	LF	d2a7fb7b0b6032cf142e87684d8a8b6946d091dc021d94cfaa8a51f4c772dde6a
pyproject.toml	44	LF	536774c15d69d08ec17acd662efa7ace556ac35d713c8a9fa5d38dc4831e0ee5
requirements.txt	1	LF	97a344f15656255459c548b3281186ad0860a97ae2c8d962911507d6102ec837
scripts/_py.bat	17	CRLF	ee03008dcfe04647b013bb12a8f75bfaabc1c2ebbc3395986b8829ffdd9cc96
scripts/build_rebuild_md.py	147	LF	d2630ee8d7d70ea5e6b533de0df3eeef1ac32d115e34c772f4536277000bac0
scripts/build_rebuild_pdf.py	224	LF	943c126f1de7b3ad64188014fe7989b1d3a5eb3b87a28e54683c7accd1025109
scripts/fake_vero.py	76	LF	8ac166e8e0e0b845a37161c46ed3192ded0a91a35beb10222bfeac86289a6fd7
scripts/seed_demo_data.py	59	LF	de8000130adac8ddad6337cd3c3105120af9247814f2238cb89f535f6afe53f5
setup.bat	34	CRLF	1676ff1a7460c049868868237db86fba6173e107ea78a2f321b56f40b3ae02f6
setup_demo.bat	12	CRLF	4881f05e4c2bf053ed4d0cc03fe585ef4544b549d7e43a9f1b9b28291de648c
src/vero_monitor/__init__.py	4	LF	c77b5ab0da53e27487203d17afb38f8dc9ba0c68a7962759827afd5c9c27aee6
src/vero_monitor/__main__.py	5	LF	13a1a5b340cdcf1902b62be90e508c7c71886000d5bf087e7854aadf09fb35e
src/vero_monitor/checks/__init__.py	46	LF	4e22f94dd92edc74066e117897f3a52506ed23460fdcc8b42192aeede28a3fc38
src/vero_monitor/checks/process.py	100	LF	bfeebac9be3fe90a93c0cb0d68f4b1d121aac015d5154b8b7626a021c509fd94
src/vero_monitor/checks/vero_chat.py	90	LF	f08bf1bf4287d3ccb0e20395517fdd5f49481f91c0bedbab4373e4af4e9e9209
src/vero_monitor/checks/vero_cli.py	137	LF	3ddceef9910b68e3f54311b3996aff4e53d8175c82761e855231ad712c35fefef
src/vero_monitor/cli.py	137	LF	4c4986cdc26cc381eff0ac137e195eb8105c238a196b138f2b2dc4f5dc8d5b56
src/vero_monitor/config.py	209	LF	e7d56144b413831fe7acef7313ec9ef7890f4750dbbc9753b4f07c5c636d1aec
src/vero_monitor/domain.py	152	LF	a6baf2d88794a41ffa5a50cfcdf60a8c0fe0556afe8e8621a556bb6474b0a2cd

src/vero_monitor/report.py	91	LF	ecf99cc7db72004040d9d44f247243b701d26e431bb30af6d1734221cbf5cbe
src/vero_monitor/runner.py	142	LF	9c72b7358f12d5c068731c7560eac9efe744f95008c68e3175235a9b5ca27834
src/vero_monitor/scheduler.py	295	LF	758cf1aba70498ae6a42afc6e603f77839af12855ed453bf3e9cb05a3c462b0b
src/vero_monitor/server.py	184	LF	42a1833e37c8f670a622242e5b6d6d8ffb7c685a20b25ab8ee88328256b58da6
src/vero_monitor/store.py	166	LF	b144a73706a236cc8bc60e847995175e4fc644cd8495ff497110845747de4e0a
src/vero_monitor/web/dashboard.html	306	LF	d43abb27805424ff80a82f8b157d7d76edcf9c18c09db758502daa3f1723b679
start_dashboard.bat	7	CRLF	c9b963844055acd44896d5c42375bf0b84e41edb5190aa65cdfa576dd6fa6b9b
status.bat	7	CRLF	6a754b5b8c1f74cd0842a9641d535ea25baf4dd5349a1819ce56703ec33d65b
tests/__init__.py	1	LF	7e9af23c4d8769e855436c950c52874ef9d5ac0c36f24531913b3ca965d6860f
tests/fixtures/.gitkeep	1	LF	2be8225f3a9ad9e9ee49e2dbc7a20ef74b7d127b93047271e68eb3ba7d3992ec
tests/fixtures/vero_task_stream.ndjson	8	LF	e4018110354d711892f0e577d030159c43140f0e405edc88af74ccd2569a220b
tests/helpers.py	29	LF	0b35f2b3014752b801454367100e14ba0b2744ee9fdb9ad8a4c6d0957d6a28e2
tests/integration/__init__.py	1	LF	629eee0283bc7304250576c851f51e068a0625c70ada33c06f5abb2e3f8dedbc
tests/integration/test_chat.py	104	LF	4462c0068b40fc9d56a3800129a92653e849006c84976a65ed6c5345964133d4
tests/integration/test_checks.py	118	LF	d033eb5c33fbbed346aa1700d1ac3aafc9eafc67c5d67b9d03a9d052809592d6
tests/integration/test_runner_server.py	147	LF	3bd30361d1bed25273da1911bf7a96321d22ddbf21b52a123bc44e63d41311b3
tests/unit/__init__.py	1	LF	34d8097b45b98a68acd466b69b9ba1cec7b2d93164ec960b913db0795e38c290
tests/unit/test_config.py	81	LF	b089b300e44f15cafe6b466898610440389f4cc31a61eda5660877c56b1d2a91
tests/unit/test_domain.py	112	LF	9401a448b001527ce750da702d1db15a9c27ccea51a2a4523d0b4c1b6c0339c2
tests/unit/test_parsers.py	80	LF	83b0765d6362af1683ef5d3d6452e3707bea16cd03deb240b2036bee96e2fb3a
tests/unit/test_scheduler.py	49	LF	6d6460ec569ce5a126ab5a706932b6c7b197f61854b171360fe65deb2a15d959
uninstall.bat	7	CRLF	bce5f932e3942a7e39f3611867464061e55713a513f0526388368abed454e464
uv.lock	8	LF	e01fcba4c5572550cf4f0d84f325540253b782395df574ab4981bbc8d033a9fd
vam.py	12	LF	25f30ade67d0e13c78622e8bc991228535d2535de206892795e9da55f7d5f8f1

=== FILE: .ci/README.md (9 lines) ===

```
1 | # CI
2 |
3 | Plan `QAI-vero-availability-monitor-PRGATE`: ruff check, mypy (strict), unit + integration tests, secret scan.
4 | ``
5 |
6 | uvx ruff check .
7 | uvx mypy
8 | python -m unittest discover -s tests -t .
9 | ``
```

=== END FILE: .ci/README.md ===

=== FILE: .gitattributes (8 lines) ===

```
1 | * text=auto
2 | *.pyuutext eol=lf
3 | *.shuutext eol=lf
4 | *.mduutext
5 | *.bat text eol=crlf
6 | *.png binary
7 | *.jpg binary
8 | START_HERE.txt text eol=crlf
=== END FILE: .gitattributes ===
```

=== FILE: .gitignore (23 lines) ===

```
1 | __pycache__/  
2 | *.py[cod]  
3 | .venv/  
4 | venv/  
5 | .env  
6 | .env.*  
7 | *.log  
8 | .idea/  
9 | .vscode/  
10 | dist/  
11 | build/  
12 | *.egg-info/  
13 | .pytest_cache/  
14 | .mypy_cache/  
15 | .ruff_cache/  
16 | data/  
17 | data-demo/  
18 | reports/  
19 | config/monitor.json  
20 | *.docx  
21 | *.xlsx  
22 | *.pdf  
23 | *.pptx
```

=== END FILE: .gitignore ===

```
=== FILE: AGENTS.md (20 lines) ===
```

```

1 | # Agents: vero-availability-monitor
2 |
3 | ## Role
4 | Monitors whether Vero is available. It is not an AI agent itself: it starts the Vero CLI with one fixed, harmless prompt and checks that an answer comes back.
5 |
6 | ## Rules
7 | - Never pass `--yolo`; the task runs in an empty sandbox folder (`-c data/sandbox`). See ADR-003.
8 | - Only `completion_result` counts as an answer. Never store or log the event stream; `api_req_started` contains the full upstream prompt.
9 | - Never store model answers, tokens or `vero.env` values. Error details are one line, max 200 chars, with credential-like text masked.
10 | - The Vero Chat token is read from an environment variable at run time only (ADR-004).
11 | - Every run has a run_id `-vero-availability` and a JSON manifest that records the model reported by Vero (`modelInfo`), not the configured one.
12 |
13 | ## Tools allowed
14 | Python 3.8+ standard library (ADR-001). Vero CLI and Vero Chat as configured.
15 |
16 | ## For coding agents
17 | - Layers: `domain.py` (pure) <- `checks/` (adapters) <- `runner.py`, `report.py` (application) <- `cli.py`, `server.py`, `scheduler.py` (interfaces). Do not import upwards.
18 | - Before every commit: `uvx ruff check .`, `uvx ruff format --check .`, `uvx mypy`, `uvx mypy --platform win32`, `python -m unittest discover -s tests -t .`
19 | - Tests use `scripts/fake_vero.py` and a local fake MCP server, never the real Vero.
20 | - Commit and PR title: `ASPF-1578: <imperative summary>`.
=== END FILE: AGENTS.md ===
```

```
=== FILE: CHANGELOG.md (19 lines) ===
```

```

1 | # Changelog
2 |
3 | ## Unreleased
4 |
5 | ## 3.0.0
6 | Rebuilt from scratch as an **availability** monitor (scope change from the 0.x latency monitor; ADR-002).
7 | - Checks: `cli` (vero version), `task` (`vero task --json`, end-to-end, `completion_result` only), optional `chat`
8 |   (Vero Chat MCP initialize handshake, no model call). Overall status per run: available / degraded / unavailable.
9 | - Status page: current status and since when, components, availability 24 h / 7 d / 30 d, status timeline,
10 |  incidents (outages, or degradation lasting 2+ checks), recent checks; live (SSE); CSV; self-contained snapshot.
11 | - Uses the measured Vero CLI 2.3.x contract: `--json` event stream, pinned `-m`, `-t` timeout, `modelInfo` recorded.
12 |   No `--yolo`; runs in an empty sandbox folder (ADR-003). Vero Chat token from an environment variable (ADR-004).
13 | - Structure: pure domain layer, adapters, application services, interfaces; strict typing (mypy strict, Linux and
14 |   Windows), ruff lint + format, 64 unit and integration tests on Python 3.8 to 3.13.
15 | - Kept from 0.2: guided setup, Task Scheduler XML (laptop-safe), dashboard at logon, Host/Origin checks, lock file,
16 |   stale-lock recovery, rotating log, run manifests, rebuild document for channels without archives.
17 | - Coexists with the 0.x latency monitor: different port (8766), task names and data.
18 | - Distribution as documents only: `..._Source.pdf` (numbered rows, visible space mark, SHA-256 verify script;
19 |   rebuild verified byte-identical with pypdf and pdfplumber extraction) and the guide as PDF.

```

```
=== END FILE: CHANGELOG.md ===
```

```
=== FILE: CODEOWNERS (2 lines) ===
  1 | # path reviewers
  2 | * @qai-reviewers
=== END FILE: CODEOWNERS ===
```

=== FILE: CONTRIBUTING.md (13 lines) ===

```
1 | # Contributing
2 |
3 | - Branch per NXP internal strategy, with Jira key
4 | - Commit and PR title: <JIRA-KEY>: <summary>
5 | - PR into main: 1 peer approval required, SME optional
6 |
7 | ## PR checklist
8 | - [ ] Jira key in branch, commits, title
9 | - [ ] Tests added or updated (synthetic fixtures only)
10 | - [ ] Check logic, AI agent, model or coverage change: golden set re-run, SME invited
11 | - [ ] PRD / ARCHITECTURE / AGENTS updated if behaviour changed
12 | - [ ] No secrets, office documents or run outputs
13 | - [ ] CHANGELOG updated
=== END FILE: CONTRIBUTING.md ===
```

```
=== FILE: README.md (42 lines) ===
```

```

1 | # vero-availability-monitor
2 |
3 | Purpose: answer one question continuously: **is Vero available right now?** A status page for Vero CLI (and optionally Vero Chat).
4 | Owner: << OWNER >> (Quality AI Automation, GenAI Group 3)
5 | Jira: ASPF-1578 (AES SW Process Framework) · Version 3.0.0 · Rules: SCMP (ASPF-1619), CONTRIBUTING.md
6 |
7 | Guide: [docs/VeroAvailabilityMonitor_Guide.html](docs/VeroAvailabilityMonitor_Guide.html) · AI context: [VERO_CONTEXT.md](VERO_CONTEXT.md)
8 |
9 | ## What it checks (every 15 min by default)
10 |
11 | | Check | How | Cost |
12 | |---|---|---|
13 | | `cli` | `vero version`: installed and starts | free, ~1 s |
14 | | `task` | `vero task --json -m <model> -t 180 -c <empty dir>` "Reply with exactly: OK": end-to-end answer, `completion_result` event | 2 small model calls, ~1 min |
15 | | `chat` (optional) | Vero Chat MCP `initialize` handshake, token from an environment variable | free, no model call |
16 |
17 | Result per run: **available**, **degraded** (slow, unexpected answer, or a secondary check down), **unavailable**.
18 |
19 | ## Install (Windows)
20 |
21 | Python 3.8+ only, no pip, no admin rights.
22 |
23 | 1. Get the folder to e.g. `C:\Tools\vero-availability-monitor\` (git clone, or let an AI rebuild it from `VeroAvailabilityMonitor_3.0.0_Source.pdf`).
24 | 2. Optional: `setup_demo.bat` (fake CLI + sample data).
25 | 3. `setup.bat`: questions -> one real test -> **Y** installs the scheduled check, the status page at logon and a desktop shortcut "Vero Status".
26 | 4. `status.bat` any time; `uninstall.bat` to remove (data kept).
27 |
28 | ## Commands
29 |
30 | ```
31 | python vam.py configure | doctor | check | serve [--open] | install | uninstall | status | open | export | report | init [--demo]
32 | ```
33 |
34 | ## Develop
35 |
36 | ```
37 | python -m unittest discover -s tests -t . -p *_*.py --pattern win32_* # 64 tests, fake CLI + fake MCP server only
38 | uvx ruff check . && uvx ruff format --check . # lint + format
39 | uvx mypy && uvx mypy --platform win32_* # strict typing, Linux and Windows
40 | python scripts/build_rebuild_md.py -d dist/VeroAvailabilityMonitor_<ver>_REBUILD.md
41 | uv run --no-project --with reportlab python scripts/build_rebuild_pdf.py -d dist/..._Source.pdf + verify_rebuild.py
42 | ```

```

=== END FILE: README.md ===

```

=== FILE: START_HERE.txt (32 lines) ===
1 | VERO AVAILABILITY MONITOR 3.0.0 (ASPF-1578)
2 | =====
3 |
4 | Shows whether Vero is available right now, with history, on a local status page.
5 | Needs Python 3.8+ only. No admin rights, no internet download, nothing to install with pip.
6 |
7 | 0. If you received VeroAvailabilityMonitor_3.0.0_Source.pdf: ask Vero CLI (or another AI
8 |   assistant) to follow the instructions on page 1 and rebuild the project in C:\Tools.
9 |
10 | 1. Put this folder somewhere local and fixed, e.g. C:\Tools\vero-availability-monitor
11 |    (not Downloads, not a OneDrive folder)
12 |
13 | 2. (optional) Double-click setup_demo.bat -> status page with a fake CLI and sample data
14 |
15 | 3. Double-click setup.bat
16 |    - answer the questions (Vero program, model, checks, timing; Enter = default)
17 |    - it tests Vero once (the task check takes about a minute)
18 |    - answer Y to install background monitoring
19 |    The status page opens. Later: desktop shortcut "Vero Status" or http://127.0.0.1:8766/
20 |
21 | Other files:
22 |    start_dashboard.bat - open the status page (starts it if needed)
23 |    check_now.bat - check Vero now
24 |    status.bat - what is configured and running
25 |    uninstall.bat - remove scheduled tasks and shortcut (data is kept)
26 |
27 | If the old "Vero Latency Monitor" (0.x) is installed, run uninstall.bat in its folder:
28 | both would call Vero every 15 minutes.
29 |
30 | Full guide: docs\VeroAvailabilityMonitor_Guide.html
31 | For Vero CLI / AI agents: VERO_CONTEXT.md
32 | Problems: status.bat, then data\monitor.log, then the guide, section Troubleshooting.
=== END FILE: START_HERE.txt ===

```

=== FILE: VERO_CONTEXT.md (166 lines) ===

```

1 | # VERO_CONTEXT: vero-availability-monitor 3.0.0
2 |
3 | > **For the AI reading this:** this file describes the whole project.
4 | > - Use it to answer questions or make changes safely.
5 | > - Read a file before editing it, and keep the rules in section 7.
6 | > - Answer from this file and the repository only. Items marked `<< TO CONFIRM >>` are open, so do not guess.
7 |
8 | ## 1. What it is
9 |
10 | A small local tool that answers one question continuously: **is Vero available right now?** It shows the answer on a status page at `http://127.0.0.1:8766`.
11 |
12 | - **Jira:** ASPF-1578 (AES SW Process Framework). Quality AI Automation team, NXP.
13 | - **Governed by:** the SCMP of ASPF-1619.
14 | - **Runtime:** Python 3.8+ standard library only. Windows first; Linux and macOS work too.
15 | - **History:** 3.0.0 replaces the 0.x "latency monitor" (ADR-002). Both can run side by side: they use different ports, task names and data folders.
16 |
17 | ## 2. Checks, every 15 min by default
18 |
19 | | Check | Command / call | Up when | Degraded when | Down when | Cost |
20 | |---|---|---|---|---|---|
21 | | `cli` | `vero version` | exit 0 | - | not found, exit ≠ 0, or timeout (30 s) | free |
22 | | `task` | `vero task --json -m {model} -t {timeout} -c {workdir} {prompt}` | a `completion_result` event arrives | slower than `thresholds_s.task_slow` (90 s), or the answer lacks `expect` | no,
+ `completion_result`, exit ≠ 0, or the hard limit is hit | 2 small model calls, about 1 min |
23 | | `chat` (optional) | POST JSON-RPC `initialize` to Vero Chat MCP, `Authorization: Bearer ${token_env}` | a result with `serverInfo` | answer slower than 10 s | token variable missing, HTTP 401/4
+ 03, unreachable, MCP error, or a reply that is not MCP | free, no model call |
24 |
25 | Rules that tie the checks together:
26 | - **Order:** `cli`, then `chat`, then `task`. If `cli` is down, `task` is not run and is recorded as down ("skipped").
27 | - **Overall run status:**
28 |   - DOWN if the primary check is down. The primary check is `task` when it is enabled, otherwise `cli`.
29 |   - DEGRADED if any other check is down, or any check is degraded.
30 |   - UP otherwise.
31 | - **Availability** = runs that are not DOWN, divided by all runs.
32 | - **Incident** = consecutive non-UP runs that include an outage, or a degradation lasting 2 or more runs. It ends at the next UP run.
33 |
34 | ### Vero CLI `--json` contract (Vero CLI 2.3.x, integration guide §3.3)
35 |
36 | - The stream has one JSON object per line. The **only** answer is `say == "completion_result"`.
37 | - Ignore `partial: true` fragments.
38 | - Record `modelInfo.providerId` and `modelInfo.modelId`. Never assume them from the config.
39 | - `task_started.taskId` identifies the task.
40 | - `api_req_started.text` contains the full upstream prompt, so it is **never** stored or logged. The parser keeps only the fields above.
41 | - Never pass `--yolo` (ADR-003). The task runs with `-c data/sandbox`, an empty folder.
42 | - The process tree is killed at `-t + min(30 s, -t)`; with the defaults that is 210 s.
43 |
44 | ## 3. Structure
45 |
46 | Dependencies point downwards only.
47 |
48 | ```
49 | vam.py#####entry point (adds src/ to sys.path)
50 | src/vero_monitor/
51 |   domain.py#####PURE: Status, Check, CheckResult, RunResult, Point, Incident; overall_status,
52 |   #####availability_percent, incidents, timeline, current_since, worst
53 |   config.py#####typed frozen dataclasses (Config, VeroSettings, ChatSettings); DEFAULTS; build/load/validate
54 |   checks/
55 |     process.py#####run(args, timeout) -> ProcessResult; kill_tree; resolve (shutil.which)
56 |     vero_cli.py#####check_cli, check_task, parse_stream, task_command, hard_limit_s, clean (mask secrets)
57 |     vero_chat.py#####check_chat, parse_reply (JSON or SSE)
58 |     __init__.py#####enabled(cfg), run_all(cfg)#####(contains crashes, skips task when cli down)
59 |     store.py#####SQLite data/availability.db: runs, checks (ON DELETE CASCADE)
60 |     runner.py#####run_once: RunLock (data/run.lock, stale recovery) -> run_all -> overall -> save -> prune -> manifest
61 |     report.py#####payload(cfg, store, hours) for the status page; static_html snapshot
62 |     scheduler.py#####Loop (in-process timer); Task Scheduler XML / cron install, remove, status; desktop shortcut
63 |     server.py#####HTTP: / , /api/status, /api/stream (SSE), POST /api/check, /api/export.csv, /api/health
64 |     cli.py#####commands (section 5)

```

```

65 | web/dashboard.html status page, one file, vanilla JS, no CDN
66 | scripts/_fake_vero.py (stand-in CLI), seed_demo_data.py, build_rebuild_md.py, build_rebuild_pdf.py (dev, reportlab), _py.bat
67 | tests/unit/ (domain, parsers, config, scheduler) integration/ (fake CLI, fake MCP, runner, HTTP)
68 | docs/guide HTML, PRD, ARCHITECTURE, DESIGN_SYSTEM, decisions/ADR-001..004
69 | *.bat setup, setup_demo, start_dashboard, check_now, status, uninstall
70 | ```
71 |
72 | ## 4. Data
73 |
74 | **`config/monitor.json`**
75 | - Defaults live in `config.DEFAULTS`, and `monitor.example.json` equals them.
76 | - The JSON keys are:
77 |
78 | | Key | Contents |
79 | |---|---|
80 | | `vero` | `command`, `model`, `prompt`, `expect`, `task_timeout_s`, `task_args`, `version_args`, `env` |
81 | | `checks` | `cli`, `task`, `chat` |
82 | | `chat` | `url`, `token_env`, `timeout_s`, `ca_bundle` |
83 | | `thresholds_s` | `task_slow`, `chat_slow` |
84 | | `schedule` | `interval_minutes`, `run_in_dashboard` |
85 | | | `retention_days` |
86 | | `server` | `host`, `port` (8766) |
87 | | | `data_dir` |
88 |
89 | - Validation raises `ConfigError` with a readable message. Among other things it enforces that the worst-case run (`timeout` + min(30, timeout) + 30` s) fits inside the interval.
90 | - The file is read as `utf-8-sig`, so a BOM from Notepad is fine. Environment variables in `data_dir` are expanded.
91 |
92 | **SQLite**
93 |
94 | | Table | Columns |
95 | |---|---|
96 | | `runs` | `run_id` (PK), `started_at`, `epoch`, `trigger`, `overall`, `reason`, `host`, `tool_version` |
97 | | `checks` | `id`, `run_id` (FK cascade), `epoch`, `check_name`, `status`, `duration_ms`, `detail`, `version`, `provider_id`, `model_id` |
98 |
99 | **Run manifest** (`data/runs/<YYYY-MM>.<run_id>.json`):
100 |
101 | ```json
102 | {"run_id","component":"vero-availability","tool_version","trigger","host","started_at","overall","reason",
103 | "model":{"provider","id","configured_id"},"checks":[{"check","status","duration_ms","detail","version"}]}
104 | ```
105 |
106 | The run_id format is `<YYYYMMDD-HHMM>-vero-availability`, with a `-2` suffix for a second run in the same minute.
107 |
108 | ## 5. Commands
109 |
110 | Run as `python vam.py <command>`.
111 |
112 | | Command | What it does |
113 | |---|---|
114 | | `configure` | Guided questions |
115 | | `init [--demo]` | Writes the config. `--demo` uses the fake CLI and `data-demo/`. |
116 | | `doctor` | Runs all checks once **without storing** *. Exit 1 if the result is DOWN. |
117 | | `check [--trigger] [-q]` | Stores one run. Exit 2 if DOWN; always 0 with `--trigger task`. |
118 | | `serve [--open] [--port] [--no-scheduler]` | Status page. If one is already running, it only opens the browser. |
119 | | `install [--every N] [--no-dashboard]` | Registers the Task Scheduler tasks `VeroAvailabilityCheck` (every N min: on battery, catch-up, no overlap) and `VeroAvailabilityDashboard` (at logon, hi
+ dden), adds the desktop shortcut "Vero Status", and sets `run_in_dashboard=false`. On Linux/macOS it uses cron instead. |
120 | | `uninstall` | Removes the tasks and the shortcut. Data is kept. |
121 | | `status` | Shows configuration, last run and task state |
122 | | `open` | Opens the running status page |
123 | | `export` | Writes CSV |
124 | | `report --hours` | Writes a self-contained HTML snapshot |
125 |
126 | ## 6. Quality gates
127 |
128 | ```
129 | uvx ruff check . && uvx ruff format --check .# lint (incl. bandit S, bugbear B) + format, 130 cols
130 | uvx mypy && uvx mypy --platform win32# strict typing for Linux and Windows

```

```

131 | python -m unittest discover -s tests -t .# 64 tests, Python 3.8-3.13, warnings-as-errors clean
132 | ``
133 |
134 | **Fake CLI modes** (`FAKE_VERO_MODE`):
135 |
136 | | Mode | Behaviour |
137 | |---|---|
138 | | `ok` | Answers normally |
139 | | `slow` | Answers late |
140 | | `down` | Provider unavailable, exit 1 |
141 | | `auth` | Unauthorised, with a decoy token in stderr (must be masked) |
142 | | `nocompletion` | Exit 0 without a `completion_result` |
143 | | `refuse` | Answers, but not with `OK` |
144 | | `hang` | Never answers (must be killed) |
145 | | `random` | Default; mostly ok, sometimes slow or failing |
146 |
147 | `FAKE_VERO_DELAY` sets the answer delay. The fake emits a decoy secret in `api_req_started`, and a test proves it is never persisted.
148 |
149 | ## 7. Rules (do not break)
150 |
151 | 1. Standard library only, Python 3.8 compatible. No CDN, no pip dependencies.
152 | 2. Keep `domain.py` pure (no I/O). Never import upwards across the layers.
153 | 3. No `--yolo`. Only `completion_result` is an answer. Never persist the event stream, model answers, tokens or `vero.env` values.
154 | 4. Bind to 127.0.0.1 only, and keep the Host and Origin checks.
155 | 5. One run at a time, through `runner.run_once` (it takes `RunLock`).
156 | 6. Tests never call the real Vero. Every subprocess uses `creationflags=NO_WINDOW`, and every timeout kills the process tree.
157 | 7. Commit and PR title: `ASPF-1578: <summary>`. Never commit `data/`, `data-demo/`, `dist/`, `reports/` or `config/monitor.json`. Update the CHANGELOG, the guide and this file when behaviour chan
+ ges.
158 |
159 | ## 8. Open items (`<< TO CONFIRM >>`)
160 |
161 | - The owner, and the repository location in Bitbucket project QAI.
162 | - Whether Vero is an approved platform component at all. SCMP v0.5 does not mention it (integration guide §9).
163 | - The cheapest pinned model id. The default is Haiku 4.5 on Bedrock, as measured.
164 | - The `task_slow` threshold (default 90 s, based on the measured p95 of 59 s).
165 | - Whether `vero task` without `--yolo` ever waits for approval on newer Vero versions (ADR-003).
166 | - Who owns the Vero Chat token, if the `chat` check is enabled (a functional account, per the SCMP).
=== END FILE: VERO_CONTEXT.md ===

```

```
=== FILE: check_now.bat (7 lines) ===
1 | @echo off
2 | rem Runs all enabled checks once and stores the result.
3 | setlocal
4 | cd /d "%~dp0"
5 | call scripts\_py.bat || exit /b 1
6 | %PY% vam.py check
7 | pause
=== END FILE: check_now.bat ===
```

```
=== FILE: config/monitor.example.json (49 lines) ===
```

```

1 | {
2 |   "vero": {
3 |     "command": "vero",
4 |     "model": "us.anthropic.claude-haiku-4-5-20251001-v1:0",
5 |     "prompt": "Reply with exactly: OK",
6 |     "expect": "OK",
7 |     "task_timeout_s": 180,
8 |     "task_args": [
9 |       "task",
10 |      "--json",
11 |      "-m",
12 |      "{model}",
13 |      "-t",
14 |      "{timeout}",
15 |      "-c",
16 |      "{workdir}",
17 |      "{prompt}"
18 |    ],
19 |     "version_args": [
20 |       "version"
21 |     ],
22 |     "env": {}
23 |   },
24 |   "checks": {
25 |     "cli": true,
26 |     "task": true,
27 |     "chat": false
28 |   },
29 |   "chat": {
30 |     "url": "https://verostudio.sw.nxp.com/mcp",
31 |     "token_env": "VERO_CHAT_TOKEN",
32 |     "timeout_s": 30,
33 |     "ca_bundle": ""
34 |   },
35 |   "thresholds_s": {
36 |     "task_slow": 90,
37 |     "chat_slow": 10
38 |   },
39 |   "schedule": {
40 |     "interval_minutes": 15,
41 |     "run_in_dashboard": true
42 |   },
43 |   "retention_days": 90,
44 |   "server": {
45 |     "host": "127.0.0.1",
46 |     "port": 8766
47 |   },
48 |   "data_dir": "data"
49 | }
```

```
=== END FILE: config/monitor.example.json ===
```

```
=== FILE: docs/ARCHITECTURE.md (38 lines) ===
```

```

1 | # Architecture: vero-availability-monitor
2 |
3 | ## Context
4 | Runs on one Windows laptop (or Linux/macOS). Talks only to the local Vero CLI and, optionally, to Vero Chat
5 | (`https://verostudio.sw.nxp.com/mcp`). Status page on `http://127.0.0.1:8766`.
6 |
7 | ## Components (dependencies point down only)
8 | ```
9 | interfaces_cli.py (commands) server.py (HTTP, SSE) scheduler.py (timer, Task Scheduler, cron)
10 | |-----|-----|-----|
11 | application_runner.py (lock, run, store, manifest) report.py (status payload)
12 | |-----|-----|-----|
13 | adapters_checks/process.py checks/vero_cli.py checks/vero_chat.py store.py (SQLite)
14 | |-----|
15 | domain_domain.py: Status, Check, CheckResult, RunResult, overall_status, availability_percent,
16 | |----- incidents, timeline, current_since (pure, no I/O)
17 | config_config.py: typed, validated, frozen dataclasses
18 | ```
19 |
20 | ## Data flow
21 | trigger -> `runner.run_once` -> lock `data/run.lock` -> `checks.run_all` (cli -> chat -> task; task skipped if cli down)
22 | -> `domain.overall_status` -> `store.save` (runs + checks) -> prune -> manifest `data/runs/<YYYY-MM>/<run_id>.json`
23 | -> `/api/status` + `/api/stream` -> status page.
24 |
25 | ## Status rules (domain.py)
26 | - check: up / degraded / down. Task degraded = slower than `thresholds_s.task_slow` or answer without `expect`.
27 | - run: DOWN if the primary check is down (task if enabled, else cli); DEGRADED if any other check is down or any is degraded; else UP.
28 | - availability = runs not DOWN / runs. Incident = consecutive non-UP runs (outage, or degradation of 2+ runs).
29 |
30 | ## Interfaces
31 | - CLI: `vam configure|init|doctor|check|serve|install|uninstall|status|open|export|report`
32 | - HTTP: `GET /`, `GET /api/status?hours=1|6|24|168|720`, `GET /api/stream` (SSE: run, state), `POST /api/check`,
33 | | `GET /api/export.csv`, `GET /api/health`. Host and Origin checked.
34 | - Vero CLI: `vero version`; `vero task --json -m {model} -t {timeout} -c {workdir} {prompt}` (configurable).
35 | - Vero Chat: JSON-RPC `initialize`, `Authorization: Bearer $VERO_CHAT_TOKEN`.
36 |
37 | ## Dependencies
38 | Python 3.8+ stdlib. Vero CLI 2.3.x (version recorded per run). Model from `modelInfo` recorded per run.
=== END FILE: docs/ARCHITECTURE.md ===
```

=== FILE: docs/DESIGN_SYSTEM.md (16 lines) ===

```
1 | # Design system: vero-availability-monitor
2 |
3 | ## Colours
4 | SCMP tokens: available `#1f7a45`, degraded `#a8500b`, unavailable `#9b1c2e`, no data `#b7bfcf`, brand `#0a6aa1`; tinted
5 | backgrounds for the status banner; full dark-mode token set.
6 |
7 | ## Typography
8 | IBM Plex Sans / Segoe UI; tabular numbers for figures.
9 |
10 | ## Components
11 | Status banner (icon, title, since, reason), component card (dot, name, status, facts), availability tile, status
12 | timeline (bars = worst status per slot), incident table, recent-checks table with per-check dots, live pill.
13 |
14 | ## Rules
15 | Words, not only colour: every status has a label (available / degraded / unavailable / no data). The banner says
16 | what users need first: is Vero available, since when, and why not.
```

=== END FILE: docs/DESIGN_SYSTEM.md ===

```
=== FILE: docs/PRD.md (25 lines) ===
```

```

1 | # PRD: vero-availability-monitor
2 |
3 | ## Problem
4 | The Quality AI Automation team depends on Vero (Vero CLI, Vero Chat). When Vero is unavailable nobody knows until a workflow fails, and there is no record of how often it happens.
5 |
6 | ## Goal
7 | Know at a glance whether Vero is available now, since when, and how available it has been over 24 h, 7 d and 30 d, at the lowest possible cost.
8 |
9 | ## Users
10 | Quality Lead (owner of the figures), Quality AI Automation team, SW QA Engineers.
11 |
12 | ## Requirements
13 | | ID | Requirement | Jira |
14 | |----|-----|-----|
15 | | VAM-001 | Use the cheapest possible end-to-end call (fixed minimal prompt, pinned cheap model) plus free checks | ASPF-1578 AC1 |
16 | | VAM-002 | Time trigger: Task Scheduler (laptop-safe) / cron, or the status page's own timer | ASPF-1578 AC2 |
17 | | VAM-003 | Record every run (SQLite + run manifest) and show it on a status page | ASPF-1578 AC3 |
18 | | VAM-004 | Fully automated after setup (scheduled checks, status page at logon, pruning, log rotation) | ASPF-1578 AC4 |
19 | | VAM-005 | Status per run: available / degraded / unavailable, with the reason | ASPF-1578 |
20 | | VAM-006 | Availability 24 h / 7 d / 30 d, status timeline, incidents | ASPF-1578 |
21 | | VAM-007 | Optional Vero Chat check without any model call | ASPF-1578 |
22 | | VAM-008 | Install from a guided script; distributable as a readable document | ASPF-1578 |
23 |
24 | ## Out of scope
25 | Latency analytics (0.x tool), alerting by e-mail/Teams, measuring interactive sessions, Vero use in pipeline stage 4.
=== END FILE: docs/PRD.md ===
```

```
=== FILE: docs/decisions/ADR-001_stdlib-only.md (6 lines) ===
 1 | # ADR-001: Python standard library only
 2 | Status: accepted (kept from 0.x)
 3 |
 4 | Context: distributed to laptops without admin rights or PyPI access; archives and scripts are often blocked.
 5 | Decision: Python 3.8+ standard library only; dashboard is one HTML file with inline JS/CSS, no CDN.
 6 | Consequences: nothing to install or patch; dev tools (ruff, mypy) are used only by developers via `uvx`.
=== END FILE: docs/decisions/ADR-001_stdlib-only.md ===
```

=== FILE: docs/decisions/ADR-002_availability-not-latency.md (8 lines) ===

```
1 | # ADR-002: Monitor availability, not latency
2 | Status: accepted (3.0.0)
3 |
4 | Context: Vero CLI `task` latency is dominated by start-up and agent orchestration (~52 s for "Reply with OK",
5 | integration guide 3.4). Latency analytics of a cold-start agent say little; what the team needs is "is Vero usable now".
6 | Decision: the tool reports available / degraded / unavailable per run, availability %, timeline and incidents.
7 | Duration is kept only as a check attribute and as the "slow" (degraded) criterion.
8 | Consequences: simpler status page; the 0.x latency monitor is superseded.
```

=== END FILE: docs/decisions/ADR-002_availability-not-latency.md ===

=== FILE: docs/decisions/ADR-003_no-yolo-sandbox.md (9 lines) ===

```
1 | # ADR-003: No --yolo; run the task in an empty sandbox folder
2 | Status: accepted (3.0.0)
3 |
4 | Context: `vero task` starts an agent that can read/write files and run shell commands; `--yolo` auto-approves them
5 | (integration guide 3.7). The availability probe only needs `attempt_completion`.
6 | Decision: never pass `--yolo`; pass `-c data/sandbox` (an empty folder) and `-t` (timeout); kill the process tree at
7 | `-t + min(30 s, -t)`. The prompt is fixed and harmless.
8 | Consequences: if a Vero version starts asking for approval even for completion, the task check reports DOWN
9 | ("no completion"); the guide explains how to diagnose it.
```

=== END FILE: docs/decisions/ADR-003_no-yolo-sandbox.md ===

```
=== FILE: docs/decisions/ADR-004_vero-chat-handshake.md (8 lines) ===
1 | # ADR-004: Vero Chat checked by MCP handshake, token from an environment variable
2 | Status: accepted (3.0.0)
3 |
4 | Context: Vero Chat is an MCP server; a `tools/call` runs an agent that can take minutes. The `initialize` handshake
5 | proves the endpoint and the token work without any model call. Tokens must never be stored in config or Git.
6 | Decision: the optional `chat` check sends `initialize` only; the bearer token is read from the environment variable
7 | named in `chat.token_env` (default `VERO_CHAT_TOKEN`) at run time.
8 | Consequences: free and fast; it does not prove the Chat agent can answer (secondary check: its failure only degrades).
=== END FILE: docs/decisions/ADR-004_vero-chat-handshake.md ===
```

```

=== FILE: pyproject.toml (44 lines) ===
1 | [build-system]
2 | requires = ["setuptools>=61"]
3 | build-backend = "setuptools.build_meta"
4 |
5 | [project]
6 | name = "vero-availability-monitor"
7 | version = "3.0.0"
8 | description = "Is Vero available? Status dashboard for Vero CLI and Vero Chat (ASPF-1578)"
9 | requires-python = ">=3.8"
10 | dependencies = []
11 |
12 | [project.scripts]
13 | vam = "vero_monitor.cli:main"
14 |
15 | [tool.setuptools.packages.find]
16 | where = ["src"]
17 |
18 | [tool.setuptools.package-data]
19 | vero_monitor = ["web/*.html"]
20 |
21 | [tool.ruff]
22 | line-length = 130
23 | target-version = "py38"
24 | src = ["src", "tests"]
25 |
26 | [tool.ruff.lint]
27 | select = ["E", "F", "W", "I", "B", "UP", "S", "SIM", "RUF"]
28 | ignore = [
29 |     "S603", "S607",_# subprocess calls are the purpose of this tool (fixed argument lists, no shell)
30 |     "S310",_# urlopen on a configured https URL (Vero Chat)
31 |     "S104",_# 0.0.0.0 only compared, never bound by default
32 |     "UP006", "UP007", "UP035", "UP045",_# keep typing.List/Optional for Python 3.8 readability
33 |     "RUF001", "RUF003",
34 | ]
35 |
36 | [tool.ruff.lint.per-file-ignores]
37 | "tests/**" = ["S101", "S108", "S105", "S106", "S314"]_# S314: parses XML the tool itself generated
38 | "scripts/**" = ["S311"]
39 |
40 | [tool.mypy]
41 | python_version = "3.9"
42 | files = ["src"]
43 | strict = true
44 | warn_unused_ignores = true
=== END FILE: pyproject.toml ===

```

```
=== FILE: requirements.txt (1 lines) ===
```

```
1 | # No third-party dependencies: Python 3.8+ standard library only (docs/decisions/ADR-001_stdlib-only.md).
```

```
=== END FILE: requirements.txt ===
```

```
=== FILE: scripts/_py.bat (17 lines) ===
```

```
1 | @echo off
2 | rem Finds Python 3.8+ and sets PY. Called by the other .bat files.
3 | set "PY="
4 | where py >nul 2>nul && set "PY=py -3"
5 | if not defined PY (where python >nul 2>nul && set "PY=python")
6 | if not defined PY goto nopy
7 | %PY% -c "import sys; sys.exit(0 if sys.version_info >= (3, 8) else 1)" >nul 2>nul || goto nopy
8 | exit /b 0
9 | :nopy
10 | echo.
11 | echo_ Python 3.8 or newer was not found.
12 | echo_ Install it from Software Center, or python.org (tick "Add python.exe to PATH"),
13 | echo_ or run: _winget install Python.Python.3.12
14 | echo_ Then run this file again.
15 | echo.
16 | pause
17 | exit /b 1
```

```
=== END FILE: scripts/_py.bat ===
```

```
=== FILE: scripts/build_rebuild_md.py (147 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Build dist/VeroAvailabilityMonitor_<version>_REBUILD.md: the complete source as one Markdown document.
3 |
4 | For channels that accept no archives and no scripts. Every file is a readable fenced code block;
5 | an AI assistant (or a person) recreates the folder from it and checks each file with SHA-256.
6 | The HTML guide is left out (it is sent as its own file); everything else is included."""
7 |
8 | import hashlib
9 | import json
10 | import re
11 | import sys
12 | from pathlib import Path
13 |
14 | ROOT = Path(__file__).resolve().parents[1]
15 | sys.path.insert(0, str(ROOT / "src"))
16 | from vero_monitor import __version__ # noqa: E402
17 |
18 | SKIP_DIRS = {
19 |     "data",
20 |     "data-demo",
21 |     "reports",
22 |     "dist",
23 |     "__pycache__",
24 |     ".pytest_cache",
25 |     ".mypy_cache",
26 |     ".ruff_cache",
27 |     ".venv",
28 |     "venv",
29 |     ".git",
30 | }
31 | SKIP_FILES = {"config/monitor.json", "docs/VeroAvailabilityMonitor_Guide.html"}
32 | CRLF_SUFFIXES = (".bat",)
33 | CRLF_FILES = {"START_HERE.txt"}
34 | LANG = {".py": "python", ".json": "json", ".md": "markdown", ".html": "html", ".bat": "bat", ".toml": "toml", ".txt": "text"}
35 |
36 |
37 | def lf_sha(text: str) -> str:
38 |     return hashlib.sha256(text.replace("\r\n", "\n").encode("utf-8")).hexdigest()
39 |
40 |
41 | def fence_for(text: str) -> str:
42 |     longest = max((len(m) for m in re.findall(r"``+", text)), default=0)
43 |     return "``" * max(4, longest + 1)
44 |
45 |
46 | def files():
47 |     for f in sorted(ROOT.rglob("*")):
48 |         rel = f.relative_to(ROOT)
49 |         if f.is_dir() or SKIP_DIRS & set(rel.parts) or rel.as_posix() in SKIP_FILES or f.suffix in (".pyc", ".log"):
50 |             continue
51 |         yield rel.as_posix(), f.read_bytes().decode("utf-8")
52 |
53 |
54 | VERIFY = r"""import hashlib, json, pathlib, re, sys
55 | root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "vero-availability-monitor")
56 | md = pathlib.Path(sys.argv[2] if len(sys.argv) > 2 else "REBUILD.md").read_text(encoding="utf-8")
57 | manifest = json.loads(re.search(r"<!-- MANIFEST\n(?:.*)\n-->", md, re.S).group(1))
58 | bad = 0
59 | for path, info in manifest["files"].items():
60 |     p = root / path
61 |     if not p.exists():
62 |         print("MISSING ", path); bad += 1; continue
63 |     text = p.read_bytes().decode("utf-8").replace("\r\n", "\n")
64 |     if hashlib.sha256(text.encode("utf-8")).hexdigest() != info["sha256_lf"]:
65 |         print("DIFFERENT", path); bad += 1
66 |     elif info["crlf"] and b"\r\n" not in p.read_bytes():

```

```

67 | p.write_bytes(text.replace("\n", "\r\n").encode("utf-8")); print("fixed CRLF", path)
68 | print("OK: all files match" if not bad else f"{bad} file(s) wrong: recreate them from the document")
69 | sys.exit(1 if bad else 0)
70 | """
71 |
72 |
73 | def main() -> None:
74 |     items = list(files())
75 |     manifest = {
76 |         "version": __version__,
77 |         "files": {
78 |             p: {"sha256_lf": lf_sha(t), "crlf": p.endswith(CRLF_SUFFIXES) or p in CRLF_FILES, "bytes": len(t.encode("utf-8"))}
79 |             for p, t in items
80 |         },
81 |     }
82 |     out = ROOT / "dist" / f"VeroAvailabilityMonitor_{__version__}_REBUILD.md"
83 |     out.parent.mkdir(exist_ok=True)
84 |     w = []
85 |     w.append(f"""\# Vero Availability Monitor {__version__}: complete source for rebuild
86 |
87 | **What this is.** The full source of the Vero Availability Monitor (Jira ASPF-1578), a local tool that checks
88 | whether Vero (CLI, optionally Chat) is available and shows it on a status page. It is sent as a readable document because archives
89 | and scripts cannot be attached. Nothing in it runs by itself. Read it before rebuilding.
90 |
91 | - {len(items)} files, all plain text, Python 3.8+ standard library only (no downloads, no pip).
92 | - The HTML guide `VeroAvailabilityMonitor_Guide.html` is sent separately; put it in `docs/`.
93 | - `VERO_CONTEXT.md` (included below) explains the architecture, contracts and rules.
94 |
95 | ---
96 |
97 | ## Instructions for the AI assistant (Vero CLI or other)
98 |
99 | You are asked to recreate a project folder exactly from this document.
100 |
101 | 1. Create the folder `vero-availability-monitor/` in the location the user gives you (default: `C:\\Tools\\`).
102 | 2. For every section below that starts with `### FILE:`, create that file at that relative path,
103 |    with the content of the code block that follows, byte for byte:
104 |    - do not reformat, re-indent, translate, "improve", summarise or skip anything;
105 |    - copy only the lines between the opening and closing fence (the fence is 4 or more backticks);
106 |    - keep a trailing newline at the end of each file, exactly as in the block;
107 |    - encoding UTF-8 without BOM.
108 | 3. Line endings: LF for every file, except those marked `CRLF` in the table (the `.bat` files and
109 |    `START_HERE.txt`), which must use CRLF (Windows).
110 | 4. Create empty folders if needed: `data/` is not needed (the tool creates it).
111 | 5. Verify: save the Python block in the section Verify the rebuild as `verify_rebuild.py` next to this
112 |    document (outside the project folder), then run
113 |    py verify_rebuild.py C:\\Tools\\vero-availability-monitor VeroAvailabilityMonitor_{__version__}_REBUILD.md`.
114 |    It checks every file against its SHA-256 (computed on LF line endings) and fixes CRLF where needed.
115 |    If it reports `DIFFERENT` or `MISSING`, recreate those files again from this document.
116 | 6. Then run the tests: py -m unittest discover -s tests -t .` inside the folder. Expected: `OK` (64 tests, about 10 s).
117 | 7. Report the result to the user, then tell them to double-click `setup.bat` (see `START_HERE.txt`).
118 |
119 | Do not run `setup.bat`, `install`, `doctor`, `check` or the real Vero CLI yourself unless the user asks.
120 |
121 | ---
122 |
123 | ## File list
124 |
125 | | # | Path | Bytes | Line endings | SHA-256 (LF) |
126 | |---|-----|-----|:---|:---|
127 | |""")
128 | for i, (p, _text) in enumerate(items, 1):
129 |     m = manifest["files"][p]
130 |     w.append(f"| {i} | `{p}` | {m['bytes']} | {'CRLF' if m['crlf'] else 'LF'} | `{m['sha256_lf'][:16]}`... `\\n`")
131 |     w.append(f"\\n---\\n\\n## Files\\n\\n")
132 |     for p, t in items:
133 |         text = t.replace("\\r\\n", "\\n")

```

```

134 |         f = fence_for(text)
135 |         lang = LANG.get(Path(p).suffix, "") if not p.endswith(("CODEOWNERS", ".gitignore", ".gitattributes")) else "text"
136 |         body = text if text.endswith("\n") else text + "\n"
137 |         w.append(f"### FILE: `{p}`\n\n{{f}}\n\n{{lang}}\n\n{{body}}\n\n{{f}}\n\n")
138 |         w.append("---\n\n## Verify the rebuild\n\n")
139 |         w.append("Save as `verify_rebuild.py` (outside the project folder) and run it as described in step 5.\n\n")
140 |         w.append(f"````python\n\n{{VERIFY}}\n\n")
141 |         w.append("<!-- MANIFEST\n" + json.dumps(manifest, indent=1) + "\n-->\n")
142 |         out.write_text("".join(w), encoding="utf-8", newline="\n")
143 |         print(f"wrote {out} ({{len(items)}} files, {{out.stat().st_size // 1024}} KB)")
144 |
145 |
146 | if __name__ == "__main__":
147 |     main()
=== END FILE: scripts/build_rebuild_md.py ===

```

```
=== FILE: scripts/build_rebuild_pdf.py (224 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Build dist/VeroAvailabilityMonitor_<version>_Source.pdf: the complete source as a PDF document.
3 |
4 | For channels that accept only documents. Developer tool (needs reportlab):
5 | uv run --with reportlab python scripts/build_rebuild_pdf.py
6 |
7 | Listing format (survives PDF text extraction, which drops blank lines and collapses spaces):
8 | """
9 | line 12 | content"
10 | line 12 of the file; every MARK (U+2423, open box) in content is one space
11 | (used for leading spaces, runs of 2+ spaces and spaces at a row edge)
12 | line 12 | "
13 | line 12 is empty
14 | "
15 | + more"
16 | continuation of the previous line (joined with nothing in between)
17 | The MARK character never appears in the sources (checked at build time).
18 | """
19 |
20 | import hashlib
21 | import re
22 | import sys
23 | from pathlib import Path
24 |
25 | from reportlab.lib.pagesizes import A4, landscape
26 | from reportlab.pdfbase import pdfmetrics
27 | from reportlab.pdfbase.ttfonts import TTFont
28 | from reportlab.pdfgen import canvas
29 |
30 | ROOT = Path(__file__).resolve().parents[1]
31 | sys.path.insert(0, str(ROOT / "src"))
32 | from vero_monitor import __version__ # noqa: E402
33 |
34 | SKIP_DIRS = {
35 |     "data",
36 |     "data-demo",
37 |     "reports",
38 |     "dist",
39 |     "__pycache__",
40 |     ".pytest_cache",
41 |     ".mypy_cache",
42 |     ".ruff_cache",
43 |     ".venv",
44 |     "venv",
45 | }
46 |
47 | SKIP_SUFFIXES = (".pyc", ".log", ".egg-info")
48 | SKIP_FILES = {"config/monitor.json", "docs/VeroAvailabilityMonitor_Guide.html"}
49 | CRLF = (".bat",)
50 | CRLF_FILES = {"START_HERE.txt"}
51 | FONT_PATHS = ["/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", "C:/Windows/Fonts/consola.ttf"]
52 | SIZE, LEADING, MARGIN = 6.4, 7.7, 28
53 | PAGE_W, PAGE_H = landscape(A4)
54 | MARK = "\u2423" # open box, shown for a space that extraction could lose
55 |
56 | def files():
57 |     for f in sorted(ROOT.rglob("*")):
58 |         rel = f.relative_to(ROOT)
59 |         if (
60 |             f.is_dir()
61 |             or SKIP_DIRS & set(rel.parts)
62 |             or any(part.endswith(SKIP_SUFFIXES) for part in rel.parts)
63 |             or rel.as_posix() in SKIP_FILES
64 |         ):
65 |             continue
66 |         yield rel.as_posix(), f.read_bytes().decode("utf-8").replace("\r\n", "\n")
67 |
68 | def lf_sha(text: str) -> str:
69 |     return hashlib.sha256(text.encode("utf-8")).hexdigest()

```

```

67 |
68 | def protect(chunk: str) -> str:
69 |     """Mark every space an extractor could drop: leading, trailing, and runs of 2+."""
70 |     chunk = re.sub(r" {2,}", lambda m: MARK * len(m.group()), chunk)
71 |     if chunk.startswith(" "):
72 |         chunk = MARK + chunk[1:]
73 |     if chunk.endswith(" "):
74 |         chunk = chunk[:-1] + MARK
75 |     return chunk
76 |
77 |
78 | def encode(lines, width):
79 |     """Source lines -> listing rows (see module docstring)."""
80 |     rows = []
81 |     for n, line in enumerate(lines, 1):
82 |         head = f"{n:>4} |"
83 |         if not line:
84 |             rows.append(head)
85 |             continue
86 |         chunks = [line[i : i + width] for i in range(0, len(line), width)]
87 |         rows.append(f"{head} {protect(chunks[0])}")
88 |         rows.extend(f"      {protect(c)}" for c in chunks[1:])
89 |     return rows
90 |
91 |
92 | VERIFY_TEMPLATE = """import hashlib, pathlib, sys
93 | # usage: py verify_rebuild.py C:\\Tools\\vero-availability-monitor
94 | root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "vero-availability-monitor")
95 | CRLF = {crlf!r}
96 | FILES = {{
97 |     {entries}}}
98 | bad = 0
99 | for path, sha in FILES.items():
100 |     p = root / path
101 |     if not p.exists():
102 |         print("MISSING", path); bad += 1; continue
103 |     text = p.read_bytes().decode("utf-8").replace("\\r\\n", "\\n")
104 |     if hashlib.sha256(text.encode("utf-8")).hexdigest() != sha:
105 |         print("DIFFERENT", path); bad += 1; continue
106 |     want = text.replace("\\n", "\\r\\n") if path in CRLF else text
107 |     if p.read_bytes() != want.encode("utf-8"):
108 |         p.write_bytes(want.encode("utf-8")); print("fixed line endings", path)
109 |     print("OK: all files match" if not bad else str(bad) + " file(s) wrong: recreate them from the document")
110 |     sys.exit(1 if bad else 0)
111 | """
112 |
113 |
114 | class Doc:
115 |     def __init__(self, path: Path, font: str):
116 |         self.c = canvas.Canvas(str(path), pagesize=(PAGE_W, PAGE_H))
117 |         self.c.setTitle(f"Vero Availability Monitor {__version__} - complete source for rebuild")
118 |         self.c.setAuthor("Quality AI Automation (ASPF-1578)")
119 |         self.c.setSubject("Source listing. A document only: nothing in it runs.")
120 |         self.font = font
121 |         self.page = 0
122 |         self.y = 0.0
123 |         self.new_page()
124 |
125 |     def new_page(self):
126 |         if self.page:
127 |             self.c.showPage()
128 |             self.page += 1
129 |             self.c.setFont(self.font, 6)
130 |             self.c.setFillGray(0.45)
131 |             self.c.drawString(MARGIN, PAGE_H - 16, f"Vero Availability Monitor {__version__} · source for rebuild · ASPF-1578")
132 |             self.c.drawRightString(PAGE_W - MARGIN, 12, f"page {self.page}")
133 |             self.c.setFillGray(0)

```

```

134 |         self.y = PAGE_H - MARGIN - 8
135 |
136 |     def row(self, text: str, size: float = SIZE, gray: float = 0):
137 |         if self.y < MARGIN:
138 |             self.new_page()
139 |             self.c.setFont(self.font, size)
140 |             self.c.setFillGray(gray)
141 |             self.c.drawString(MARGIN, self.y, text)
142 |             self.y -= max(LEADING, size * 1.25)
143 |
144 |     def gap(self, n: float = 1):
145 |         self.y -= LEADING * n
146 |
147 |     def save(self):
148 |         self.c.save()
149 |
150 |
151 | def main() -> None:
152 |     font_path = next((p for p in FONT_PATHS if Path(p).exists()), None)
153 |     if not font_path:
154 |         sys.exit("no monospace TTF font found (DejaVu Sans Mono or Consolas)")
155 |     pdfmetrics.registerFont(TTFont("Mono", font_path))
156 |     char_w = pdfmetrics.stringWidth("M", "Mono", SIZE)
157 |     width = int((PAGE_W - 2 * MARGIN) / char_w) - 8 # minus the "NNNN | " gutter
158 |     items = list(files())
159 |     bad = [p for p, t in items if MARK in t]
160 |     if bad:
161 |         sys.exit(f"MARK character found in {bad}: the listing would be ambiguous")
162 |     crlf = sorted(p for p, _ in items if p.endswith(CRLF) or p in CRLF_FILES)
163 |     out = ROOT / "dist" / f"VeroAvailabilityMonitor_{__version__}_Source.pdf"
164 |     out.parent.mkdir(exist_ok=True)
165 |     d = Doc(out, "Mono")
166 |
167 |     d.row(f"VERO AVAILABILITY MONITOR {__version__} - COMPLETE SOURCE FOR REBUILD", 13)
168 |     d.row("Jira ASPF-1578 · Quality AI Automation · a document only: nothing in it runs", 8, 0.35)
169 |     d.gap()
170 |     intro = f"""\
171 | What this is: the full source ({len(items)} text files) of a local tool that checks whether Vero (CLI, optionally Chat) is
172 | available and shows it on a status page. Python 3.8+ standard library only. The HTML/PDF guide is sent separately.
173 |
174 | INSTRUCTIONS FOR THE AI ASSISTANT (Vero CLI or other) - recreate the project folder EXACTLY:
175 | 1. Create the folder vero-availability-monitor in the location the user gives (default C:\\Tools\\).
176 | 2. Every file starts with a row_{n}=== FILE: <path> (<n> lines) === and ends with_{n}=== END FILE: <path> ===.
177 | 3. Rows between them encode the file, one source line per numbered row:
178 |    _{n}12 | content_{n}-> line 12. Every {MARK} (open box) in the content is ONE SPACE. It marks leading
179 |    spaces, runs of 2+ spaces and spaces at a row edge; single spaces between words
180 |    are printed normally. The {MARK} character never occurs in the real files.
181 |    _{n}12 | " _{n}-> line 12 is an empty line.
182 |    " _{n}+ more_{n}-> continuation of the previous line: append "more" directly (no space, no newline).
183 |    Ignore page headers/footers ("Vero Availability Monitor ... source for rebuild", "page N").
184 |    Line numbers must run 1..n without gaps; n is in the FILE row. Every file ends with a newline.
185 | 4. Write files as UTF-8 without BOM, LF line endings. Do not reformat, translate, summarise or "fix" anything.
186 | 5. Create verify_rebuild.py from the section VERIFY SCRIPT at the end (same row rules), OUTSIDE the project
187 |    folder, and run:_{n}py verify_rebuild.py C:\\Tools\\vero-availability-monitor
188 |    It checks every file's SHA-256 and sets CRLF line endings where Windows needs them (.bat, START_HERE.txt).
189 |    Recreate any file reported MISSING or DIFFERENT, then run it again until it prints "OK: all files match".
190 | 6. Run the tests inside the folder:_{n}py -m unittest discover -s tests -t _{n}Expected: OK (64 tests).
191 | 7. Tell the user it is ready and to double-click setup.bat (see START_HERE.txt).
192 | Do not run setup.bat, install, doctor, check or the real Vero CLI unless the user asks.
193 | """
194 |     for line in intro.splitlines():
195 |         d.row(line, 7.2)
196 |     d.gap()
197 |     d.row("FILE LIST (path · lines · line endings · SHA-256 of the content with LF endings)", 7.2)
198 |     for p, t in items:
199 |         lines = t.split("\n")[:-1] if t.endswith("\n") else t.split("\n")
200 |         d.row(f'_{n}{p[:52]} {len(lines):>5}_{n}' + ('CRLF' if p in crlf else 'LF_{n}') + '{lf_sha(t)}', 6.2)

```

```

201 |
202 |     for p, t in items:
203 |         lines = t.split("\n")[:-1] if t.endswith("\n") else t.split("\n")
204 |         d.new_page()
205 |         d.row(f"=== FILE: {p} ({len(lines)} lines) ===", 7.5)
206 |         for r in encode(lines, width):
207 |             d.row(r)
208 |         d.row(f"=== END FILE: {p} ===", 7.5)
209 |
210 |     entries = "".join(f"{p!r}: {lf_sha(t)!r},\n" for p, t in items)
211 |     verify = VERIFY_TEMPLATE.format(crlf=crlf, entries=entries)
212 |     d.new_page()
213 |     vlines = verify.split("\n")[:-1]
214 |     d.row(f"=== VERIFY SCRIPT: verify_rebuild.py ({len(vlines)} lines) ===", 7.5)
215 |     for r in encode(vlines, width):
216 |         d.row(r)
217 |     d.row("=== END VERIFY SCRIPT ===", 7.5)
218 |     d.save()
219 |     (ROOT / "dist" / "verify_rebuild.py").write_text(verify, encoding="utf-8")
220 |     print(f"wrote {out} ({len(items)} files, {d.page} pages, {out.stat().st_size // 1024} KB, {width} chars per row)")
221 |
222 |
223 | if __name__ == "__main__":
224 |     main()
=== END FILE: scripts/build_rebuild_pdf.py ===

```

```
=== FILE: scripts/fake_vero.py (76 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Stand-in for the Vero CLI (demo and tests). Emulates `vero version` and
3 | `vero task --json -m <model> -t <s> -c <dir> <prompt>` with the event stream of Vero CLI 2.3.x.
4 |
5 | FAKE_VERO_MODE: ok | slow | down | auth | nocompletion | refuse | hang | random (default)
6 | FAKE_VERO_DELAY: seconds before the answer (default: about 1-3 s)
7 | """
8 |
9 | import json
10 | import os
11 | import random
12 | import sys
13 | import time
14 |
15 | MODEL = {"providerId": "bedrock", "modelId": "us.anthropic.claude-haiku-4-5-20251001-v1:0", "mode": "act"}
16 |
17 |
18 | def emit(obj):
19 |     print(json.dumps(obj), flush=True)
20 |
21 |
22 | def main(argv):
23 |     if not argv or argv[0] in ("version", "--version", "-V"):
24 |         print("vero 2.3.3 (fake0000)")
25 |         return 0
26 |     if argv[0] not in ("task", "t"):
27 |         print(f"unknown command {argv[0]}", file=sys.stderr)
28 |         return 2
29 |     args = argv[1:]
30 |     model = MODEL["modelId"]
31 |     if "-m" in args:
32 |         model = args[args.index("-m") + 1]
33 |     prompt = args[-1] if args else ""
34 |     mode = os.environ.get("FAKE_VERO_MODE", "random")
35 |     if mode == "random":
36 |         r = random.random()
37 |         mode = "down" if r < 0.03 else "nocompletion" if r < 0.05 else "slow" if r < 0.09 else "ok"
38 |         delay = float(os.environ.get("FAKE_VERO_DELAY", random.uniform(1.0, 3.0)))
39 |         info = dict(MODEL, modelId=model)
40 |         ts = int(time.time() * 1000)
41 |         if "--json" in args:
42 |             emit({"type": "task_started", "taskId": str(ts)})
43 |             emit({"ts": ts, "type": "say", "say": "task", "text": prompt, "modelInfo": info})
44 |             # the real CLI puts the full upstream prompt here; the monitor must never keep it
45 |             emit(
46 |                 {
47 |                     "ts": ts,
48 |                     "type": "say",
49 |                     "say": "api_req_started",
50 |                     "text": json.dumps({"request": "SYSTEM PROMPT ... FAKE-SECRET-api_req_started ..."}),
51 |                     "modelInfo": info,
52 |                 }
53 |             )
54 |         if mode == "hang":
55 |             time.sleep(3600)
56 |         if mode == "auth":
57 |             print("error: Unauthorized (token=abc123secret) - run vero auth", file=sys.stderr)
58 |             return 1
59 |         if mode == "down":
60 |             time.sleep(min(delay, 1))
61 |             print("error: provider unavailable: connect ETIMEDOUT bedrock-runtime.us-west-2", file=sys.stderr)
62 |             return 1
63 |         time.sleep(delay * (6 if mode == "slow" else 1))
64 |         answer = "I cannot help with that." if mode == "refuse" else "OK"
65 |         if "--json" in args:
66 |             emit({"ts": ts, "type": "say", "say": "text", "text": answer[:1], "partial": True, "modelInfo": info})

```

```
67 |         emit({"ts": ts, "type": "say", "say": "text", "text": answer, "partial": False, "modelInfo": info})
68 |         if mode != "nocompletion":
69 |             emit({"ts": ts, "type": "say", "say": "completion_result", "text": answer, "modelInfo": info})
70 |         else:
71 |             print(answer)
72 |         return 0
73 |
74 |
75 | if __name__ == "__main__":
76 |     sys.exit(main(sys.argv[1:]))
=== END FILE: scripts/fake_vero.py ===
```

```
=== FILE: scripts/seed_demo_data.py (59 lines) ===
```

```

1 | #!/usr/bin/env python3
2 | """Fill data-demo/ with 7 days of synthetic checks (with a few incidents) so the status page
3 | can be shown before Vero is connected. Refuses any data folder other than data-demo."""
4 |
5 | import datetime as dt
6 | import random
7 | import sys
8 | from pathlib import Path
9 |
10 | ROOT = Path(__file__).resolve().parents[1]
11 | sys.path.insert(0, str(ROOT / "src"))
12 |
13 | from vero_monitor import config_# noqa: E402
14 | from vero_monitor.domain import Check, CheckResult, RunResult, Status, overall_status_# noqa: E402
15 | from vero_monitor.runner import iso_# noqa: E402
16 | from vero_monitor.store import Store_# noqa: E402
17 |
18 | cfg = config.load()
19 | if not cfg.data_dir.name.startswith("data-demo"):
20 |     sys.exit(f"refusing to seed demo data into {cfg.data_dir}: run_vam init --demo_first")
21 | store = Store(cfg.data_dir)
22 | rnd = random.Random(1578)
23 | now = dt.datetime.now(dt.timezone.utc).replace(second=0, microsecond=0)
24 | t = now - dt.timedelta(days=7)
25 | outages = [
26 |     (now - dt.timedelta(days=5, hours=3), 70),
27 |     (now - dt.timedelta(days=2, hours=9), 35),
28 |     (now - dt.timedelta(hours=20), 25),
29 | ]
30 | n = 0
31 | while t < now:
32 |     down = any(s <= t < s + dt.timedelta(minutes=m) for s, m in outages)
33 |     task_s = rnd.lognormvariate(0, 0.12) * 52
34 |     checks = [CheckResult(Check.CLI, Status.UP, rnd.uniform(800, 1500), version="vero 2.3.3 (demo)")]
35 |     if down:
36 |         checks.append(
37 |             CheckResult(
38 |                 Check.TASK, Status.DOWN, 21000.0, "no completion within 210 s", provider_id="bedrock", model_id=cfg.vero.model
39 |             )
40 |         )
41 |     elif rnd.random() < 0.04:
42 |         checks.append(
43 |             CheckResult(
44 |                 Check.TASK,
45 |                 Status.DEGRADED,
46 |                 task_s * 2000,
47 |                 f"slow: {task_s * 2:.0f} s > 90 s",
48 |                 provider_id="bedrock",
49 |                 model_id=cfg.vero.model,
50 |             )
51 |         )
52 |     else:
53 |         checks.append(CheckResult(Check.TASK, Status.UP, task_s * 1000, provider_id="bedrock", model_id=cfg.vero.model))
54 |     run_id = f"{t:%Y%m%d-%H%M}-vero-availability-demo"
55 |     if not store.run_exists(run_id):
56 |         store.save(RunResult(run_id, iso(t), t.timestamp(), "demo", tuple(checks), overall_status(checks)), "demo", "demo")
57 |         n += 1
58 |         t += dt.timedelta(minutes=15)
59 |     print(f"seeded {n} demo runs into {store.path}")

```

```
=== END FILE: scripts/seed_demo_data.py ===
```

```
=== FILE: setup.bat (34 lines) ===
```

```

1 | @echo off
2 | rem Guided first-time setup: configure -> test -> install background monitoring.
3 | setlocal
4 | cd /d "%~dp0"
5 | title Vero availability monitor - setup
6 | call scripts\_py.bat || exit /b 1
7 | echo.
8 | echo_%%Vero availability monitor 3_%%(Python: %PY%)
9 | echo_%%-----
10 | :configure
11 | %PY% vam.py configure
12 | if errorlevel 1 goto again
13 | echo.
14 | echo_%%Testing Vero once (nothing stored). The task check can take about a minute...
15 | %PY% vam.py doctor
16 | if errorlevel 1 goto again
17 | echo.
18 | choice /C YN /M " Install background monitoring (check on a timer + status page at every logon)"
19 | if errorlevel 2 goto manual
20 | %PY% vam.py install
21 | %PY% vam.py open
22 | goto end
23 | :manual
24 | echo.
25 | echo_%%Not installed. Use start_dashboard.bat (it also checks while its window is open).
26 | goto end
27 | :again
28 | echo.
29 | choice /C YN /M " Not ready. Run the configuration again"
30 | if errorlevel 2 goto end
31 | goto configure
32 | :end
33 | echo.
34 | pause

```

```
=== END FILE: setup.bat ===
```

```
=== FILE: setup_demo.bat (12 lines) ===
1 | @echo off
2 | rem Demo: fake Vero CLI + 7 days of sample data in data-demo\ (separate from real data).
3 | setlocal
4 | cd /d "%~dp0"
5 | call scripts\_py.bat || exit /b 1
6 | %PY% vam.py init --demo --force || goto end
7 | %PY% scripts\seed_demo_data.py || goto end
8 | echo.
9 | echo_ Demo ready. The status page opens now; close this window to stop it.
10 | %PY% vam.py serve --open
11 | :end
12 | pause
=== END FILE: setup_demo.bat ===
```

```
=== FILE: src/vero_monitor/__init__.py (4 lines) ===
 1 | """Vero availability monitor (ASPF-1578): is Vero CLI / Vero Chat available right now?"""
 2 |
 3 | __version__ = "3.0.0"
 4 | COMPONENT = "vero-availability"
=== END FILE: src/vero_monitor/__init__.py ===
```

```
=== FILE: src/vero_monitor/__main__.py (5 lines) ===
 1 | import sys
 2 |
 3 | from .cli import main
 4 |
 5 | sys.exit(main())
=== END FILE: src/vero_monitor/__main__.py ===
```

```

=== FILE: src/vero_monitor/checks/__init__.py (46 lines) ===
1 | """Check registry: which checks run, in which order. Cheap checks first."""
2 |
3 | from __future__ import annotations
4 |
5 | import logging
6 | import time
7 | from typing import Callable, List, Tuple
8 |
9 | from ..config import Config
10 | from ..domain import Check, CheckResult, Status
11 | from .vero_chat import check_chat
12 | from .vero_cli import check_cli, check_task
13 |
14 | log = logging.getLogger("vero_monitor")
15 |
16 | CheckFn = Callable[[Config], CheckResult]
17 |
18 |
19 | def enabled(cfg: Config) -> List[Tuple[Check, CheckFn]]:
20 |     plan: List[Tuple[Check, CheckFn]] = []
21 |     if cfg.check_cli:
22 |         plan.append((Check.CLI, check_cli))
23 |     if cfg.check_chat:
24 |         plan.append((Check.CHAT, check_chat))
25 |     if cfg.check_task:
26 |         plan.append((Check.TASK, check_task))
27 |     return plan
28 |
29 |
30 | def run_all(cfg: Config) -> List[CheckResult]:
31 |     results: List[CheckResult] = []
32 |     cli_down = False
33 |     for name, fn in enabled(cfg):
34 |         if name is Check.TASK and cli_down:
35 |             # no point spending a minute on a task when the CLI cannot even start
36 |             results.append(CheckResult(Check.TASK, Status.DOWN, None, "skipped: Vero CLI not available"))
37 |             continue
38 |             t0 = time.perf_counter()
39 |             try:
40 |                 res = fn(cfg)
41 |             except Exception as e: # a bug in one check must not hide the others
42 |                 log.exception("check %s crashed", name.value)
43 |                 res = CheckResult(name, Status.DOWN, (time.perf_counter() - t0) * 1000, f"monitor error: {type(e).__name__}")
44 |                 results.append(res)
45 |                 cli_down = cli_down or (name is Check.CLI and res.status is Status.DOWN)
46 |     return results
=== END FILE: src/vero_monitor/checks/__init__.py ===

```

```
=== FILE: src/vero_monitor/checks/process.py (100 lines) ===
```

```
1 | """Run one external process with a hard timeout. Kills the whole process tree on timeout
2 | (vero.cmd -> node -> ...), never opens a console window on Windows."""
3 |
4 | from __future__ import annotations
5 |
6 | import contextlib
7 | import os
8 | import shutil
9 | import signal
10 | import subprocess
11 | import sys
12 | import threading
13 | import time
14 | from dataclasses import dataclass
15 | from typing import List, Mapping, Optional, Sequence
16 |
17 | IS_WINDOWS = os.name == "nt"
18 | NO_WINDOW = 0x08000000 if IS_WINDOWS else 0_0_# CREATE_NO_WINDOW
19 |
20 |
21 | @dataclass(frozen=True)
22 | class ProcessResult:
23 |     """started: bool
24 |     """exit_code: Optional[int]
25 |     """stdout: str
26 |     """stderr: str
27 |     """duration_ms: float
28 |     """timed_out: bool
29 |
30 |
31 | def resolve(program: str) -> Optional[str]:
32 |     """Full path of the program, or None. Finds vero.cmd / vero.exe for 'vero' on Windows."""
33 |     return shutil.which(program)
34 |
35 |
36 | def kill_tree(proc: subprocess.Popen[bytes]) -> None:
37 |     """Kill the process and everything it started (vero.cmd -> node -> ...)."""
38 |     with contextlib.suppress(NotFoundError):
39 |         if sys.platform == "win32":
40 |             subprocess.run(
41 |                 ["taskkill", "/T", "/F", "/PID", str(proc.pid)], capture_output=True, creationflags=NO_WINDOW, check=False
42 |             )
43 |         else:
44 |             os.killpg(proc.pid, signal.SIGKILL)_0_# own process group: start_new_session=True
45 |     with contextlib.suppress(NotFoundError):
46 |         proc.kill()
47 |
48 |
49 | def run(
50 |     args: Sequence[str], timeout_s: float, env: Optional[Mapping[str, str]] = None, cwd: Optional[str] = None
51 | ) -> ProcessResult:
52 |     full_env = dict(os.environ)
53 |     full_env.update(env or {})
54 |     t0 = time.perf_counter()
55 |     try:
56 |         proc = subprocess.Popen(
57 |             list(args),
58 |             stdin=subprocess.DEVNULL,
59 |             stdout=subprocess.PIPE,
60 |             stderr=subprocess.PIPE,
61 |             env=full_env,
62 |             cwd=cwd,
63 |             creationflags=NO_WINDOW,
64 |             start_new_session=not IS_WINDOWS,
65 |         )
66 |     except OSError as e:
```

```

67 |         return ProcessResult(False, None, "", f"cannot start {args[0]!r}: {e}", 0.0, False)
68 |
69 |         out: List[bytes] = []
70 |         err: List[bytes] = []
71 |         stdout, stderr = proc.stdout, proc.stderr
72 |         if stdout is None or stderr is None: # cannot happen with PIPE; keeps the types honest
73 |             kill_tree(proc)
74 |             return ProcessResult(False, None, "", "no output pipes", 0.0, False)
75 |         readers = [
76 |             threading.Thread(target=lambda: out.append(stdout.read()), daemon=True),
77 |             threading.Thread(target=lambda: err.append(stderr.read()), daemon=True),
78 |         ]
79 |         for r in readers:
80 |             r.start()
81 |         timed_out = False
82 |         try:
83 |             proc.wait(timeout=timeout_s)
84 |         except subprocess.TimeoutExpired:
85 |             timed_out = True
86 |             kill_tree(proc)
87 |             proc.wait()
88 |         duration = (time.perf_counter() - t0) * 1000
89 |         for r in readers:
90 |             r.join(timeout=5)
91 |         stdout.close() # release OS handles (long-running dashboard on Windows)
92 |         stderr.close()
93 |         return ProcessResult(
94 |             started=True,
95 |             exit_code=proc.returncode,
96 |             stdout=b"".join(out).decode("utf-8", "replace"),
97 |             stderr=b"".join(err).decode("utf-8", "replace"),
98 |             duration_ms=duration,
99 |             timed_out=timed_out,
100 |         )
=== END FILE: src/vero_monitor/checks/process.py ===

```

```
=== FILE: src/vero_monitor/checks/vero_chat.py (90 lines) ===
```

```
1 | """Vero Chat check: MCP `initialize` handshake over streamable HTTP (integration guide 2.1).
2 |
3 | No tool is called, so no model runs and nothing is billed. The reply may be plain JSON or
4 | Server-Sent Events (`data:` lines). The bearer token comes from an environment variable;
5 | it is never stored, logged or shown.
6 | """
7 |
8 | from __future__ import annotations
9 |
10 | import json
11 | import os
12 | import ssl
13 | import time
14 | import urllib.error
15 | import urllib.request
16 | from typing import Any, Dict, Optional
17 |
18 | from .. import __version__
19 | from ..config import Config
20 | from ..domain import Check, CheckResult, Status
21 | from .vero_cli import clean
22 |
23 | INITIALIZE = {
24 |     "jsonrpc": "2.0",
25 |     "id": 1,
26 |     "method": "initialize",
27 |     "params": {
28 |         "protocolVersion": "2024-11-05",
29 |         "capabilities": {},
30 |         "clientInfo": {"name": "vero-availability-monitor", "version": __version__},
31 |     },
32 | }
33 |
34 |
35 | def parse_reply(body: str) -> Optional[Dict[str, Any]]:
36 |     """Return the JSON-RPC message from a JSON or SSE body, or None."""
37 |     candidates = [body.strip()]
38 |     candidates += [ln[5:].strip() for ln in body.splitlines() if ln.startswith("data:")]
39 |     for c in candidates:
40 |         if not c.startswith("{"):
41 |             continue
42 |         try:
43 |             msg = json.loads(c)
44 |         except json.JSONDecodeError:
45 |             continue
46 |         if isinstance(msg, dict) and ("result" in msg or "error" in msg):
47 |             return msg
48 |     return None
49 |
50 |
51 | def check_chat(cfg: Config) -> CheckResult:
52 |     token = os.environ.get(cfg.chat.token_env, "").strip()
53 |     if not token:
54 |         return CheckResult(
55 |             Check.CHAT, Status.DOWN, None, f"environment variable {cfg.chat.token_env} is not set (see guide: Vero Chat token)"
56 |         )
57 |     req = urllib.request.Request(
58 |         cfg.chat.url,
59 |         data=json.dumps(INITIALIZE).encode(),
60 |         method="POST",
61 |         headers={
62 |             "Authorization": f"Bearer {token}",
63 |             "Content-Type": "application/json",
64 |             "Accept": "application/json, text/event-stream",
65 |         },
66 |     )
```

```

67 |     ctx = ssl.create_default_context(cafile=cfg.chat.ca_bundle or None)
68 |     t0 = time.perf_counter()
69 |     try:
70 |         with urllib.request.urlopen(req, timeout=cfg.chat.timeout_s, context=ctx) as r:
71 |             body = r.read(256 * 1024).decode("utf-8", "replace")
72 |     except urllib.error.HTTPError as e:
73 |         hint = " (token rejected or expired)" if e.code in (401, 403) else ""
74 |         return CheckResult(Check.CHAT, Status.DOWN, (time.perf_counter() - t0) * 1000, f"HTTP {e.code}{hint}")
75 |     except (urllib.error.URLError, OSError, ValueError) as e:
76 |         reason = getattr(e, "reason", e)
77 |         return CheckResult(Check.CHAT, Status.DOWN, (time.perf_counter() - t0) * 1000, clean(f"unreachable: {reason}"))
78 |     ms = (time.perf_counter() - t0) * 1000
79 |     msg = parse_reply(body)
80 |     if msg is None:
81 |         return CheckResult(Check.CHAT, Status.DOWN, ms, "reply is not an MCP JSON-RPC message")
82 |     if "error" in msg:
83 |         err = msg["error"] if isinstance(msg["error"], dict) else {}
84 |         return CheckResult(Check.CHAT, Status.DOWN, ms, clean(f"MCP error: {err.get('message', msg['error'])}"))
85 |     info = (msg.get("result") or {}).get("serverInfo") or {}
86 |     version = clean(f"{info.get('name', '?')} {info.get('version', '')}".strip())
87 |     slow = ms > cfg.chat_slow_s * 1000
88 |     return CheckResult(
89 |         Check.CHAT, Status.DEGRADED if slow else Status.UP, ms, f"slow: {ms / 1000:.1f} s" if slow else "", version=version
90 |     )
=== END FILE: src/vero_monitor/checks/vero_chat.py ===

```

```
=== FILE: src/vero_monitor/checks/vero_cli.py (137 lines) ===
```

```

1 | """Vero CLI checks.
2 |
3 | cli_░: `vero version`░░░░░░░░░░-> installed and starts. Free, a few seconds.
4 | task : `vero task --json ...`░░-> end-to-end: the agent reaches the model and completes.
5 |
6 | Contract of the --json stream (Vero CLI 2.3.x, see the integration guide section 3.3):
7 | one JSON object per line; the answer is ONLY the `say == "completion_result"` event; `partial: true`
8 | fragments are ignored; `modelInfo` gives the real provider/model; `task_started` gives the taskId.
9 | `api_req_started.text` holds the full upstream prompt and is never kept or logged.
10 | """
11 |
12 | from __future__ import annotations
13 |
14 | import json
15 | import re
16 | from dataclasses import dataclass
17 | from pathlib import Path
18 | from typing import Iterable, Optional
19 |
20 | from ..config import Config
21 | from ..domain import Check, CheckResult, Status
22 | from . import process
23 |
24 | DETAIL_MAX = 200
25 | _SECRETISH = re.compile(r"(?i)(bearer\s+\S+|token[=:]?\s*\S+|aws_secret\S*|password[=:]?\s*\S+)")
26 |
27 |
28 | def clean(text: str) -> str:
29 |     """One short line, with anything that looks like a credential masked."""
30 |     line = " ".join(text.split())
31 |     return _SECRETISH.sub("[masked]", line)[:DETAIL_MAX]
32 |
33 |
34 | @dataclass(frozen=True)
35 | class TaskStream:
36 |     task_id: Optional[str]
37 |     completion: Optional[str]
38 |     provider_id: Optional[str]
39 |     model_id: Optional[str]
40 |     error: Optional[str]
41 |     events: int
42 |
43 |
44 | def parse_stream(lines: Iterable[str]) -> TaskStream:
45 |     task_id = completion = provider = model = error = None
46 |     events = 0
47 |     for raw in lines:
48 |         raw = raw.strip()
49 |         if not raw.startswith("{"):
50 |             continue
51 |         try:
52 |             ev = json.loads(raw)
53 |             except json.JSONDecodeError:
54 |                 continue
55 |             if not isinstance(ev, dict):
56 |                 continue
57 |             events += 1
58 |             if ev.get("type") == "task_started" and ev.get("taskId") is not None:
59 |                 task_id = str(ev["taskId"])
60 |                 info = ev.get("modelInfo")
61 |                 if isinstance(info, dict):
62 |                     provider = info.get("providerId") or provider
63 |                     model = info.get("modelId") or model
64 |                 if ev.get("partial") is True:
65 |                     continue
66 |                 say = ev.get("say")

```

```

67 |         if say == "completion_result":
68 |             completion = str(ev.get("text") or "")
69 |         elif say in ("error", "api_req_failed") or ev.get("type") == "error":
70 |             error = clean(str(ev.get("text") or ev.get("message") or say))
71 |         return TaskStream(task_id, completion, provider, model, error, events)
72 |
73 |
74 |     def _program(cfg: Config) -> Optional[str]:
75 |         return process.resolve(cfg.vero.command)
76 |
77 |
78 |     def check_cli(cfg: Config) -> CheckResult:
79 |         exe = _program(cfg)
80 |         if not exe:
81 |             return CheckResult(Check.CLI, Status.DOWN, None, f"'{cfg.vero.command}' not found on PATH")
82 |         r = process.run([exe, *cfg.vero.version_args], 30, dict(cfg.vero.env))
83 |         if r.timed_out:
84 |             return CheckResult(Check.CLI, Status.DOWN, r.duration_ms, "version command timed out")
85 |         if r.exit_code != 0:
86 |             return CheckResult(Check.CLI, Status.DOWN, r.duration_ms, f"exit {r.exit_code}: {clean(r.stderr or r.stdout)}")
87 |         first = (r.stdout.strip().splitlines() or [""])[0]
88 |         return CheckResult(Check.CLI, Status.UP, r.duration_ms, "", version=clean(first)[:80])
89 |
90 |
91 |     def task_command(cfg: Config, exe: str, workdir: Path) -> list[str]:
92 |         fill = {
93 |             "{model}": cfg.vero.model,
94 |             "{timeout}": str(int(cfg.vero.task_timeout_s)),
95 |             "{workdir}": str(workdir),
96 |             "{prompt}": cfg.vero.prompt,
97 |         }
98 |         args = [exe]
99 |         for a in cfg.vero.task_args:
100 |             for k, v in fill.items():
101 |                 a = a.replace(k, v)
102 |             args.append(a)
103 |         return args
104 |
105 |
106 |     def hard_limit_s(cfg: Config) -> float:
107 |         """Our own kill deadline: Vero's -t plus a grace period (min(30 s, -t)) in case the CLI ignores -t."""
108 |         return cfg.vero.task_timeout_s + min(30.0, cfg.vero.task_timeout_s)
109 |
110 |
111 |     def check_task(cfg: Config) -> CheckResult:
112 |         exe = _program(cfg)
113 |         if not exe:
114 |             return CheckResult(Check.TASK, Status.DOWN, None, f"'{cfg.vero.command}' not found on PATH")
115 |         workdir = cfg.data_dir / "sandbox"  # empty folder: the agent has nothing to read or change
116 |         workdir.mkdir(parents=True, exist_ok=True)
117 |         limit = hard_limit_s(cfg)
118 |         r = process.run(task_command(cfg, exe, workdir), limit, dict(cfg.vero.env), str(workdir))
119 |         if not r.started:
120 |             return CheckResult(Check.TASK, Status.DOWN, None, clean(r.stderr))
121 |         s = parse_stream(r.stdout.splitlines())
122 |
123 |         def result(status: Status, detail: str) -> CheckResult:
124 |             return CheckResult(Check.TASK, status, r.duration_ms, clean(detail), provider_id=s.provider_id, model_id=s.model_id)
125 |
126 |         if r.timed_out:
127 |             return result(Status.DOWN, f"no completion within {limit:g} s")
128 |         if s.completion is None:
129 |             why = s.error or (f"exit {r.exit_code}: {r.stderr}" if r.exit_code else "")
130 |             why = why or ("no JSON events: is --json in vero.task_args?" if s.events == 0 else "no completion_result")
131 |             return result(Status.DOWN, why)
132 |         notes = []
133 |         if cfg.vero.expect and cfg.vero.expect.lower() not in s.completion.lower():

```

```
134 |         notes.append(f"answered, but not with {cfg.vero.expect!r} ({len(s.completion)} chars)")
135 |         if r.duration_ms > cfg.task_slow_s * 1000:
136 |             notes.append(f"slow: {r.duration_ms / 1000:.0f} s > {cfg.task_slow_s:g} s")
137 |         return result(Status.DEGRADED if notes else Status.UP, "; ".join(notes))
=== END FILE: src/vero_monitor/checks/vero_cli.py ===
```

```
=== FILE: src/vero_monitor/cli.py (317 lines) ===
```

```

1 | """Command line: python vam.py <command>. Run with -h for help."""
2 |
3 | from __future__ import annotations
4 |
5 | import argparse
6 | import csv
7 | import logging
8 | import logging.handlers
9 | import os
10 | import shutil
11 | import sys
12 | import time
13 | import webbrowser
14 | from pathlib import Path
15 | from typing import Callable, Dict, List, Optional
16 |
17 | from . import __version__, config, scheduler
18 | from .checks import run_all
19 | from .checks.vero_cli import task_command
20 | from .config import ROOT, Config, ConfigError
21 | from .domain import Status, overall_status
22 | from .report import static_html
23 | from .runner import in_progress, run_once
24 | from .server import already_running, serve
25 | from .store import CSV_HEADER, Store
26 |
27 | log = logging.getLogger("vero_monitor")
28 | FAKE = ROOT / "scripts" / "fake_vero.py"
29 | ICON = {"up": "UP", "degraded": "WARN", "down": "DOWN", "unknown": "?"}
30 |
31 |
32 | def setup_logging(data_dir: Path, verbose: bool) -> None:
33 |     data_dir.mkdir(parents=True, exist_ok=True)
34 |     handlers: List[logging.Handler] = [
35 |         logging.handlers.RotatingFileHandler(data_dir / "monitor.log", maxBytes=1_000_000, backupCount=3, encoding="utf-8")
36 |     ]
37 |     if sys.stdout is not None: # None under pythonw (Task Scheduler)
38 |         handlers.append(logging.StreamHandler(sys.stdout))
39 |     logging.basicConfig(
40 |         level=logging.DEBUG if verbose else logging.INFO,
41 |         handlers=handlers,
42 |         format="%(asctime)s %(levelname)s %(message)s",
43 |         force=True,
44 |     )
45 |
46 |
47 | def ask(question: str, default: str) -> str:
48 |     try:
49 |         answer = input(f"{question} [{default}]: ").strip()
50 |     except EOFError:
51 |         answer = ""
52 |     return answer or default
53 |
54 |
55 | def yes(question: str, default: bool) -> bool:
56 |     return ask(question + " (y/n)", "y" if default else "n").lower().startswith("y")
57 |
58 |
59 | # commands without a loaded config -----
60 |
61 |
62 | def cmd_init(a: argparse.Namespace) -> int:
63 |     target = Path(a.config) if a.config else config.CONFIG_FILE
64 |     if target.exists() and not a.force:
65 |         print(f"{target} already exists (--force to overwrite, or: vam configure)")
66 |     return 0

```

```

67 |     raw = config.read_json(config.EXAMPLE_FILE)
68 |     if a.demo:
69 |         # the fake CLI is a Python script: run it with this Python, script path as first argument
70 |         raw["vero"]["command"] = sys.executable
71 |         raw["vero"]["version_args"] = [str(FAKE)] + raw["vero"]["version_args"]
72 |         raw["vero"]["task_args"] = [str(FAKE)] + raw["vero"]["task_args"]
73 |         raw["schedule"]["interval_minutes"] = 2
74 |         raw["vero"]["task_timeout_s"] = 30
75 |         raw["thresholds_s"]["task_slow"] = 8
76 |         raw["data_dir"] = "data-demo" # never mixed with real measurements
77 |         config.write_json(target, raw)
78 |         print(f"wrote {target}" + (" (demo: fake Vero CLI, data in data-demo/)" if a.demo else ""))
79 |         return 0
80 |
81 |
82 | def cmd_configure(a: argparse.Namespace) -> int:
83 |     target = Path(a.config) if a.config else config.CONFIG_FILE
84 |     raw = config.read_json(config.EXAMPLE_FILE)
85 |     if target.exists():
86 |         try:
87 |             cur = config.read_json(target)
88 |             if not str(cur.get("data_dir", "")).startswith("data-demo"):
89 |                 raw = config.merge(raw, cur)
90 |             except ConfigError as e:
91 |                 print(f"current config is broken, starting from the defaults ({e})")
92 |                 raw = config.merge(config.DEFAULTS, raw)
93 |             v = raw["vero"]
94 |             print("\nVero availability monitor: configuration. Enter keeps the value in [brackets].\n")
95 |             while True:
96 |                 v["command"] = ask("1. Vero CLI program (name or full path; `where vero` shows it)", v["command"]).strip('')
97 |                 found = shutil.which(v["command"])
98 |                 if found:
99 |                     print(f"found: {found}")
100 |                     break
101 |                 print(f"{v['command']}' was not found on PATH.")
102 |                 if yes("Keep it anyway", False):
103 |                     break
104 |                 v["model"] = ask("2. Model id to pin (-m), cheapest first", v["model"])
105 |                 raw["checks"]["task"] = yes("3. End-to-end task check (~1 min, 2 small model calls per run)", raw["checks"]["task"])
106 |                 raw["checks"]["chat"] = yes("4. Also check Vero Chat (MCP handshake, free; needs a token variable)", raw["checks"]["chat"])
107 |                 if raw["checks"]["chat"]:
108 |                     raw["chat"]["token_env"] = ask(
109 |                         "Name of the environment variable holding the Vero Chat token", raw["chat"]["token_env"]
110 |                     )
111 |                     if not os.environ.get(raw["chat"]["token_env"]):
112 |                         print(f"note: {raw['chat']['token_env']} is not set in this session (see guide, Vero Chat token)")
113 |                     try:
114 |                         raw["schedule"]["interval_minutes"] = int(ask("5. Check every N minutes", str(raw["schedule"]["interval_minutes"])))
115 |                         v["task_timeout_s"] = float(ask("6. Task timeout, seconds", f"{float(v['task_timeout_s']):g}"))
116 |                         raw["thresholds_s"]["task_slow"] = float(
117 |                             ask("7. Task counts as slow (degraded) above, seconds", f"{float(raw['thresholds_s']['task_slow']):g}")
118 |                         )
119 |                         config.build(raw, target)
120 |                     except (ConfigError, ValueError) as e:
121 |                         print(f"not saved: {e}")
122 |                     return 1
123 |                 config.write_json(target, raw)
124 |                 print(f"saved {target}")
125 |                 return 0
126 |
127 |
128 | # commands with a config -----
129 |
130 |
131 | def cmd_doctor(a: argparse.Namespace, cfg: Config) -> int:
132 |     print(f"config {cfg.path}\ndata {cfg.data_dir}")
133 |     print(f"program {cfg.vero.command} -> {shutil.which(cfg.vero.command) or 'NOT FOUND'}")

```

```

134 |     if cfg.check_task:
135 |         exe = shutil.which(cfg.vero.command) or cfg.vero.command
136 |         cmd = task_command(cfg, exe, cfg.data_dir / "sandbox")
137 |         print("task cmd" + " ".join(f"{a}" if " " in a else a for a in cmd))
138 |         print("running the enabled checks once (nothing is stored)...")
139 |         results = run_all(cfg)
140 |         for r in results:
141 |             ms = f"{r.duration_ms / 1000:6.1f} s" if r.duration_ms is not None else "-----"
142 |             extra = " ".join(x for x in (r.version or "", r.model_id or "", r.detail) if x)
143 |             print(f"{ICON[r.status.value]} {r.check.value:<5} {ms} {extra}")
144 |         overall = overall_status(results)
145 |         print(
146 |             f"RESULT {overall.value.upper()}"
147 |             + (" -> ready" if overall is not Status.DOWN else " -> fix the items above (guide: Troubleshooting)")
148 |         )
149 |         return 0 if overall is not Status.DOWN else 1
150 |
151 |
152 | def cmd_check(a: argparse.Namespace, cfg: Config) -> int:
153 |     run = run_once(cfg, Store(cfg.data_dir), a.trigger)
154 |     if run is None:
155 |         print("another run is in progress; skipped")
156 |         return 0
157 |     if not a.quiet:
158 |         for r in run.checks:
159 |             print(f"{ICON[r.status.value]} {r.check.value:<5} {r.detail}")
160 |         print(f"{run.run_id}: {run.overall.value.upper()}")
161 |     if a.trigger == "task":
162 |         return 0 if a.DOWN result is data, not a task failure (Task Scheduler 'Last Result' stays 0)
163 |     return 0 if run.overall is not Status.DOWN else 2
164 |
165 |
166 | def cmd_serve(a: argparse.Namespace, cfg: Config) -> int:
167 |     if a.port:
168 |         cfg = config.build(**config.read_json(cfg.path), "server": {"host": cfg.host, "port": a.port}, cfg.path)
169 |     return serve(cfg, Store(cfg.data_dir), cfg.run_in_dashboard and not a.no_scheduler, a.open)
170 |
171 |
172 | def _set_run_in_dashboard(cfg: Config, value: bool) -> None:
173 |     raw = config.read_json(cfg.path)
174 |     raw.setdefault("schedule", {})[f"run_in_dashboard"] = value
175 |     config.write_json(cfg.path, raw)
176 |
177 |
178 | def cmd_install(a: argparse.Namespace, cfg: Config) -> int:
179 |     every = int(a.every or cfg.interval_minutes)
180 |     if not 1 <= every <= 1440:
181 |         print("--every must be between 1 and 1440")
182 |         return 1
183 |     url = f"http://127.0.0.1:{cfg.port}/"
184 |     for line in scheduler.install(every, url, dashboard=not a.no_dashboard):
185 |         print(line)
186 |     _set_run_in_dashboard(cfg, False)
187 |     print(f"config: schedule.run_in_dashboard = false\nopen the dashboard: {url}")
188 |     return 0
189 |
190 |
191 | def cmd_uninstall(a: argparse.Namespace, cfg: Config) -> int:
192 |     for line in scheduler.remove():
193 |         print(line)
194 |     _set_run_in_dashboard(cfg, True)
195 |     print(f"config: schedule.run_in_dashboard = true. Data kept in {cfg.data_dir}")
196 |     return 0
197 |
198 |
199 | def cmd_status(a: argparse.Namespace, cfg: Config) -> int:
200 |     store = Store(cfg.data_dir)

```

```

201 |     last = store.runs(limit=1)
202 |     print(f"version_{__version__}\nconfig_{cfg.path}\ndata_{cfg.data_dir} ({store.count()} runs)")
203 |     if last:
204 |         r = last[0]
205 |         print(f"last run_{r['run_id']}_{'overall'.upper()}_{r['reason'] or ''}")
206 |     else:
207 |         print("last run none yet")
208 |         print(f"running_{'yes' if in_progress(cfg) else 'no'}")
209 |         up = already_running(cfg.port)
210 |         print(f"dashboard {f'up at http://127.0.0.1:{cfg.port}/' if up else 'not running'}\n")
211 |         print(scheduler.status())
212 |         return 0
213 |
214 |
215 | def cmd_open(a: argparse.Namespace, cfg: Config) -> int:
216 |     url = f"http://127.0.0.1:{cfg.port}/"
217 |     deadline = time.time() + a.wait
218 |     while not already_running(cfg.port):
219 |         if time.time() > deadline:
220 |             print(f"dashboard not running at {url}: use start_dashboard.bat, see data/monitor.log")
221 |             return 1
222 |             time.sleep(1)
223 |             webbrowser.open(url)
224 |             print(f"opened {url}")
225 |             return 0
226 |
227 |
228 | def cmd_export(a: argparse.Namespace, cfg: Config) -> int:
229 |     out = Path(a.out or "vero_availability.csv")
230 |     out.parent.mkdir(parents=True, exist_ok=True)
231 |     rows = Store(cfg.data_dir).csv_rows(time.time() - a.hours * 3600 if a.hours else 0)
232 |     with out.open("w", newline="", encoding="utf-8") as f:
233 |         w = csv.writer(f)
234 |         w.writerow(CSV_HEADER)
235 |         w.writerows(rows)
236 |         print(f"wrote {out} ({len(rows)} rows)")
237 |         return 0
238 |
239 |
240 | def cmd_report(a: argparse.Namespace, cfg: Config) -> int:
241 |     out = Path(a.out or "vero_status_report.html")
242 |     out.parent.mkdir(parents=True, exist_ok=True)
243 |     out.write_text(static_html(cfg, Store(cfg.data_dir), a.hours), encoding="utf-8")
244 |     print(f"wrote {out} (last {a.hours} h, self-contained)")
245 |     return 0
246 |
247 |
248 | def build_parser() -> argparse.ArgumentParser:
249 |     p = argparse.ArgumentParser(prog="vam", description="Vero availability monitor (ASPF-1578)")
250 |     p.add_argument("--version", action="version", version=__version__)
251 |     p.add_argument("--config", help="config file (default config/monitor.json, or env VAM_CONFIG)")
252 |     p.add_argument("-v", "--verbose", action="store_true")
253 |     sub = p.add_subparsers(dest="cmd", metavar="command")
254 |     sub.required = True
255 |     s = sub.add_parser("init", help="create config/monitor.json from the example")
256 |     s.add_argument("--demo", action="store_true", help="bundled fake Vero CLI, data in data-demo/")
257 |     s.add_argument("--force", action="store_true")
258 |     sub.add_parser("configure", help="guided configuration")
259 |     sub.add_parser("doctor", help="run the checks once without storing, and explain the result")
260 |     s = sub.add_parser("check", help="run the checks now and store the result")
261 |     s.add_argument("--trigger", default="manual", choices=["manual", "task", "schedule", "dashboard"])
262 |     s.add_argument("-q", "--quiet", action="store_true")
263 |     s = sub.add_parser("serve", help="status dashboard (with the built-in timer unless installed)")
264 |     s.add_argument("--port", type=int)
265 |     s.add_argument("--no-scheduler", action="store_true")
266 |     s.add_argument("--open", action="store_true")
267 |     s = sub.add_parser("install", help="scheduled check + dashboard at logon + desktop shortcut")

```

```

268 | s.add_argument("--every", type=int)
269 | s.add_argument("--no-dashboard", action="store_true")
270 | sub.add_parser("uninstall", help="remove the tasks and the shortcut (data is kept)")
271 | sub.add_parser("status", help="configuration, last run, dashboard and task state")
272 | s = sub.add_parser("open", help="open the running dashboard")
273 | s.add_argument("--wait", type=float, default=20)
274 | s = sub.add_parser("export", help="CSV of runs and checks")
275 | s.add_argument("--hours", type=float, default=0)
276 | s.add_argument("--out")
277 | s = sub.add_parser("report", help="self-contained HTML snapshot")
278 | s.add_argument("--hours", type=int, default=168, choices=[1, 6, 24, 168, 720])
279 | s.add_argument("--out")
280 | return p
281 |
282 |
283 | HANDLERS: Dict[str, Callable[[argparse.Namespace, Config], int]] = {
284 |     "doctor": cmd_doctor,
285 |     "check": cmd_check,
286 |     "serve": cmd_serve,
287 |     "install": cmd_install,
288 |     "uninstall": cmd_uninstall,
289 |     "status": cmd_status,
290 |     "open": cmd_open,
291 |     "export": cmd_export,
292 |     "report": cmd_report,
293 | }
294 |
295 |
296 | def main(argv: Optional[List[str]] = None) -> int:
297 |     a = build_parser().parse_args(argv)
298 |     try:
299 |         if a.cmd == "init":
300 |             return cmd_init(a)
301 |         if a.cmd == "configure":
302 |             return cmd_configure(a)
303 |         cfg = config.load(a.config)
304 |     except ConfigError as e:
305 |         print(f"config error: {e}")
306 |         return 1
307 |     setup_logging(cfg.data_dir, a.verbose)
308 |     try:
309 |         return HANDLERS[a.cmd](a, cfg)
310 |     except KeyboardInterrupt:
311 |         return 130
312 |     except (RuntimeError, OSError) as e:
313 |         log.error("%s failed: %s", a.cmd, e)
314 |         return 1
315 |     except Exception:
316 |         log.exception("%s crashed", a.cmd)  visible in data/monitor.log even under pythonw
317 |         return 1
=== END FILE: src/vero_monitor/cli.py ===

```

```
=== FILE: src/vero_monitor/config.py (209 lines) ===
```

```

1 | """Typed configuration loaded from config/monitor.json. Non-secret settings only:
2 | the Vero Chat token is read from an environment variable named in the config, never stored."""
3 |
4 | from __future__ import annotations
5 |
6 | import copy
7 | import json
8 | import os
9 | from dataclasses import dataclass
10 | from pathlib import Path
11 | from typing import Any, Dict, Tuple
12 |
13 | ROOT = Path(__file__).resolve().parents[2]
14 | CONFIG_FILE = ROOT / "config" / "monitor.json"
15 | EXAMPLE_FILE = ROOT / "config" / "monitor.example.json"
16 |
17 | DEFAULTS: Dict[str, Any] = {
18 |     "vero": {
19 |         "command": "vero",
20 |         "model": "us.anthropic.claude-haiku-4-5-20251001-v1:0",
21 |         "prompt": "Reply with exactly: OK",
22 |         "expect": "OK",
23 |         "task_timeout_s": 180,
24 |         "task_args": ["task", "--json", "-m", "{model}", "-t", "{timeout}", "-c", "{workdir}", "{prompt}"],
25 |         "version_args": ["version"],
26 |         "env": {},
27 |     },
28 |     "checks": {"cli": True, "task": True, "chat": False},
29 |     "chat": {
30 |         "url": "https://verostudio.sw.nxp.com/mcp",
31 |         "token_env": "VERO_CHAT_TOKEN",
32 |         "timeout_s": 30,
33 |         "ca_bundle": "",
34 |     },
35 |     "thresholds_s": {"task_slow": 90, "chat_slow": 10},
36 |     "schedule": {"interval_minutes": 15, "run_in_dashboard": True},
37 |     "retention_days": 90,
38 |     "server": {"host": "127.0.0.1", "port": 8766},
39 |     "data_dir": "data",
40 | }
41 |
42 |
43 | class ConfigError(Exception):
44 |     pass
45 |
46 |
47 | @dataclass(frozen=True)
48 | class VeroSettings:
49 |     command: str
50 |     model: str
51 |     prompt: str
52 |     expect: str
53 |     task_timeout_s: float
54 |     task_args: Tuple[str, ...]
55 |     version_args: Tuple[str, ...]
56 |     env: Tuple[Tuple[str, str], ...]
57 |
58 |
59 | @dataclass(frozen=True)
60 | class ChatSettings:
61 |     url: str
62 |     token_env: str
63 |     timeout_s: float
64 |     ca_bundle: str
65 |
66 |

```

```

67 | @dataclass(frozen=True)
68 | class Config:
69 |     """vero: VeroSettings
70 |     """chat: ChatSettings
71 |     """check_cli: bool
72 |     """check_task: bool
73 |     """check_chat: bool
74 |     """task_slow_s: float
75 |     """chat_slow_s: float
76 |     """interval_minutes: int
77 |     """run_in_dashboard: bool
78 |     """retention_days: int
79 |     """host: str
80 |     """port: int
81 |     """data_dir: Path
82 |     """path: Path
83 |
84 |     """def public(self) -> Dict[str, Any]:
85 |         """What the dashboard may show. No env values, no token, no token variable value."""
86 |         """return {
87 |             """model": self.vero.model,
88 |             """prompt": self.vero.prompt,
89 |             """checks": {"cli": self.check_cli, "task": self.check_task, "chat": self.check_chat},
90 |             """chat_url": self.chat.url if self.check_chat else None,
91 |             """interval_minutes": self.interval_minutes,
92 |             """task_timeout_s": self.vero.task_timeout_s,
93 |             """task_slow_s": self.task_slow_s,
94 |             """retention_days": self.retention_days,
95 |         }
96 |
97 |
98 | def merge(base: Dict[str, Any], over: Dict[str, Any]) -> Dict[str, Any]:
99 |     """out = copy.deepcopy(base)
100 |     """for k, v in over.items():
101 |         """if k.startswith("_"):
102 |             """continue
103 |             """if isinstance(v, dict) and isinstance(out.get(k), dict):
104 |                 """out[k] = merge(out[k], v)
105 |             """else:
106 |                 """out[k] = v
107 |     """return out
108 |
109 |
110 | def read_json(path: Path) -> Dict[str, Any]:
111 |     """try:
112 |         """text = Path(path).read_text(encoding="utf-8-sig")"""# Notepad may add a BOM
113 |     """except OSError as e:
114 |         """raise ConfigError(f"cannot read {path}: {e}") from e
115 |     """try:
116 |         """data = json.loads(text)
117 |     """except json.JSONDecodeError as e:
118 |         """hint = " (Windows paths: C:\\\\Tools\\\\x or C:/Tools/x)" if "escape" in str(e).lower() else ""
119 |         """raise ConfigError(f"{path}: invalid JSON: {e}{hint}") from e
120 |     """if not isinstance(data, dict):
121 |         """raise ConfigError(f"{path}: the top level must be a JSON object")
122 |     """return data
123 |
124 |
125 | def write_json(path: Path, data: Dict[str, Any]) -> None:
126 |     """path.parent.mkdir(parents=True, exist_ok=True)
127 |     """path.write_text(json.dumps(data, indent=2) + "\\n", encoding="utf-8")
128 |
129 |
130 | def _str_list(value: Any, name: str) -> Tuple[str, ...]:
131 |     """if not isinstance(value, list) or not all(isinstance(a, str) for a in value):
132 |         """raise ConfigError(f"{name} must be a list of strings")
133 |     """return tuple(value)

```

```

134 |
135 |
136 | def _positive(value: Any, name: str, kind: type = float) -> Any:
137 |     try:
138 |         v = kind(value)
139 |     except (TypeError, ValueError) as e:
140 |         raise ConfigError(f"{name} must be a number, got {value!r}") from e
141 |     if v <= 0:
142 |         raise ConfigError(f"{name} must be > 0")
143 |     return v
144 |
145 |
146 | def build(raw: Dict[str, Any], path: Path) -> Config:
147 |     d = merge(DEFAULTS, raw)
148 |     v, c, ch = d["vero"], d["checks"], d["chat"]
149 |     task_args = _str_list(v["task_args"], "vero.task_args")
150 |     if not any("{prompt}" in a for a in task_args):
151 |         raise ConfigError("vero.task_args needs a {prompt} placeholder")
152 |     if not isinstance(v["command"], str) or not v["command"].strip():
153 |         raise ConfigError('vero.command must be the program name or full path, e.g. "vero"')
154 |     env = {} if v.get("env") is None else v["env"]
155 |     if not isinstance(env, dict):
156 |         raise ConfigError('vero.env must be an object, e.g. {"HTTPS_PROXY": "..."}')
157 |     flags = {k: c.get(k) for k in ("cli", "task", "chat")}
158 |     if any(not isinstance(x, bool) for x in flags.values()):
159 |         raise ConfigError("checks.cli / checks.task / checks.chat must be true or false")
160 |     if not (flags["cli"] or flags["task"] or flags["chat"]):
161 |         raise ConfigError("enable at least one check")
162 |     interval = _positive(d["schedule"]["interval_minutes"], "schedule.interval_minutes", int)
163 |     if interval > 1440:
164 |         raise ConfigError("schedule.interval_minutes must be between 1 and 1440")
165 |     timeout = _positive(v["task_timeout_s"], "vero.task_timeout_s")
166 |     worst_case = timeout + min(30.0, timeout) + 30.0 # task hard limit + cli/chat checks
167 |     if flags["task"] and worst_case > interval * 60:
168 |         raise ConfigError(
169 |             f"a run can take up to {worst_case:g} s, longer than the {interval} min interval:"
170 |             " lower vero.task_timeout_s or raise schedule.interval_minutes"
171 |         )
172 |     data_dir = Path(os.path.expandvars(os.path.expanduser(str(d["data_dir"]))))
173 |     return Config(
174 |         vero=VeroSettings(
175 |             command=v["command"].strip(),
176 |             model=str(v["model"]),
177 |             prompt=str(v["prompt"]),
178 |             expect=str(v["expect"]),
179 |             task_timeout_s=timeout,
180 |             task_args=task_args,
181 |             version_args=_str_list(v["version_args"], "vero.version_args"),
182 |             env=tuple((str(k), str(x)) for k, x in env.items()),
183 |         ),
184 |         chat=ChatSettings(
185 |             url=str(ch["url"]),
186 |             token_env=str(ch["token_env"]),
187 |             timeout_s=_positive(ch["timeout_s"], "chat.timeout_s"),
188 |             ca_bundle=str(ch["ca_bundle"]),
189 |         ),
190 |         check_cli=flags["cli"],
191 |         check_task=flags["task"],
192 |         check_chat=flags["chat"],
193 |         task_slow_s=_positive(d["thresholds_s"]["task_slow"], "thresholds_s.task_slow"),
194 |         chat_slow_s=_positive(d["thresholds_s"]["chat_slow"], "thresholds_s.chat_slow"),
195 |         interval_minutes=interval,
196 |         run_in_dashboard=bool(d["schedule"]["run_in_dashboard"]),
197 |         retention_days=_positive(d["retention_days"], "retention_days", int),
198 |         host=str(d["server"]["host"]),
199 |         port=_positive(d["server"]["port"], "server.port", int),
200 |         data_dir=data_dir if data_dir.is_absolute() else ROOT / data_dir,

```

```
201 |         path=path,
202 |     )
203 |
204 |
205 | def load(path: Path | str | None = None) -> Config:
206 |     p = Path(path or os.environ.get("VAM_CONFIG") or CONFIG_FILE)
207 |     if not p.exists():
208 |         raise ConfigError(f"config not found: {p} (run setup.bat, or: python vam.py configure)")
209 |     return build(read_json(p), p)
=== END FILE: src/vero_monitor/config.py ===
```

```
=== FILE: src/vero_monitor/domain.py (152 lines) ===
```

```

1 | """Domain model and pure rules. No I/O here: everything is unit-testable.
2 |
3 | Vocabulary
4 | - check: one probe of one component (cli, task, chat) -> CheckResult
5 | - run: all enabled checks at one moment -> RunResult with an overall Status
6 | - incident: consecutive runs whose overall status is not UP
7 | """
8 |
9 | from __future__ import annotations
10 |
11 | from dataclasses import dataclass, field
12 | from enum import Enum
13 | from typing import Iterable, Sequence
14 |
15 |
16 | class Status(str, Enum):
17 |     UP = "up"
18 |     DEGRADED = "degraded"
19 |     DOWN = "down"
20 |     UNKNOWN = "unknown" # no data
21 |
22 |     @property
23 |     def rank(self) -> int:
24 |         return {"unknown": 0, "up": 1, "degraded": 2, "down": 3}[self.value]
25 |
26 |
27 | def worst(statuses: Iterable[Status]) -> Status:
28 |     result = Status.UNKNOWN
29 |     for s in statuses:
30 |         if s.rank > result.rank:
31 |             result = s
32 |     return result
33 |
34 |
35 | class Check(str, Enum):
36 |     CLI = "cli" # `vero version`: installed and starts (free, seconds)
37 |     TASK = "task" # `vero task --json`: end-to-end answer through the model (2 API calls, ~1 min)
38 |     CHAT = "chat" # Vero Chat MCP `initialize` handshake (free, no model call)
39 |
40 |
41 | @dataclass(frozen=True)
42 | class CheckResult:
43 |     check: Check
44 |     status: Status
45 |     duration_ms: float | None
46 |     detail: str = ""
47 |     version: str | None = None # cli version or chat server version
48 |     provider_id: str | None = None # from task modelInfo (never assumed from config)
49 |     model_id: str | None = None
50 |
51 |
52 | @dataclass(frozen=True)
53 | class RunResult:
54 |     run_id: str
55 |     started_at: str
56 |     epoch: float
57 |     trigger: str
58 |     checks: tuple[CheckResult, ...]
59 |     overall: Status = field(default=Status.UNKNOWN)
60 |
61 |
62 | def overall_status(results: Sequence[CheckResult]) -> Status:
63 |     """DOWN if the primary check is down (task when enabled, else cli).
64 |     DEGRADED if any other check is down or any check is degraded. UP otherwise."""
65 |     if not results:
66 |         return Status.UNKNOWN

```

```

67 | by = {r.check: r for r in results}
68 | primary = by.get(Check.TASK) or by.get(Check.CLI) or results[0]
69 | if primary.status is Status.DOWN:
70 |     return Status.DOWN
71 | if any(r.status in (Status.DOWN, Status.DEGRADED) for r in results):
72 |     return Status.DEGRADED
73 | return Status.UP
74 |
75 |
76 | def availability_percent(statuses: Sequence[Status]) -> float | None:
77 |     """Share of runs where Vero answered (UP or DEGRADED)."""
78 |     known = [s for s in statuses if s is not Status.UNKNOWN]
79 |     if not known:
80 |         return None
81 |     return round(sum(1 for s in known if s is not Status.DOWN) / len(known) * 100, 2)
82 |
83 |
84 | @dataclass(frozen=True)
85 | class Point:
86 |     """One run on the time axis (what the incident and timeline rules need)."""
87 |
88 |     epoch: float
89 |     status: Status
90 |     reason: str = ""
91 |
92 |
93 | @dataclass(frozen=True)
94 | class Incident:
95 |     start: float
96 |     end: float | None = None # None = ongoing
97 |     worst: Status
98 |     runs: int
99 |     reason: str
100 |
101 |     def duration_s(self, now: float) -> float:
102 |         return (self.end if self.end is not None else now) - self.start
103 |
104 |
105 | def incidents(points: Sequence[Point], min_degraded_runs: int = 2) -> list[Incident]:
106 |     """Group consecutive non-UP runs; the incident ends at the first UP run after it.
107 |     Any outage (DOWN) is an incident; a degradation only when it lasts min_degraded_runs runs,
108 |     so one slow answer does not flood the list (it still shows on the timeline)."""
109 |     out: list[Incident] = []
110 |     cur: list[Point] = []
111 |     for p in sorted(points, key=lambda x: x.epoch):
112 |         if p.status in (Status.DOWN, Status.DEGRADED):
113 |             cur.append(p)
114 |             continue
115 |         if cur and p.status is Status.UP:
116 |             out.append(_incident(cur, end=p.epoch))
117 |             cur = []
118 |         if cur:
119 |             out.append(_incident(cur, end=None))
120 |     return [i for i in out if i.worst is Status.DOWN or i.runs >= min_degraded_runs]
121 |
122 |
123 | def _incident(group: list[Point], end: float | None) -> Incident:
124 |     w = worst(p.status for p in group)
125 |     reason = next((p.reason for p in group if p.status is w and p.reason), "")
126 |     return Incident(start=group[0].epoch, end=end, worst=w, runs=len(group), reason=reason)
127 |
128 |
129 | def timeline(points: Sequence[Point], start: float, end: float, bins: int) -> list[tuple[float, Status, int]]:
130 |     """Split [start, end) into equal bins; each bin shows the worst status in it (UNKNOWN if empty)."""
131 |     if bins <= 0 or end <= start:
132 |         return []
133 |     width = (end - start) / bins

```

```

134 |     acc: list[list[Status]] = [[] for _ in range(bins)]
135 |     for p in points:
136 |         if start <= p.epoch < end:
137 |             acc[min(int((p.epoch - start) / width), bins - 1)].append(p.status)
138 |     return [(start + i * width, worst(a), len(a)) for i, a in enumerate(acc)]
139 |
140 |
141 | def current_since(points: Sequence[Point]) -> tuple[Status, float | None]:
142 |     """Current overall status and since when it has been continuously so."""
143 |     pts = sorted(points, key=lambda x: x.epoch)
144 |     if not pts:
145 |         return Status.UNKNOWN, None
146 |     status = pts[-1].status
147 |     since = pts[-1].epoch
148 |     for p in reversed(pts):
149 |         if p.status is not status:
150 |             break
151 |     since = p.epoch
152 |     return status, since
=== END FILE: src/vero_monitor/domain.py ===

```

```
=== FILE: src/vero_monitor/report.py (91 lines) ===
```

```

1 | """Builds the status-page payload (JSON) from stored runs, using the pure domain rules."""
2 |
3 | from __future__ import annotations
4 |
5 | import datetime as dt
6 | import json
7 | import time
8 | from pathlib import Path
9 | from typing import Any, Dict, Optional
10 |
11 | from . import __version__
12 | from .config import Config
13 | from .domain import Check, Status, availability_percent, current_since, incidents, timeline
14 | from .store import Store
15 |
16 | RANGES = {1: 60, 6: 72, 24: 96, 168: 168, 720: 120}# hours -> timeline bars
17 | WEB = Path(__file__).parent / "web"
18 |
19 |
20 | def parse_hours(value: Optional[str], default: int = 24) -> int:
21 |     try:
22 |         h = int(float(value)) if value is not None else default
23 |     except ValueError:
24 |         return default
25 |     return h if h in RANGES else default
26 |
27 |
28 | def payload(
29 |     cfg: Config,
30 |     store: Store,
31 |     hours: int,
32 |     *,
33 |     running: bool = False,
34 |     scheduler: str = "external",
35 |     next_run: Optional[float] = None,
36 |     now: Optional[float] = None,
37 | ) -> Dict[str, Any]:
38 |     now = time.time() if now is None else now
39 |     start = now - hours * 3600
40 |     month = store.points(now - 30 * 86400)
41 |     in_range = [p for p in month if p.epoch >= start] if hours <= 720 else store.points(start)
42 |     status, since = current_since(month)
43 |
44 |     def window(h: int) -> Optional[float]:
45 |         return availability_percent([p.status for p in month if p.epoch >= now - h * 3600])
46 |
47 |     components = {}
48 |     for chk, on in ((Check.CLI, cfg.check_cli), (Check.TASK, cfg.check_task), (Check.CHAT, cfg.check_chat)):
49 |         last = store.latest_check(chk.value) if on else None
50 |         components[chk.value] = {
51 |             "enabled": on,
52 |             **{
53 |                 k: last[k] for k in ("status", "epoch", "duration_ms", "detail", "version", "provider_id", "model_id")}
54 |             if last
55 |             else {}
56 |         },
57 |     }
58 |
59 |     incs = sorted(incidents(in_range), key=lambda i: -i.start)[:20]
60 |     return {
61 |         "generated_at": dt.datetime.fromtimestamp(now, dt.timezone.utc).isoformat(timespec="seconds"),
62 |         "now": now,
63 |         "tool_version": __version__,
64 |         "range_hours": hours,
65 |         "config": cfg.public(),
66 |         "state": {"running": running, "scheduler": scheduler, "next_run_epoch": next_run},

```

```

67 |         "current": {"status": status.value, "since": since},
68 |         "availability": {"24h": window(24), "7d": window(168), "30d": window(720)},
69 |         "counts": {s.value: sum(1 for p in in_range if p.status is s) for s in (Status.UP, Status.DEGRADED, Status.DOWN)},
70 |         "timeline": [{"t": t, "status": s.value, "n": n} for t, s, n in timeline(in_range, start, now, RANGES[hours])],
71 |         "incidents": [
72 |             {
73 |                 "start": i.start,
74 |                 "end": i.end,
75 |                 "worst": i.worst.value,
76 |                 "runs": i.runs,
77 |                 "duration_s": round(i.duration_s(now)),
78 |                 "reason": i.reason,
79 |             }
80 |             for i in incs
81 |         ],
82 |         "components": components,
83 |         "recent": store.runs(start, limit=20),
84 |     }
85 |
86 |
87 | def static_html(cfg: Config, store: Store, hours: int) -> str:
88 |     """Self-contained HTML snapshot (e-mailable). Every '<' in the data is escaped."""
89 |     blob = json.dumps(payload(cfg, store, hours)).replace("<", "\\u003c")
90 |     html = (WEB / "dashboard.html").read_text(encoding="utf-8")
91 |     return html.replace("<!--STATIC_DATA-->", f"<script>window.STATIC_DATA={blob};</script>", 1)
=== END FILE: src/vero_monitor/report.py ===

```

```
=== FILE: src/vero_monitor/runner.py (142 lines) ===
```

```

1 | """Application service: one monitoring run = lock -> checks -> overall status -> store -> manifest."""
2 |
3 | from __future__ import annotations
4 |
5 | import contextlib
6 | import datetime as dt
7 | import json
8 | import logging
9 | import os
10 | import platform
11 | import time
12 | from pathlib import Path
13 | from typing import Callable, List, Optional
14 |
15 | from . import COMPONENT, __version__
16 | from .checks import run_all
17 | from .checks.vero_cli import hard_limit_s
18 | from .config import Config
19 | from .domain import CheckResult, RunResult, overall_status
20 | from .store import Store, reason_of
21 |
22 | log = logging.getLogger("vero_monitor")
23 |
24 |
25 | def iso(t: dt.datetime) -> str:
26 |     """Return t.astimezone(dt.timezone.utc).isoformat(timespec="seconds").replace("+00:00", "Z")
27 |
28 |
29 | def new_run_id(store: Store, when: dt.datetime) -> str:
30 |     """SCMP naming <YYYYMMDD-HHMM>-<component>, suffix -2, -3 within the same minute."""
31 |     base = f"{when.astimezone(dt.timezone.utc).strftime('%Y%m%d-%H%M')}-{COMPONENT}"
32 |     run_id, n = base, 2
33 |     while store.run_exists(run_id):
34 |         run_id, n = f"{base}-{n}", n + 1
35 |     return run_id
36 |
37 |
38 | def stale_after_s(cfg: Config) -> float:
39 |     """A lock older than the longest possible run is left over from a crash."""
40 |     return hard_limit_s(cfg) + cfg.chat.timeout_s + 30 + 120
41 |
42 |
43 | class RunLock:
44 |     """Cross-process lock file. A Task Scheduler run and a dashboard run never overlap."""
45 |
46 |     def __init__(self, data_dir: Path, stale_s: float):
47 |         self.path = Path(data_dir) / "run.lock"
48 |         self.stale_s = stale_s
49 |         self.held = False
50 |
51 |     def __enter__(self) -> RunLock:
52 |         self.path.parent.mkdir(parents=True, exist_ok=True)
53 |         for _ in range(2):
54 |             try:
55 |                 fd = os.open(str(self.path), os.O_CREAT | os.O_EXCL | os.O_WRONLY)
56 |             except FileExistsError:
57 |                 try:
58 |                     if time.time() - self.path.stat().st_mtime > self.stale_s:
59 |                         self.path.unlink()
60 |                         continue
61 |                     except FileNotFoundError:
62 |                         continue
63 |                 return self
64 |             os.write(fd, str(os.getpid()).encode())
65 |             os.close(fd)
66 |             self.held = True

```

```

67 |         return self
68 |     return self
69 |
70 |     def __exit__(self, *exc: object) -> None:
71 |         if self.held:
72 |             with contextlib.suppress(FileNotFoundError):
73 |                 self.path.unlink()
74 |
75 |
76 |     def in_progress(cfg: Config) -> bool:
77 |         try:
78 |             return time.time() - (cfg.data_dir / "run.lock").stat().st_mtime < stale_after_s(cfg)
79 |         except FileNotFoundError:
80 |             return False
81 |
82 |
83 |     Checker = Callable[[Config], List[CheckResult]]
84 |
85 |
86 |     def run_once(cfg: Config, store: Store, trigger: str, checker: Checker = run_all) -> Optional[RunResult]:
87 |         """Run every enabled check once. None if another run holds the lock."""
88 |         with RunLock(cfg.data_dir, stale_after_s(cfg)) as lock:
89 |             if not lock.held:
90 |                 log.info("another run is in progress; skipped")
91 |                 return None
92 |                 now = dt.datetime.now(dt.timezone.utc)
93 |                 run_id = new_run_id(store, now)
94 |                 results = checker(cfg)
95 |                 run = RunResult(
96 |                     run_id=run_id,
97 |                     started_at=iso(now),
98 |                     epoch=now.timestamp(),
99 |                     trigger=trigger,
100 |                     checks=tuple(results),
101 |                     overall=overall_status(results),
102 |                 )
103 |                 store.save(run, platform.node(), __version__)
104 |                 store.prune(cfg.retention_days)
105 |                 write_manifest(cfg, run)
106 |                 log.info("%s %s %s", run_id, run.overall.value.upper(), " ".join(f"{r.check.value}={r.status.value}" for r in results))
107 |                 return run
108 |
109 |
110 |     def write_manifest(cfg: Config, run: RunResult) -> Path:
111 |         """SCMP run manifest: what was checked, with which CLI/model, and the outcome."""
112 |         task = next((c for c in run.checks if c.check.value == "task"), None)
113 |         manifest = {
114 |             "run_id": run.run_id,
115 |             "component": COMPONENT,
116 |             "tool_version": __version__,
117 |             "trigger": run.trigger,
118 |             "host": platform.node(),
119 |             "started_at": run.started_at,
120 |             "overall": run.overall.value,
121 |             "reason": reason_of(run),
122 |             "model": {
123 |                 "provider": task.provider_id if task else None,
124 |                 "id": task.model_id if task else None,
125 |                 "configured_id": cfg.vero.model,
126 |             },
127 |             "checks": [
128 |                 {
129 |                     "check": c.check.value,
130 |                     "status": c.status.value,
131 |                     "duration_ms": None if c.duration_ms is None else round(c.duration_ms, 1),
132 |                     "detail": c.detail,
133 |                     "version": c.version,

```

```
134 |     {}
135 |     {}for c in run.checks
136 |     {},
137 |     {}
138 |     d = cfg.data_dir / "runs" / run.started_at[:7]
139 |     d.mkdir(parents=True, exist_ok=True)
140 |     p = d / f"{run.run_id}.json"
141 |     p.write_text(json.dumps(manifest, indent=2), encoding="utf-8")
142 |     return p
=== END FILE: src/vero_monitor/runner.py ===
```

```
=== FILE: src/vero_monitor/scheduler.py (295 lines) ===
```

```

1 | """Time triggers.
2 |
3 | - Loop: in-process scheduler used while the dashboard runs in "dashboard" mode.
4 | - install()/remove()/status(): the OS scheduler for unattended use.
5 | _Windows: two Task Scheduler tasks, registered from XML so laptop-safe settings apply
6 | _ (runs on battery, catches up after sleep, never two at once):
7 | _ _ _ _ _ VeroAvailabilityCheck _ _ _ _ _ every N minutes: pythonw vam.py check --trigger task
8 | _ _ _ _ _ VeroAvailabilityDashboard at login: _ _ _ _ _ pythonw vam.py serve --no-scheduler (hidden, background)
9 | _ _ _ _ _ Linux/macOS: two crontab lines (check every N minutes, dashboard @reboot).
10 | """
11 |
12 | from __future__ import annotations
13 |
14 | import datetime as dt
15 | import logging
16 | import os
17 | import subprocess
18 | import sys
19 | import tempfile
20 | import threading
21 | import time
22 | from pathlib import Path
23 | from xml.sax.saxutils import escape
24 |
25 | from .checks.process import NO_WINDOW
26 | from .config import ROOT, Config
27 | from .runner import run_once
28 | from .store import Store
29 |
30 | log = logging.getLogger("vero_monitor")
31 |
32 | PROBE_TASK = "VeroAvailabilityCheck"
33 | DASH_TASK = "VeroAvailabilityDashboard"
34 | CRON_MARK = "# vero-availability-monitor"
35 | SHORTCUT = "Vero Status.url"
36 |
37 |
38 | def next_aligned(interval_s: int, now: float | None = None) -> float:
39 |     """Next wall-clock boundary (:00, :15, ...) Task Scheduler and the Loop use the same grid."""
40 |     now = time.time() if now is None else now
41 |     return (now // interval_s + 1) * interval_s
42 |
43 |
44 | class Loop:
45 |     """Runs the checks every interval while the dashboard is open."""
46 |
47 |     def __init__(self, cfg: Config, store: Store):
48 |         self.cfg, self.store = cfg, store
49 |         self.interval = cfg.interval_minutes * 60
50 |         self.active = False
51 |         self.next_run: float | None = None
52 |         self.running = False
53 |         self._stop = threading.Event()
54 |         self._busy = threading.Lock()
55 |
56 |     def expected_next(self) -> float:
57 |         """Next run time; in external mode the OS scheduler uses the same aligned grid."""
58 |         return self.next_run if self.active and self.next_run else next_aligned(self.interval)
59 |
60 |     def trigger(self, source: str) -> bool:
61 |         """Start a cycle in the background. False if one is already running here."""
62 |         if not self._busy.acquire(blocking=False):
63 |             return False
64 |
65 |     def work() -> None:
66 |         self.running = True

```

```

67 |     """try:
68 |         """run_once(self.cfg, self.store, source)
69 |     """except Exception:
70 |         """log.exception("run failed")
71 |     """finally:
72 |         """self.running = False
73 |         """self._busy.release()
74 |
75 |     """threading.Thread(target=work, daemon=True).start()
76 |     """return True
77 |
78 |     """def start(self) -> None:
79 |         """self.active = True
80 |
81 |     """def loop() -> None:
82 |         """self.next_run = next_aligned(self.interval)
83 |         """while not self._stop.wait(max(0.0, self.next_run - time.time())):
84 |         """self.trigger("schedule")
85 |         """self.next_run = next_aligned(self.interval)
86 |
87 |     """threading.Thread(target=loop, daemon=True).start()
88 |
89 |     """def stop(self) -> None:
90 |         """self._stop.set()
91 |
92 |
93 | # commands the OS scheduler runs -----
94 |
95 |
96 | def _python(background: bool) -> str:
97 |     """exe = Path(sys.executable)
98 |     """if os.name == "nt" and background:
99 |     """w = exe.with_name("pythonw.exe")"""# no console window
100 |     """if w.exists():
101 |     """return str(w)
102 |     """return str(exe)
103 |
104 |
105 | def probe_command() -> list[str]:
106 |     """return [_python(True), str(ROOT / "vam.py"), "check", "--trigger", "task", "--quiet"]
107 |
108 |
109 | def dashboard_command() -> list[str]:
110 |     """return [_python(True), str(ROOT / "vam.py"), "serve", "--no-scheduler"]
111 |
112 |
113 | # Windows -----
114 |
115 |
116 | def _win_user() -> str:
117 |     """user = os.environ.get("USERNAME", "")
118 |     """dom = os.environ.get("USERDOMAIN", "")
119 |     """return f"{dom}\\{user}" if dom and user else user
120 |
121 |
122 | def _args(cmd: list[str]) -> str:
123 |     """return " ".join(f'"{a}"' if (" " in a or a.endswith(".py")) else a for a in cmd)
124 |
125 |
126 | def _task_xml(description: str, trigger_xml: str, cmd: list[str], time_limit: str) -> str:
127 |     """user = escape(_win_user())
128 |     """return f'"""<?xml version="1.0" encoding="UTF-16"?>
129 |     <Task version="1.2" xmlns="http://schemas.microsoft.com/windows/2004/02/mit/task">
130 |     <RegistrationInfo><Description>{escape(description)}</Description></RegistrationInfo>
131 |     <Triggers>{trigger_xml}</Triggers>
132 |     <Principals><Principal id="Author"><UserId>{user}</UserId><LogonType>InteractiveToken</LogonType>
133 |     <RunLevel>LeastPrivilege</RunLevel></Principal></Principals>

```

```

134 | <<Settings>
135 | <<<MultipleInstancesPolicy>IgnoreNew</MultipleInstancesPolicy>
136 | <<<DisallowStartIfOnBatteries>>false</DisallowStartIfOnBatteries>
137 | <<<StopIfGoingOnBatteries>>false</StopIfGoingOnBatteries>
138 | <<<StartWhenAvailable>>true</StartWhenAvailable>
139 | <<<RunOnlyIfNetworkAvailable>>false</RunOnlyIfNetworkAvailable>
140 | <<<IdleSettings><<StopOnIdleEnd>>false</StopOnIdleEnd><RestartOnIdle>>false</RestartOnIdle></IdleSettings>
141 | <<<AllowStartOnDemand>>true</AllowStartOnDemand>
142 | <<<Enabled>true</Enabled>
143 | <<<Hidden>>false</Hidden>
144 | <<<ExecutionTimeLimit>{time_limit}</ExecutionTimeLimit>
145 | <<<RestartOnFailure><Interval>PT1M</Interval><Count>3</Count></RestartOnFailure>
146 | <<</Settings>
147 | <<<Actions Context="Author">
148 | <<<<Exec>
149 | <<<<<Command>{escape(cmd[0])}</Command>
150 | <<<<<Arguments>{escape(_args(cmd[1:]))}</Arguments>
151 | <<<<<WorkingDirectory>{escape(str(ROOT))}</WorkingDirectory>
152 | <<<</Exec>
153 | <<</Actions>
154 | </Task>
155 | """
156 |
157 |
158 | def probe_task_xml(interval_minutes: int) -> str:
159 | <<<<start = dt.datetime.fromtimestamp(next_aligned(interval_minutes * 60)).strftime("%Y-%m-%dT%H:%M:%S")
160 | <<<<trig = (
161 | <<<<<<TimeTrigger><Repetition><Interval>PT{interval_minutes}M</Interval>"
162 | <<<<<<StopAtDurationEnd>>false</StopAtDurationEnd></Repetition>"
163 | <<<<<<StartBoundary>{start}</StartBoundary><Enabled>true</Enabled></TimeTrigger>"
164 | <<<<)
165 | <<<<return _task_xml("Vero availability check (ASPF-1578)", trig, probe_command(), "PT1H")
166 |
167 |
168 | def dashboard_task_xml() -> str:
169 | <<<<trig = f"<LogonTrigger><Enabled>true</Enabled><UserId>{escape(_win_user())}</UserId></LogonTrigger>"
170 | <<<<return _task_xml("Vero status dashboard, http://127.0.0.1 (ASPF-1578)", trig, dashboard_command(), "PT0S")
171 |
172 |
173 | def _schtasks(*args: str, check: bool = True) -> subprocess.CompletedProcess[str]:
174 | <<<<r = subprocess.run(["schtasks", *args], capture_output=True, text=True, creationflags=NO_WINDOW)
175 | <<<<if check and r.returncode != 0:
176 | <<<<<<raise RuntimeError(f"schtasks {args[0]} failed: {(r.stderr or r.stdout).strip()}")
177 | <<<<return r
178 |
179 |
180 | def _register(name: str, xml: str) -> None:
181 | <<<<fd, path = tempfile.mkstemp(suffix=".xml")
182 | <<<<os.close(fd)
183 | <<<<try:
184 | <<<<<<Path(path).write_text(xml, encoding="utf-16")
185 | <<<<<<schtasks("/Create", "/F", "/TN", name, "/XML", path)
186 | <<<<<<finally:
187 | <<<<<<os.unlink(path)
188 |
189 |
190 | def _desktop() -> Path:
191 | <<<<"""The real Desktop folder (it may be redirected to OneDrive on Windows)."""
192 | <<<<if sys.platform == "win32":
193 | <<<<<<import winreg
194 |
195 | <<<<<<try:
196 | <<<<<<<<with winreg.OpenKey(
197 | <<<<<<<<<<<<winreg.HKEY_CURRENT_USER, r"Software\Microsoft\Windows\CurrentVersion\Explorer\User Shell Folders"
198 | <<<<<<<<) as key:
199 | <<<<<<<<<<<<value = winreg.QueryValueEx(key, "Desktop")[0]
200 | <<<<<<<<<<<<return Path(os.path.expandvars(str(value)))

```

```

201 | except OSError:
202 |     pass
203 |     return Path.home() / "Desktop"
204 |
205 |
206 | def _shortcut(url: str) -> Path | None:
207 |     d = _desktop()
208 |     if not d.is_dir():
209 |         return None
210 |     p = d / SHORTCUT
211 |     p.write_text(f"[InternetShortcut]\nURL={url}\n", encoding="utf-8")
212 |     return p
213 |
214 |
215 | # public -----
216 |
217 |
218 | def install(interval_minutes: int, url: str, dashboard: bool = True) -> list[str]:
219 |     out = []
220 |     if os.name == "nt":
221 |         _register(PROBE_TASK, probe_task_xml(interval_minutes))
222 |         out.append(f"task '{PROBE_TASK}': check every {interval_minutes} min (also on battery, catches up after sleep)")
223 |         if dashboard:
224 |             _register(DASH_TASK, dashboard_task_xml())
225 |             _schtasks("/Run", "/TN", DASH_TASK, check=False)
226 |             out.append(f"task '{DASH_TASK}': dashboard starts hidden at every logon (started now)")
227 |         else:
228 |             lines = [ln for ln in _crontab_lines() if CRON_MARK not in ln]
229 |             lines.append(f"{_cron_expr(interval_minutes)} {' '.join(_sh(c) for c in probe_command())} {CRON_MARK}")
230 |             if dashboard:
231 |                 lines.append(f"@reboot {' '.join(_sh(c) for c in dashboard_command())} >/dev/null 2>&1 {CRON_MARK}")
232 |                 subprocess.Popen(dashboard_command(), stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, start_new_session=True)
233 |                 _write_crontab(lines)
234 |                 out.append(f"cron: check every {interval_minutes} min" + ("", dashboard @reboot (started now)" if dashboard else ""))
235 |             if dashboard:
236 |                 sc = _shortcut(url)
237 |                 out.append(f"desktop shortcut: {sc}" if sc else "no desktop folder found; open " + url)
238 |             return out
239 |
240 |
241 | def remove() -> list[str]:
242 |     out = []
243 |     if os.name == "nt":
244 |         _schtasks("/End", "/TN", DASH_TASK, check=False)
245 |         for name in (PROBE_TASK, DASH_TASK):
246 |             r = _schtasks("/Delete", "/F", "/TN", name, check=False)
247 |             out.append(f"task '{name}': " + ("removed" if r.returncode == 0 else "not installed"))
248 |         else:
249 |             _write_crontab([ln for ln in _crontab_lines() if CRON_MARK not in ln])
250 |             out.append("cron entries removed (a running dashboard stops at the next reboot or with Ctrl+C)")
251 |             sc = _desktop() / SHORTCUT
252 |             if sc.exists():
253 |                 sc.unlink()
254 |             out.append("desktop shortcut removed")
255 |             return out
256 |
257 |
258 | def status() -> str:
259 |     if os.name == "nt":
260 |         parts = []
261 |         for name in (PROBE_TASK, DASH_TASK):
262 |             r = _schtasks("/Query", "/TN", name, "/FO", "LIST", check=False)
263 |             parts.append(r.stdout.strip() if r.returncode == 0 else f"{name}: not installed")
264 |         return "\n\n".join(parts)
265 |     try:
266 |         mine = [ln for ln in _crontab_lines() if CRON_MARK in ln]
267 |     except RuntimeError as e:

```

```

268 |         return f"OS scheduler: {e}"
269 |     return "\n".join(mine) if mine else "cron entries not installed"
270 |
271 |
272 | def _cron_expr(minutes: int) -> str:
273 |     if minutes < 60:
274 |         return f"*/*{minutes} * * * *"
275 |     if minutes % 60 == 0 and minutes < 1440:
276 |         return f"0 /*{minutes // 60} * * *"
277 |     return "0 0 * * *"
278 |
279 |
280 | def _sh(s: str) -> str:
281 |     return "'" + s.replace("'", "'\\'") + "'" if any(c in s for c in " '\$") else s
282 |
283 |
284 | def _crontab_lines() -> list[str]:
285 |     try:
286 |         r = subprocess.run(["crontab", "-l"], capture_output=True, text=True)
287 |     except FileNotFoundError:
288 |         raise RuntimeError("crontab is not installed on this machine") from None
289 |     return r.stdout.splitlines() if r.returncode == 0 else []
290 |
291 |
292 | def _write_crontab(lines: list[str]) -> None:
293 |     r = subprocess.run(["crontab", "-"], input="\n".join(lines) + "\n", capture_output=True, text=True)
294 |     if r.returncode != 0:
295 |         raise RuntimeError(r.stderr.strip())
=== END FILE: src/vero_monitor/scheduler.py ===

```

```
=== FILE: src/vero_monitor/server.py (184 lines) ===
```

```

1 | """Local status page: dashboard HTML, JSON API and a Server-Sent Events stream.
2 | Binds to 127.0.0.1. Host and Origin are checked (no DNS rebinding, no cross-site 'check now')."""
3 |
4 | from __future__ import annotations
5 |
6 | import contextlib
7 | import csv
8 | import io
9 | import json
10 | import logging
11 | import threading
12 | import time
13 | import urllib.request
14 | import webbrowser
15 | from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
16 | from typing import Any, Dict, Optional, Type
17 | from urllib.parse import parse_qs, urlparse
18 |
19 | from . import __version__
20 | from .config import Config
21 | from .report import WEB, parse_hours, payload
22 | from .runner import in_progress
23 | from .scheduler import Loop
24 | from .store import CSV_HEADER, Store
25 |
26 | log = logging.getLogger("vero_monitor")
27 | APP_ID = "vero-availability"
28 |
29 |
30 | def make_handler(cfg: Config, store: Store, loop: Optional[Loop]) -> Type[BaseHTTPRequestHandler]:
31 |     allowed_hosts = {f"127.0.0.1:{cfg.port}", f"localhost:{cfg.port}", f"{cfg.host}:{cfg.port}"}
32 |
33 |     def running() -> bool:
34 |         return bool(loop and loop.running) or in_progress(cfg)
35 |
36 |     def state() -> Dict[str, Any]:
37 |         return {
38 |             "running": running(),
39 |             "scheduler": "dashboard" if loop and loop.active else "external",
40 |             "next_run": loop.expected_next() if loop else None,
41 |         }
42 |
43 |     class Handler(BaseHTTPRequestHandler):
44 |         server_version = f"{APP_ID}/{__version__}"
45 |
46 |         def log_message(self, format: str, *args: Any) -> None:
47 |             log.debug("%s " + format, self.address_string(), *args)
48 |
49 |         def _send(self, code: int, body: bytes, ctype: str, extra: Optional[Dict[str, str]] = None) -> None:
50 |             self.send_response(code)
51 |             self.send_header("Content-Type", ctype)
52 |             self.send_header("Content-Length", str(len(body)))
53 |             self.send_header("Cache-Control", "no-store")
54 |             self.send_header("X-Content-Type-Options", "nosniff")
55 |             for k, v in (extra or {}).items():
56 |                 self.send_header(k, v)
57 |             self.end_headers()
58 |             self.wfile.write(body)
59 |
60 |         def _json(self, obj: Any, code: int = 200) -> None:
61 |             self._send(code, json.dumps(obj).encode(), "application/json")
62 |
63 |         def _host_ok(self) -> bool:
64 |             return cfg.host == "0.0.0.0" or (self.headers.get("Host") or "").lower() in allowed_hosts
65 |
66 |         def _same_origin(self) -> bool:

```

```

67 | #####Origin = self.headers.get("Origin")
68 | #####return origin is None or urlparse(origin).netloc.lower() == (self.headers.get("Host") or "").lower()
69 |
70 | #####def do_GET(self) -> None:
71 | #####try:
72 | #####if not self._host_ok():
73 | #####return self._json({"error": "forbidden host"}, 403)
74 | #####u = urlparse(self.path)
75 | #####q = {k: v[0] for k, v in parse_qs(u.query).items()}
76 | #####hours = parse_hours(q.get("hours"))
77 | #####if u.path in ("/", "/index.html"):
78 | #####self._send(200, (WEB / "dashboard.html").read_bytes(), "text/html; charset=utf-8")
79 | #####elif u.path == "/api/status":
80 | #####s = state()
81 | #####self._json(payload(cfg, store, hours, running=s["running"], scheduler=s["scheduler"], next_run=s["next_run"]))
82 | #####elif u.path == "/api/health":
83 | #####self._json({"status": "up", "app": APP_ID, "version": __version__})
84 | #####elif u.path == "/api/export.csv":
85 | #####buf = io.StringIO()
86 | #####w = csv.writer(buf, lineterminator="\n")
87 | #####w.writerow(CSV_HEADER)
88 | #####w.writerows(store.csv_rows(time.time() - hours * 3600))
89 | #####self._send(
90 | #####200,
91 | #####buf.getvalue().encode(),
92 | #####"text/csv",
93 | #####{"Content-Disposition": 'attachment; filename="vero_availability.csv"'},
94 | #####)
95 | #####elif u.path == "/api/stream":
96 | #####self._stream()
97 | #####else:
98 | #####self._json({"error": "not found"}, 404)
99 | #####except (BrokenPipeError, ConnectionResetError, ConnectionAbortedError):
100 | #####pass
101 | #####except Exception:
102 | #####log.exception("request failed: %s", self.path)
103 | #####with contextlib.suppress(OSError):
104 | #####self._json({"error": "internal error, see data/monitor.log"}, 500)
105 |
106 | #####def do_POST(self) -> None:
107 | #####if not self._host_ok() or not self._same_origin():
108 | #####return self._json({"error": "forbidden"}, 403)
109 | #####if urlparse(self.path).path != "/api/check":
110 | #####return self._json({"error": "not found"}, 404)
111 | #####if loop is None:
112 | #####return self._json({"error": "unavailable"}, 503)
113 | #####if running():
114 | #####return self._json({"started": False, "running": True}, 409)
115 | #####started = loop.trigger("dashboard")
116 | #####self._json({"started": started, "running": True}, 202 if started else 409)
117 |
118 | #####def _stream(self) -> None:
119 | #####self.send_response(200)
120 | #####self.send_header("Content-Type", "text/event-stream")
121 | #####self.send_header("Cache-Control", "no-store")
122 | #####self.end_headers()
123 | #####last = store.last_epoch()
124 | #####was: Optional[Dict[str, Any]] = None
125 | #####beat = time.time()
126 | #####while True:
127 | #####s = state()
128 | #####if s != was:
129 | #####self._event("state", s)
130 | #####was = s
131 | #####for r in store.runs(after_epoch=last, newest_first=False):
132 | #####self._event("run", r)
133 | #####last = max(last, r["epoch"])

```

```

134 | #####if time.time() - beat > 15:
135 | #####self.wfile.write(b": ping\n\n")
136 | #####self.wfile.flush()
137 | #####beat = time.time()
138 | #####time.sleep(1.5)
139 |
140 | #####def _event(self, name: str, data: Any) -> None:
141 | #####self.wfile.write(f"event: {name}\ndata: {json.dumps(data)}\n\n".encode())
142 | #####self.wfile.flush()
143 |
144 | #####return Handler
145 |
146 |
147 | def already_running(port: int) -> bool:
148 | #####try:
149 | #####with urllib.request.urlopen(f"http://127.0.0.1:{port}/api/health", timeout=2) as r:
150 | #####return bool(json.load(r).get("app") == APP_ID)
151 | #####except Exception:
152 | #####return False
153 |
154 |
155 | def serve(cfg: Config, store: Store, with_scheduler: bool, open_browser: bool) -> int:
156 | #####url = f"http://127.0.0.1:{cfg.port}/"
157 | #####if already_running(cfg.port):
158 | #####print(f"Dashboard already running: {url}")
159 | #####if open_browser:
160 | #####webbrowser.open(url)
161 | #####return 0
162 | #####loop = Loop(cfg, store)
163 | #####try:
164 | #####httpd = ThreadingHTTPServer((cfg.host, cfg.port), make_handler(cfg, store, loop))
165 | #####except OSError as e:
166 | #####msg = f"cannot listen on {cfg.host}:{cfg.port} ({e}); set server.port in config/monitor.json"
167 | #####log.error(msg)
168 | #####print(msg)
169 | #####return 1
170 | #####httpd.daemon_threads = True
171 | #####if with_scheduler:
172 | #####loop.start()
173 | #####log.info("dashboard %s (scheduler: %s)", url, "dashboard" if with_scheduler else "external")
174 | #####print(f"Dashboard: {url}###(Ctrl+C to stop)")
175 | #####if open_browser:
176 | #####threading.Timer(0.5, webbrowser.open, args=(url,)).start()### after the socket is bound
177 | #####try:
178 | #####httpd.serve_forever()
179 | #####except KeyboardInterrupt:
180 | #####pass
181 | #####finally:
182 | #####loop.stop()
183 | #####httpd.server_close()
184 | #####return 0
=== END FILE: src/vero_monitor/server.py ===

```

```
=== FILE: src/vero_monitor/store.py (166 lines) ===
```

```

1 | """SQLite persistence: <data_dir>/availability.db. One row per run, one row per check."""
2 |
3 | from __future__ import annotations
4 |
5 | import contextlib
6 | import sqlite3
7 | import time
8 | from pathlib import Path
9 | from typing import Any, Dict, Iterator, List, Optional
10 |
11 | from .domain import Point, RunResult, Status
12 |
13 | SCHEMA = """
14 | CREATE TABLE IF NOT EXISTS runs (
15 |     run_id TEXT PRIMARY KEY,
16 |     started_at TEXT NOT NULL,
17 |     epoch REAL NOT NULL,
18 |     trigger TEXT NOT NULL,
19 |     overall TEXT NOT NULL,
20 |     reason TEXT,
21 |     host TEXT,
22 |     tool_version TEXT
23 | );
24 | CREATE TABLE IF NOT EXISTS checks (
25 |     id INTEGER PRIMARY KEY AUTOINCREMENT,
26 |     run_id TEXT NOT NULL REFERENCES runs(run_id) ON DELETE CASCADE,
27 |     epoch REAL NOT NULL,
28 |     check_name TEXT NOT NULL,
29 |     status TEXT NOT NULL,
30 |     duration_ms REAL,
31 |     detail TEXT,
32 |     version TEXT,
33 |     provider_id TEXT,
34 |     model_id TEXT
35 | );
36 | CREATE INDEX IF NOT EXISTS ix_runs_epoch ON runs(epoch);
37 | CREATE INDEX IF NOT EXISTS ix_checks_run ON checks(run_id);
38 | """
39 |
40 |
41 | def reason_of(run: RunResult) -> str:
42 |     """The detail of the worst check: what the dashboard shows as the reason."""
43 |     bad = [c for c in run.checks if c.status in (Status.DOWN, Status.DEGRADED)]
44 |     bad.sort(key=lambda c: -c.status.rank)
45 |     return f"{bad[0].check.value}: {bad[0].detail}" if bad else ""
46 |
47 |
48 | class Store:
49 |     def __init__(self, data_dir: Path):
50 |         self.path = Path(data_dir) / "availability.db"
51 |         self.path.parent.mkdir(parents=True, exist_ok=True)
52 |         with self._conn() as c:
53 |             c.execute("PRAGMA journal_mode=WAL")
54 |             c.executescript(SCHEMA)
55 |
56 |     @contextlib.contextmanager
57 |     def _conn(self) -> Iterator[sqlite3.Connection]:
58 |         c = sqlite3.connect(str(self.path), timeout=30)
59 |         c.row_factory = sqlite3.Row
60 |         c.execute("PRAGMA foreign_keys=ON")
61 |         try:
62 |             with c:
63 |                 yield c
64 |         finally:
65 |             c.close()
66 |

```

```

67 |     def save(self, run: RunResult, host: str, tool_version: str) -> None:
68 |         with self._conn() as c:
69 |             c.execute(
70 |                 "INSERT INTO runs VALUES (?, ?, ?, ?, ?, ?, ?)",
71 |                 (run.run_id, run.started_at, run.epoch, run.trigger, run.overall.value, reason_of(run), host, tool_version),
72 |             )
73 |             c.executemany(
74 |                 "INSERT INTO checks(run_id, epoch, check_name, status, duration_ms, detail, version, provider_id, model_id)"
75 |                 " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
76 |                 [
77 |                     (
78 |                         run.run_id,
79 |                         run.epoch,
80 |                         r.check.value,
81 |                         r.status.value,
82 |                         r.duration_ms,
83 |                         r.detail,
84 |                         r.version,
85 |                         r.provider_id,
86 |                         r.model_id,
87 |                     )
88 |                     for r in run.checks
89 |                 ],
90 |             )
91 |
92 |     def run_exists(self, run_id: str) -> bool:
93 |         with self._conn() as c:
94 |             return c.execute("SELECT 1 FROM runs WHERE run_id=?", (run_id,)).fetchone() is not None
95 |
96 |     def prune(self, retention_days: int, now: Optional[float] = None) -> int:
97 |         cutoff = (now or time.time()) - retention_days * 86400
98 |         with self._conn() as c:
99 |             return c.execute("DELETE FROM runs WHERE epoch < ?", (cutoff,)).rowcount
100 |
101 |     def points(self, since: float) -> List[Point]:
102 |         with self._conn() as c:
103 |             rows = c.execute("SELECT epoch, overall, reason FROM runs WHERE epoch >= ? ORDER BY epoch", (since,))
104 |             return [Point(r["epoch"], Status(r["overall"]), r["reason"] or "") for r in rows]
105 |
106 |     def runs(
107 |         self, since: float = 0, limit: Optional[int] = None, newest_first: bool = True, after_epoch: Optional[float] = None
108 |     ) -> List[Dict[str, Any]]:
109 |         q = "SELECT * FROM runs WHERE epoch >= ?"
110 |         args: List[Any] = [since]
111 |         if after_epoch is not None:
112 |             q += " AND epoch > ?"
113 |             args.append(after_epoch)
114 |             q += " ORDER BY epoch DESC" if newest_first else " ORDER BY epoch"
115 |         if limit:
116 |             q += " LIMIT ?"
117 |             args.append(limit)
118 |         with self._conn() as c:
119 |             runs = [dict(r) for r in c.execute(q, args)]
120 |             for r in runs:
121 |                 r["checks"] = [
122 |                     dict(x)
123 |                     for x in c.execute(
124 |                         "SELECT check_name, status, duration_ms, detail, version, provider_id, model_id"
125 |                         " FROM checks WHERE run_id=? ORDER BY id",
126 |                         (r["run_id"],),
127 |                     )
128 |                 ]
129 |             return runs
130 |
131 |     def latest_check(self, name: str) -> Optional[Dict[str, Any]]:
132 |         with self._conn() as c:
133 |             r = c.execute("SELECT * FROM checks WHERE check_name=? ORDER BY epoch DESC, id DESC LIMIT 1", (name,)).fetchone()

```

```

134 |     return dict(r) if r else None
135 |
136 |     def last_epoch(self) -> float:
137 |         with self._conn() as c:
138 |             return float(c.execute("SELECT COALESCE(MAX(epoch), 0) FROM runs").fetchone()[0])
139 |
140 |     def count(self) -> int:
141 |         with self._conn() as c:
142 |             return int(c.execute("SELECT COUNT(*) FROM runs").fetchone()[0])
143 |
144 |     def csv_rows(self, since: float = 0) -> List[List[Any]]:
145 |         with self._conn() as c:
146 |             q = (
147 |                 "SELECT r.run_id, r.started_at, r.trigger, r.overall, k.check_name, k.status, k.duration_ms,"
148 |                 " k.detail, k.version, k.provider_id, k.model_id FROM runs r JOIN checks k USING(run_id)"
149 |                 " WHERE r.epoch >= ? ORDER BY r.epoch, k.id"
150 |             )
151 |             return [list(r) for r in c.execute(q, (since,))]
152 |
153 |
154 |     CSV_HEADER = [
155 |         "run_id",
156 |         "started_at",
157 |         "trigger",
158 |         "overall",
159 |         "check",
160 |         "status",
161 |         "duration_ms",
162 |         "detail",
163 |         "version",
164 |         "provider_id",
165 |         "model_id",
166 |     ]
=== END FILE: src/vero_monitor/store.py ===

```

=== FILE: src/vero_monitor/web/dashboard.html (306 lines) ===

```

1 | <!doctype html>
2 | <html lang="en">
3 | <head>
4 | <meta charset="utf-8">
5 | <meta name="viewport" content="width=device-width, initial-scale=1">
6 | <title>Vero Status</title>
7 | <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 16 16'%3E%3Ccircle cx='8' cy='8' r='7' fill='%231f7a45'/%3E%3C/svg%3E">
8 | <!--STATIC_DATA-->
9 | <style>
10 | :root{
11 |   --ink:#1b2230; --mut:#5d6878; --line:#dde2ea; --paper:#ffffff; --soft:#f4f6f9; --card:#ffffff;
12 |   --brand:#0a6aa1; --up:#1f7a45; --deg:#a8500b; --down:#9b1c2e; --unk:#b7bfc3;
13 |   --up-bg:#e8f4ed; --deg-bg:#fbf0e6; --down-bg:#f8e7ea; --unk-bg:#f1f3f6;
14 |   --sans:"IBM Plex Sans","Segoe UI",Roboto,Helvetica,Arial,sans-serif;
15 |   --mono:"IBM Plex Mono",ui-monospace,Consolas,"Courier New",monospace;
16 | }
17 | @media (prefers-color-scheme: dark){
18 |   :root:not([data-theme="light"]){
19 |     --ink:#e6eaf0; --mut:#9aa5b4; --line:#2c3441; --paper:#12161d; --soft:#1a2029; --card:#161b23;
20 |     --brand:#4fa8dc; --up:#4cc38a; --deg:#e39a4b; --down:#f06b7f; --unk:#4a5363;
21 |     --up-bg:#15291f; --deg-bg:#2e2214; --down-bg:#311a20; --unk-bg:#1d232c;
22 |   }
23 | }
24 | :root[data-theme="dark"]{
25 |   --ink:#e6eaf0; --mut:#9aa5b4; --line:#2c3441; --paper:#12161d; --soft:#1a2029; --card:#161b23;
26 |   --brand:#4fa8dc; --up:#4cc38a; --deg:#e39a4b; --down:#f06b7f; --unk:#4a5363;
27 |   --up-bg:#15291f; --deg-bg:#2e2214; --down-bg:#311a20; --unk-bg:#1d232c;
28 | }
29 | *{box-sizing:border-box}
30 | body{margin:0;background:var(--paper);color:var(--ink);font:14px/1.5 var(--sans)}
31 | .wrap{max-width:1180px;margin:0 auto;padding:22px 24px 48px}
32 | header{display:flex;flex-wrap:wrap;gap:12px 24px;align-items:flex-end;justify-content:space-between;
33 |   padding-bottom:16px;border-bottom:1px solid var(--line)}
34 | .kicker{font-size:12.5px;color:var(--mut)}
35 | h1{font-size:24px;margin:2px 0 0;font-weight:700;letter-spacing:-.01em}
36 | h2{font-size:15px;margin:0 0 2px}
37 | .sub{font-size:12.5px;color:var(--mut);margin-bottom:10px}
38 | .hright{display:flex;flex-wrap:wrap;gap:10px;align-items:center}
39 | .pill{display:inline-flex;align-items:center;gap:7px;font-size:12.5px;padding:4px 10px;border-radius:999px;
40 |   border:1px solid var(--line);background:var(--soft);color:var(--mut)}
41 | .dot{width:9px;height:9px;border-radius:50%;background:var(--unk);flex:none}
42 | .pill.live .dot{background:var(--up);animation:pulse 2s infinite}
43 | .pill.busy .dot{background:var(--brand);animation:pulse 1s infinite}
44 | .pill.down .dot{background:var(--down)}
45 | @keyframes pulse{50%{opacity:.35}}
46 | @media (prefers-reduced-motion:reduce){.dot{animation:none!important}}
47 | button,a.btn{font:inherit;font-size:13px;border:1px solid var(--line);background:var(--card);color:var(--ink);
48 |   border-radius:6px;padding:6px 12px;cursor:pointer;text-decoration:none;display:inline-block}
49 | button.primary{background:var(--brand);border-color:var(--brand);color:#fff;font-weight:600}
50 | button.disabled{opacity:.55;cursor:default}
51 | button:focus-visible,a:focus-visible{outline:2px solid var(--brand);outline-offset:2px}
52 | .hero{margin-top:18px;border-radius:10px;padding:20px 22px;display:flex;gap:18px;align-items:center;
53 |   border:1px solid var(--line);background:var(--unk-bg)}
54 | .hero .big{width:46px;height:46px;border-radius:50%;flex:none;display:flex;align-items:center;justify-content:center;
55 |   color:#fff;font-weight:800;font-size:22px;background:var(--unk)}
56 | .hero h2{font-size:22px;margin:0}
57 | .hero .meta{color:var(--mut);font-size:13px;margin-top:2px}
58 | .hero .why{margin-top:6px;font-size:13px}
59 | .hero.s-up{background:var(--up-bg);border-color:color-mix(in srgb,var(--up) 35%,var(--line))} .hero.s-up .big{background:var(--up)}
60 | .hero.s-degraded{background:var(--deg-bg);border-color:color-mix(in srgb,var(--deg) 35%,var(--line))} .hero.s-degraded .big{background:var(--deg)}
61 | .hero.s-down{background:var(--down-bg);border-color:color-mix(in srgb,var(--down) 35%,var(--line))} .hero.s-down .big{background:var(--down)}
62 | .grid3{display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:12px;margin-top:14px}
63 | .card{background:var(--card);border:1px solid var(--line);border-radius:8px;padding:14px 16px;min-width:0}
64 | .comp .top{display:flex;align-items:center;gap:8px;font-weight:600}
65 | .comp .st{margin-left:auto;font-size:12px;font-weight:600}
66 | .comp .line{font-size:12.5px;color:var(--mut);margin-top:4px;overflow-wrap:anywhere}

```

```

67 | .comp.off{opacity:.6}
68 | .kpi .l{font-size:12px;color:var(--mut)}
69 | .kpi .v{font-size:26px;font-weight:700;font-variant-numeric:tabular-nums}
70 | .c-up{color:var(--up)} .c-degraded{color:var(--deg)} .c-down{color:var(--down)} .c-unknown{color:var(--mut)}
71 | .bg-up{background:var(--up)} .bg-degraded{background:var(--deg)} .bg-down{background:var(--down)} .bg-unknown{background:var(--unk)}
72 | .section{margin-top:14px}
73 | .tl-head{display:flex;flex-wrap:wrap;gap:10px;align-items:center;justify-content:space-between}
74 | .seg{display:inline-flex;border:1px solid var(--line);border-radius:6px;overflow:hidden}
75 | .seg button{border:0;border-radius:0;border-right:1px solid var(--line);padding:5px 11px}
76 | .seg button:last-child{border-right:0}
77 | .seg button[aria-pressed="true"]{background:var(--ink);color:var(--paper);font-weight:600}
78 | .bars{display:flex;gap:2px;height:38px;margin:12px 0 4px;align-items:stretch}
79 | .bars span{flex:1;border-radius:2px;min-width:1px}
80 | .bars span:hover{outline:2px solid var(--ink);outline-offset:1px}
81 | .axis{display:flex;justify-content:space-between;font-size:11.5px;color:var(--mut)}
82 | .legend{display:flex;flex-wrap:wrap;gap:6px 16px;font-size:12.5px;color:var(--mut);margin-top:8px}
83 | .legend i{display:inline-block;width:10px;height:10px;border-radius:2px;margin-right:6px;vertical-align:-1px}
84 | .tw{overflow-x:auto}
85 | table{border-collapse:collapse;width:100%;font-size:13px}
86 | th,td{border-bottom:1px solid var(--line);padding:6px 8px;text-align:left;white-space:nowrap;vertical-align:top}
87 | th{font-weight:600;color:var(--mut);font-size:12px}
88 | td.n{text-align:right;font-variant-numeric:tabular-nums}
89 | td.why{white-space:normal;min-width:220px;color:var(--mut)}
90 | tr.new td{animation:flash 2.4s ease-out}
91 | @keyframes flash{from{background-color:mix(in srgb,var(--brand) 18%,transparent)}to{background:transparent}}
92 | .badge{display:inline-block;font-size:11.5px;font-weight:600;padding:1px 8px;border-radius:999px;border:1px solid currentColor}
93 | .mini{display:inline-flex;gap:4px;align-items:center;margin-right:8px;font-size:12px;color:var(--mut)}
94 | .mini .dot{width:8px;height:8px}
95 | .empty{padding:22px 8px;text-align:center;color:var(--mut)}
96 | footer{margin-top:22px;font-size:12px;color:var(--mut);display:flex;flex-wrap:wrap;gap:6px 18px}
97 | footer code{font-family:var(--mono);font-size:11.5px}
98 | .static-only{display:none}
99 | body.static .live-only{display:none!important}
100 | body.static .static-only{display:inline-flex}
101 | @media (max-width:820px){.grid3{grid-template-columns:1fr}}
102 | @media (max-width:560px){.wrap{padding:16px 16px 40px}.hero{padding:16px}.hero h2{font-size:19px}}
103 | </style>
104 | </head>
105 | <body>
106 | <div class="wrap">
107 |   <header>
108 |     <div>
109 |       <div class="kicker">Quality AI Automation · ASPF-1578</div>
110 |       <h1>Vero status</h1>
111 |     </div>
112 |     <div class="hright">
113 |       <span class="pill live-only" id="live"><span class="dot"></span><span id="liveText">Connecting...</span></span>
114 |       <span class="pill live-only"><span id="nextText">Next check: </span></span>
115 |       <span class="pill static-only"><span class="dot"></span><span id="staticText">Snapshot</span></span>
116 |       <button class="primary live-only" id="runBtn" type="button">Check now</button>
117 |       <a class="btn live-only" id="csvBtn" href="/api/export.csv">Export CSV</a>
118 |       <button id="themeBtn" type="button" aria-label="Toggle light or dark theme">Theme</button>
119 |     </div>
120 |   </header>
121 |
122 |   <section class="hero" id="hero" aria-live="polite">
123 |     <div class="big" id="heroIcon">?</div>
124 |     <div>
125 |       <h2 id="heroTitle">Loading...</h2>
126 |       <div class="meta" id="heroMeta"></div>
127 |       <div class="why" id="heroWhy"></div>
128 |     </div>
129 |   </section>
130 |
131 |   <section class="grid3" id="components" aria-label="Components"></section>
132 |   <section class="grid3" id="kpis" aria-label="Availability"></section>
133 |

```

```

134 |   <<section class="card section">
135 |     <<div class="tl-head">
136 |       <<div><h2>Status over time</h2><div class="sub" id="tlSub">Each bar is the worst status in its time slot.</div></div>
137 |       <<div class="seg live-only" id="range" role="group" aria-label="Time range">
138 |         <<button type="button" data-h="1">1 h</button><button type="button" data-h="6">6 h</button>
139 |         <<button type="button" data-h="24">24 h</button><button type="button" data-h="168">7 d</button>
140 |         <<button type="button" data-h="720">30 d</button>
141 |       <</div>
142 |     <</div>
143 |     <<div class="bars" id="bars"></div>
144 |     <<div class="axis"><span id="axL"></span><span id="axR">now</span></div>
145 |     <<div class="legend">
146 |       <<span><i class="bg-up"></i>available</span><span><i class="bg-degraded"></i>degraded (slow, partial)</span>
147 |       <<span><i class="bg-down"></i>unavailable</span><span><i class="bg-unknown"></i>no check</span>
148 |       <<span id="counts"></span>
149 |     <</div>
150 |   <</section>
151 |
152 |   <<section class="card section">
153 |     <<h2>Incidents</h2>
154 |     <<div class="sub">Periods where Vero was not fully available, newest first.</div>
155 |     <<div class="tw"><table id="incidents"></table></div>
156 |   <</section>
157 |
158 |   <<section class="card section">
159 |     <<h2>Recent checks <span class="sub live-only">· live</span></h2>
160 |     <<div class="tw"><table id="recent"></table></div>
161 |   <</section>
162 |
163 |   <<footer id="foot"></footer>
164 | </div>
165 |
166 | <script>
167 | (function(){
168 | "use strict";
169 | const STATIC = window.STATIC_DATA || null;
170 | const $ = (id) => document.getElementById(id);
171 | const S = {up:"available", degraded:"degraded", down:"unavailable", unknown:"no data"};
172 | const TITLE = {up:"Vero is available", degraded:"Vero is degraded", down:"Vero is unavailable", unknown:"No check yet"};
173 | const ICON = {up:"✓", degraded:"!", down:"×", unknown:"?"};
174 | const COMP = {cli:["Vero CLI", "installed and starts (vero version)"],
175 |   <<task:["Vero task", "end-to-end answer through the model"],
176 |   <<chat:["Vero Chat", "MCP handshake (no model call)"]};
177 | const state = {hours: 24, data: null, es: null, t: null, nextRun: null, running: false};
178 | try { const h = +localStorage.getItem("vam.hours"); if ([1,6,24,168,720].includes(h)) state.hours = h; } catch(e){}
179 | try { const t = localStorage.getItem("vam.theme"); if (t) document.documentElement.dataset.theme = t; } catch(e){}
180 |
181 | const esc = (s) => String(s == null ? "" : s).replace(/[\&<>"]/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;" }[c]));
182 | const nowS = () => STATIC ? STATIC.now : Date.now()/1000;
183 | function dur(s){ s = Math.max(0, Math.round(s)); if (s < 60) return s + " s"; if (s < 3600) return Math.round(s/60) + " min";
184 |   <<if (s < 86400) { const h = Math.floor(s/3600), m = Math.round((s%3600)/60); return h + " h" + (m ? " " + m + " min" : ""); }
185 |   <<const d = Math.floor(s/86400), h = Math.round((s%86400)/3600); return d + " d" + (h ? " " + h + " h" : ""); }
186 | const ago = (e) => e ? dur(nowS() - e) + " ago" : "never";
187 | const when = (e) => new Date(e*1000).toLocaleString(undefined,{month:"short",day:"2-digit",hour:"2-digit",minute:"2-digit"});
188 | const pct = (v) => v == null ? "-" : (v >= 99.995 ? "100" : v.toFixed(v >= 99 ? 2 : 1)) + "%";
189 | const pctClass = (v) => v == null ? "unknown" : v >= 99 ? "up" : v >= 95 ? "degraded" : "down";
190 | const badge = (s) => `<span class="badge c-${esc(s)}">${esc(S[s] || s)}</span>`;
191 | const secs = (ms) => ms == null ? "-" : (ms/1000).toFixed(ms < 10000 ? 1 : 0) + " s";
192 |
193 | async function load(){
194 |   <<if (STATIC) return render(STATIC);
195 |   <<try { const r = await fetch("/api/status?hours=" + state.hours, {cache:"no-store"}); render(await r.json()); }
196 |   <<catch(e){ setLive("down", "Server not reachable"); }
197 | }
198 | function reload(){ clearTimeout(state.t); state.t = setTimeout(load, 500); }
199 |
200 | function render(d){

```

```

201 |   state.data = d;
202 |   document.querySelectorAll("#range button").forEach(b => b.setAttribute("aria-pressed", +b.dataset.h === state.hours));
203 |   $("#csvBtn").href = "/api/export.csv?hours=" + d.range_hours;
204 |   const c = d.current, hero = $("hero");
205 |   hero.className = "hero s-" + c.status;
206 |   $("#heroIcon").textContent = ICON[c.status];
207 |   $("#heroTitle").textContent = TITLE[c.status];
208 |   const last = d.recent[0];
209 |   $("#heroMeta").textContent = c.since ? `since ${when(c.since)} (${dur(nowS() - c.since)}) · last check ${ago(last && last.epoch)}` : "Run a check to see the status.";
210 |   $("#heroWhy").textContent = (c.status !== "up" && last && last.reason) ? "Reason: " + last.reason : "";
211 |   document.title = (c.status === "up" ? "✓ " : c.status === "unknown" ? "" : "▲ ") + "Vero Status";
212 |
213 |   $("#components").innerHTML = ["cli", "task", "chat"].map(k => {
214 |     const x = d.components[k] || {enabled:false}, s = x.enabled ? (x.status || "unknown") : "unknown";
215 |     const facts = !x.enabled ? "not enabled" : [x.version, x.model_id && ("model " + x.model_id),
216 |       x.duration_ms != null && ("took " + secs(x.duration_ms)), x.epoch && ("checked " + ago(x.epoch))].filter(Boolean).join(" · ");
217 |     return `<div class="card comp${x.enabled ? "" : " off"}"><div class="top"><span class="dot bg-${s}"></span>${COMP[k][0]}
218 |     <span class="st c-${s}">${x.enabled ? esc(S[s]) : "off"}</span></div>
219 |     <div class="line">${COMP[k][1]}</div><div class="line">${esc(facts)}</div>
220 |     <div class="line c-${s}">${esc(x.detail)}</div>` : ""</div>`;
221 |   }).join("");
222 |
223 |   $("#kpis").innerHTML = [
224 |     ["Availability, last 24 h", d.availability["24h"]], ["Last 7 days", d.availability["7d"]],
225 |     ["Last 30 days", d.availability["30d"]]].map(([l, v]) =>
226 |       <div class="card kpi"><div class="l">${l}</div><div class="v c-${pctClass(v)}">${pct(v)}</div></div>`
227 |     ).join("");
228 |   const tl = d.timeline, slot = tl.length > 1 ? tl[1].t - tl[0].t : 0;
229 |   $("#bars").innerHTML = tl.map(b => `<span class="bg-${b.status}" title="${esc(when(b.t))} - ${esc(when(b.t + slot))}: ${esc(S[b.status])}${b.n ? ` (${b.n} check${b.n > 1 ? "s" : ""})` : ""}></span>`
230 |     ).join("");
231 |   const n = d.counts; $("#counts").textContent = ` ${n.up} available, ${n.degraded} degraded, ${n.down} unavailable`;
232 |
233 |   $("#incidents").innerHTML = d.incidents.length
234 |     ? `<thead><tr><th>Started</th><th>Duration</th><th>Worst</th><th class="n">Checks</th><th>Reason</th></tr></thead><tbody>` +
235 |     d.incidents.map(i => `<tr><td>${when(i.start)}</td><td>${dur(i.duration_s)}${i.end == null ? " · <b>ongoing</b>" : ""}</td>
236 |     <td>${badge(i.worst)}</td><td class="n">${i.runs}</td><td class="why">${esc(i.reason)}</td></tr>`
237 |     ).join("") + "</tbody>"
238 |     : `<tbody><tr><td class="empty">No incidents in this range.</td></tr></tbody>`;
239 |
240 |   renderRecent(d.recent);
241 |   footer(d);
242 |   state.nextRun = d.state.next_run; setRunning(d.state.running); tick();
243 |   if (STATIC) $("#staticText").textContent = `Snapshot · ${d.range_hours >= 48 ? d.range_hours/24 + " d" : d.range_hours + " h"} · ${when(d.now)}`;
244 | }
245 |
246 | function row(r, isNew){
247 |   const ck = (r.checks || []).map(c => `<span class="mini" title="${esc(c.check_name)}: ${esc(c.detail || S[c.status])}"><span class="dot bg-${esc(c.status)}"></span>${esc(c.check_name)}</span>`
248 |     ).join("");
249 |   const task = (r.checks || []).find(c => c.check_name === "task");
250 |   return `<tr>${isNew ? ` class="new" : ""}><td>${when(r.epoch)}</td><td>${badge(r.overall)}</td><td>${ck}</td>
251 |   <td class="n">${task ? secs(task.duration_ms) : "-"}</td><td class="why">${esc(r.reason || "")}</td></tr>`;
252 | }
253 | function renderRecent(list){
254 |   $("#recent").innerHTML = `<thead><tr><th>Time</th><th>Result</th><th>Checks</th><th class="n">Task time</th><th>Reason</th></tr></thead><tbody>` +
255 |   (list.length ? list.map(r => row(r, false)).join("") : `<tr><td colspan="5" class="empty">No checks yet. Click <b>Check now</b>.</td></tr>`) + "</tbody>";
256 | }
257 | function footer(d){
258 |   const c = d.config;
259 |   $("#foot").innerHTML = `Prompt <code>${esc(c.prompt)}</code>`, Model <code>${esc(c.model)}</code>`,
260 |   `Every ${c.interval_minutes} min (${d.state.scheduler === "dashboard" ? "dashboard timer" : "Task Scheduler"})`,
261 |   `Task timeout ${c.task_timeout_s} s, slow above ${c.task_slow_s} s`, `vero-availability ${esc(d.tool_version)}`
262 |   ).map(x => `<span>${x}</span>`
263 |     ).join("");
264 | }
265 |
266 | function setLive(cls, text){ $("#live").className = "pill live-only " + cls; $("#liveText").textContent = text; }
267 | function setRunning(r){
268 |   state.running = r; $("#runBtn").disabled = !!r; $("#runBtn").textContent = r ? "Checking..." : "Check now";

```

```

266 |   if (state.es && state.es.readyState === 1) setLive(r ? "busy" : "live", r ? "Checking Vero..." : "Live");
267 | }
268 | function tick(){
269 |   if (STATIC || !state.nextRun) { $("nextText").textContent = "Next check: —"; return; }
270 |   const s = Math.max(0, Math.round(state.nextRun - Date.now()/1000));
271 |   $("nextText").textContent = `Next check in ${Math.floor(s/60)}:${String(s%60).padStart(2,"0")}`;
272 | }
273 | function connect(){
274 |   const es = new EventSource("/api/stream"); state.es = es;
275 |   es.onopen = () => setRunning(state.running);
276 |   es.onerror = () => setLive("down", "Reconnecting...");
277 |   es.addEventListener("state", ev => { const s = JSON.parse(ev.data); state.nextRun = s.next_run; setRunning(s.running); tick(); });
278 |   es.addEventListener("run", ev => {
279 |     const r = JSON.parse(ev.data), tb = $("recent").tBodies[0];
280 |     if (tb) { if (tb.querySelector(".empty")) tb.innerHTML = ""; tb.insertAdjacentHTML("afterbegin", row(r, true)); }
281 |     reload();
282 |   });
283 | }
284 |
285 | if (STATIC) { document.body.classList.add("static"); render(STATIC); }
286 | else {
287 |   document.querySelectorAll("#range button").forEach(b => b.addEventListener("click", () => {
288 |     state.hours = +b.dataset.h; try { localStorage.setItem("vam.hours", state.hours); } catch(e){ load(); });
289 |   $("runBtn").addEventListener("click", async () => {
290 |     setRunning(true);
291 |     try { const r = await fetch("/api/check", {method:"POST"}); if (r.status >= 400 && r.status !== 409) setRunning(false); }
292 |     catch(e){ setRunning(false); }
293 |   });
294 |   load().then(connect);
295 |   setInterval(tick, 1000);
296 |   setInterval(load, 60000);
297 | }
298 | $("themeBtn").addEventListener("click", () => {
299 |   const cur = document.documentElement.dataset.theme || (matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light");
300 |   const n = cur === "dark" ? "light" : "dark"; document.documentElement.dataset.theme = n;
301 |   try { localStorage.setItem("vam.theme", n); } catch(e){}
302 | });
303 | })();
304 | </script>
305 | </body>
306 | </html>
=== END FILE: src/vero_monitor/web/dashboard.html ===

```

=== FILE: start_dashboard.bat (7 lines) ===

```
1 | @echo off
2 | rem Opens the status page; starts it first if it is not running (keep this window open then).
3 | setlocal
4 | cd /d "%~dp0"
5 | call scripts\_py.bat || exit /b 1
6 | %PY% vam.py serve --open
7 | pause
=== END FILE: start_dashboard.bat ===
```

```
=== FILE: status.bat (7 lines) ===
1 | @echo off
2 | rem Configuration, last result, status page and Task Scheduler state.
3 | setlocal
4 | cd /d "%~dp0"
5 | call scripts\_py.bat || exit /b 1
6 | %PY% vam.py status
7 | pause
=== END FILE: status.bat ===
```

```
=== FILE: tests/__init__.py (1 lines) ===
1 | """Test package."""
=== END FILE: tests/__init__.py ===
```

```
=== FILE: tests/fixtures/.gitkeep (1 lines) ===  
1 | # synthetic fixtures only  
=== END FILE: tests/fixtures/.gitkeep ===
```

```
=== FILE: tests/fixtures/vero_task_stream.ndjson (8 lines) ===
```

```

1 | {"type":"task_started","taskId":"1790868292727"}
2 | {"ts":1,"type":"say","say":"task","text":"Reply with exactly: OK","modelInfo":{"providerId":"bedrock","modelId":"us.anthropic.claude-haiku-4-5-20251001-v1:0","mode":"act"},"conversationHistoryInd
+ ex":-1}
3 | {"ts":2,"type":"say","say":"api_req_started","text":{"\"request\\":\"SYSTEM PROMPT secret-environment-block\\\""},"modelInfo":{"providerId":"bedrock","modelId":"us.anthropic.claude-haiku-4-5-2025100
+ 1-v1:0","mode":"act"}}
4 | {"ts":3,"type":"say","say":"reasoning","text":"thinking","partial":false}
5 | {"ts":4,"type":"say","say":"text","text":"0","partial":true}
6 | {"ts":5,"type":"say","say":"text","text":"OK","partial":false}
7 | {"ts":6,"type":"say","say":"error","text":"[ERROR] You did not use a tool in your previous response!"}
8 | {"ts":7,"type":"say","say":"completion_result","text":"OK","modelInfo":{"providerId":"bedrock","modelId":"us.anthropic.claude-haiku-4-5-20251001-v1:0","mode":"act"}}
=== END FILE: tests/fixtures/vero_task_stream.ndjson ===

```

```
=== FILE: tests/helpers.py (29 lines) ===
```

```

1 | """Shared test helpers. Tests never call the real Vero CLI or Vero Chat."""
2 |
3 | import sys
4 | from pathlib import Path
5 | from typing import Any, Dict
6 |
7 | ROOT = Path(__file__).resolve().parents[1]
8 | sys.path.insert(0, str(ROOT / "src"))
9 |
10 | from vero_monitor import config_ as noqa: E402
11 |
12 | FAKE = ROOT / "scripts" / "fake_vero.py"
13 | FIXTURES = ROOT / "tests" / "fixtures"
14 |
15 |
16 | def make_config(data_dir: Path, **over: Any) -> config.Config:
17 |     """A valid config that runs the fake Vero CLI with the current Python."""
18 |     raw: Dict[str, Any] = {
19 |         "vero": {
20 |             "command": sys.executable,
21 |             "version_args": [str(FAKE), "version"],
22 |             "task_args": [str(FAKE), "task", "--json", "-m", "{model}", "-t", "{timeout}", "-c", "{workdir}", "{prompt}"],
23 |             "task_timeout_s": 5,
24 |         },
25 |         "thresholds_s": {"task_slow": 3},
26 |         "schedule": {"interval_minutes": 2},
27 |         "data_dir": str(data_dir),
28 |     }
29 |     return config.build(config.merge(raw, over), data_dir / "monitor.json")
=== END FILE: tests/helpers.py ===
```

```
=== FILE: tests/integration/__init__.py (1 lines) ===
  1 | """Integration tests: fake Vero CLI, fake MCP server, real SQLite and HTTP."""
=== END FILE: tests/integration/__init__.py ===
```

```
=== FILE: tests/integration/test_chat.py (104 lines) ===
```

```

1 | """Vero Chat check against a local fake MCP server."""
2 |
3 | import json
4 | import os
5 | import tempfile
6 | import threading
7 | import unittest
8 | from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
9 | from pathlib import Path
10 | from unittest import mock
11 |
12 | from tests.helpers import make_config
13 | from vero_monitor.checks.vero_chat import check_chat
14 | from vero_monitor.domain import Status
15 |
16 |
17 | class FakeMcp(BaseHTTPRequestHandler):
18 |     """mode = "sse"
19 |     """seen_auth = None
20 |
21 |     def log_message(self, *a):
22 |         """pass
23 |
24 |     def do_POST(self):
25 |         """FakeMcp.seen_auth = self.headers.get("Authorization")
26 |         """body = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
27 |         """assert body["method"] == "initialize"
28 |         """if self.mode == "401":
29 |             """self.send_response(401)
30 |             """self.end_headers()
31 |             """return
32 |             """msg = {"jsonrpc": "2.0", "id": 1, "result": {"serverInfo": {"name": "WChat MCP Server", "version": "4.0.3"}}}
33 |             """if self.mode == "error":
34 |                 """msg = {"jsonrpc": "2.0", "id": 1, "error": {"code": -32600, "message": "bad request"}}
35 |                 """data = (f"event: message\\ndata: {json.dumps(msg)}\\n\\n" if self.mode != "html" else "<html>SSO</html>").encode()
36 |                 """self.send_response(200)
37 |                 """self.send_header("Content-Type", "text/event-stream")
38 |                 """self.send_header("Content-Length", str(len(data)))
39 |                 """self.end_headers()
40 |                 """self.wfile.write(data)
41 |
42 |
43 | class ChatCheckTest(unittest.TestCase):
44 |     """@classmethod
45 |     """def setUpClass(cls):
46 |         """cls.srv = ThreadingHTTPServer(("127.0.0.1", 0), FakeMcp)
47 |         """threading.Thread(target=cls.srv.serve_forever, daemon=True).start()
48 |         """cls.url = f"http://127.0.0.1:{cls.srv.server_address[1]}/mcp"
49 |
50 |     """@classmethod
51 |     """def tearDownClass(cls):
52 |         """cls.srv.shutdown()
53 |         """cls.srv.server_close()
54 |
55 |     """def setUp(self):
56 |         """self.tmp = tempfile.TemporaryDirectory()
57 |         """self.cfg = make_config(
58 |             """Path(self.tmp.name), checks={"chat": True}, chat={"url": self.url, "token_env": "VAM_TEST_TOKEN", "timeout_s": 5}
59 |             """
60 |         """self.env = mock.patch.dict(os.environ, {"VAM_TEST_TOKEN": "tok-123"})
61 |         """self.env.start()
62 |
63 |     """def tearDown(self):
64 |         """self.env.stop()
65 |         """self.tmp.cleanup()
66 |

```

```

67 |     def run_mode(self, mode):
68 |         FakeMcp.mode = mode
69 |         return check_chat(self.cfg)
70 |
71 |     def test_handshake_up(self):
72 |         r = self.run_mode("sse")
73 |         self.assertEqual((r.status, r.version), (Status.UP, "WChat MCP Server 4.0.3"))
74 |         self.assertEqual(FakeMcp.seen_auth, "Bearer tok-123")
75 |         self.assertNotIn("tok-123", repr(r))
76 |
77 |     def test_rejected_token(self):
78 |         r = self.run_mode("401")
79 |         self.assertEqual(r.status, Status.DOWN)
80 |         self.assertIn("token rejected", r.detail)
81 |
82 |     def test_mcp_error_and_non_mcp_reply(self):
83 |         self.assertIn("bad request", self.run_mode("error").detail)
84 |         self.assertIn("not an MCP", self.run_mode("html").detail)
85 |
86 |     def test_missing_token_variable(self):
87 |         os.environ.pop("VAM_TEST_TOKEN")
88 |         r = check_chat(self.cfg)
89 |         self.assertEqual(r.status, Status.DOWN)
90 |         self.assertIn("VAM_TEST_TOKEN", r.detail)
91 |
92 |     def test_unreachable(self):
93 |         cfg = make_config(
94 |             Path(self.tmp.name),
95 |             checks={"chat": True},
96 |             chat={"url": "http://127.0.0.1:9/mcp", "token_env": "VAM_TEST_TOKEN", "timeout_s": 2},
97 |         )
98 |         r = check_chat(cfg)
99 |         self.assertEqual(r.status, Status.DOWN)
100 |         self.assertIn("unreachable", r.detail)
101 |
102 |
103 | if __name__ == "__main__":
104 |     unittest.main()
=== END FILE: tests/integration/test_chat.py ===

```

```
=== FILE: tests/integration/test_checks.py (118 lines) ===
```

```

1 | """The CLI checks against scripts/fake_vero.py in every failure mode."""
2 |
3 | import os
4 | import tempfile
5 | import time
6 | import unittest
7 | from pathlib import Path
8 | from unittest import mock
9 |
10 | from tests.helpers import make_config
11 | from vero_monitor.checks import run_all
12 | from vero_monitor.checks.vero_cli import check_cli, check_task
13 | from vero_monitor.domain import Check, CheckResult, Status
14 |
15 |
16 | class CliChecksTest(unittest.TestCase):
17 |     """def setUp(self):
18 |         """self.tmp = tempfile.TemporaryDirectory()
19 |         """self.cfg = make_config(Path(self.tmp.name))
20 |         """self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
21 |         """self.env.start()
22 |
23 |     """def tearDown(self):
24 |         """self.env.stop()
25 |         """self.tmp.cleanup()
26 |
27 |     """def mode(self, m, delay="0.05"):
28 |         """os.environ["FAKE_VERO_MODE"] = m
29 |         """os.environ["FAKE_VERO_DELAY"] = delay
30 |         """return check_task(self.cfg)
31 |
32 |     """def test_cli_up_with_version(self):
33 |         """r = check_cli(self.cfg)
34 |         """self.assertEqual((r.status, r.version), (Status.UP, "vero 2.3.3 (fake0000)"))
35 |
36 |     """def test_cli_missing(self):
37 |         """cfg = make_config(Path(self.tmp.name), vero={"command": "no-such-vero-xyz"})
38 |         """self.assertEqual(check_cli(cfg).status, Status.DOWN)
39 |         """self.assertIn("not found", check_task(cfg).detail)
40 |
41 |     """def test_task_up_reports_real_model(self):
42 |         """r = self.mode("ok")
43 |         """self.assertEqual(r.status, Status.UP, r.detail)
44 |         """self.assertEqual((r.provider_id, r.model_id), ("bedrock", self.cfg.vero.model))
45 |         """self.assertTrue((self.cfg.data_dir / "sandbox").is_dir())
46 |
47 |     """def test_task_slow_is_degraded(self):
48 |         """r = self.mode("ok", delay="3.2")"""# threshold 3 s in make_config
49 |         """self.assertEqual(r.status, Status.DEGRADED)
50 |         """self.assertIn("slow", r.detail)
51 |
52 |     """def test_task_provider_down(self):
53 |         """r = self.mode("down")
54 |         """self.assertEqual(r.status, Status.DOWN)
55 |         """self.assertIn("provider unavailable", r.detail)
56 |
57 |     """def test_task_auth_error_secret_masked(self):
58 |         """r = self.mode("auth")
59 |         """self.assertEqual(r.status, Status.DOWN)
60 |         """self.assertIn("Unauthorized", r.detail)
61 |         """self.assertNotIn("abc123secret", r.detail)
62 |
63 |     """def test_no_completion_result_is_down(self):
64 |         """r = self.mode("nocompletion")
65 |         """self.assertEqual((r.status, r.detail), (Status.DOWN, "no completion_result"))
66 |

```

```

67 |     def test_refusal_is_degraded_not_down(self):
68 |         r = self.mode("refuse")
69 |         self.assertEqual(r.status, Status.DEGRADED)
70 |         self.assertIn("not with 'OK'", r.detail)
71 |
72 |     def test_hang_is_killed_at_hard_limit(self):
73 |         cfg = make_config(Path(self.tmp.name), vero={"task_timeout_s": 1})
74 |         os.environ["FAKE_VERO_MODE"] = "hang"
75 |         t0 = time.time()
76 |         r = check_task(cfg)
77 |         self.assertEqual(r.status, Status.DOWN)
78 |         self.assertIn("no completion within 2 s", r.detail)
79 |         self.assertLess(time.time() - t0, 6)
80 |
81 |     def test_missing_json_flag_explained(self):
82 |         cfg = make_config(Path(self.tmp.name), vero={"task_args": [self.cfg.vero.task_args[0], "task", "{prompt}"]})
83 |         r = check_task(cfg)
84 |         self.assertEqual(r.status, Status.DOWN)
85 |         self.assertIn("--json", r.detail)
86 |
87 |
88 | class RunAllTest(unittest.TestCase):
89 |     def setUp(self):
90 |         self.tmp = tempfile.TemporaryDirectory()
91 |
92 |     def tearDown(self):
93 |         self.tmp.cleanup()
94 |
95 |     def test_task_skipped_when_cli_down(self):
96 |         cfg = make_config(Path(self.tmp.name), vero={"command": "no-such-vero-xyz"})
97 |         res = run_all(cfg)
98 |         self.assertEqual([(r.check, r.status) for r in res], [(Check.CLI, Status.DOWN), (Check.TASK, Status.DOWN)])
99 |         self.assertIn("skipped", res[1].detail)
100 |
101 |     def test_crashing_check_is_contained(self):
102 |         cfg = make_config(Path(self.tmp.name))
103 |
104 |         def boom(_):
105 |             raise ValueError("bug")
106 |
107 |         cli_up = lambda c: CheckResult(Check.CLI, Status.UP, 1.0)  # noqa: E731
108 |         with mock.patch("vero_monitor.checks.check_task", boom), mock.patch(
109 |             "vero_monitor.checks.check_cli", cli_up
110 |         ), self.assertLogs("vero_monitor", "ERROR") as logs:
111 |             res = run_all(cfg)
112 |             self.assertIn("check task crashed", logs.output[0])
113 |             self.assertEqual(res[-1].status, Status.DOWN)
114 |             self.assertEqual(res[-1].detail, "monitor error: ValueError")
115 |
116 |
117 | if __name__ == "__main__":
118 |     unittest.main()
=== END FILE: tests/integration/test_checks.py ===

```

=== FILE: tests/integration/test_runner_server.py (147 lines) ===

```

1 | """Run lifecycle (lock, store, manifest, privacy) and the HTTP status page."""
2 |
3 | import json
4 | import os
5 | import socket
6 | import tempfile
7 | import threading
8 | import time
9 | import unittest
10 | import urllib.error
11 | import urllib.request
12 | from http.server import ThreadingHTTPServer
13 | from pathlib import Path
14 | from unittest import mock
15 |
16 | from tests.helpers import make_config
17 | from vero_monitor.domain import Check, CheckResult, Status
18 | from vero_monitor.report import payload, static_html
19 | from vero_monitor.runner import RunLock, in_progress, run_once
20 | from vero_monitor.scheduler import Loop
21 | from vero_monitor.server import make_handler
22 | from vero_monitor.store import Store
23 |
24 |
25 | def free_port():
26 |     with socket.socket() as s:
27 |         s.bind(("127.0.0.1", 0))
28 |         return s.getsockname()[1]
29 |
30 |
31 | class RunnerTest(unittest.TestCase):
32 |     def setUp(self):
33 |         self.tmp = tempfile.TemporaryDirectory()
34 |         self.cfg = make_config(Path(self.tmp.name))
35 |         self.store = Store(self.cfg.data_dir)
36 |         self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
37 |         self.env.start()
38 |
39 |     def tearDown(self):
40 |         self.env.stop()
41 |         self.tmp.cleanup()
42 |
43 |     def test_run_stores_and_writes_manifest(self):
44 |         run = run_once(self.cfg, self.store, "manual")
45 |         self.assertEqual(run.overall, Status.UP)
46 |         self.assertRegex(run.run_id, r"^\d{8}-\d{4}-vero-availability$")
47 |         m = json.loads(next((self.cfg.data_dir / "runs").rglob("*.json")).read_text())
48 |         self.assertEqual(m["model"]["id"], self.cfg.vero.model)
49 |         self.assertEqual([c["check"] for c in m["checks"]], ["cli", "task"])
50 |         self.assertEqual(self.store.count(), 1)
51 |         self.assertNotEqual(run_once(self.cfg, self.store, "manual").run_id, run.run_id)  # -2 suffix
52 |         self.assertFalse(in_progress(self.cfg))
53 |
54 |     def test_full_prompt_never_persisted(self):
55 |         run_once(self.cfg, self.store, "manual")
56 |         blobs = [p.read_bytes() for p in self.cfg.data_dir.rglob("*") if p.is_file()]
57 |         self.assertFalse(any(b"FAKE-SECRET-api_req_started" in b for b in blobs))
58 |
59 |     def test_lock_blocks_overlap_and_stale_lock_is_cleared(self):
60 |         with RunLock(self.cfg.data_dir, 999) as held:
61 |             self.assertTrue(held.held)
62 |             self.assertTrue(in_progress(self.cfg))
63 |             self.assertIsNone(run_once(self.cfg, self.store, "manual"))
64 |             lock = self.cfg.data_dir / "run.lock"
65 |             lock.write_text("1")
66 |             old = time.time() - 99999

```

```

67 | os.utime(lock, (old, old))
68 | self.assertFalse(in_progress(self.cfg))
69 | self.assertIsNotNone(run_once(self.cfg, self.store, "manual"))
70 |
71 | def test_down_run_reason_and_prune(self):
72 |     down = [CheckResult(Check.CLI, Status.UP, 1.0), CheckResult(Check.TASK, Status.DOWN, 5.0, "provider unavailable")]
73 |     run_once(self.cfg, self.store, "manual", checker=lambda c: down)
74 |     r = self.store.runs(limit=1)[0]
75 |     self.assertEqual((r["overall"], r["reason"]), ("down", "task: provider unavailable"))
76 |     self.assertEqual(len(r["checks"]), 2)
77 |     self.assertEqual(self.store.prune(1, now=time.time() + 3 * 86400), 1)
78 |     self.assertEqual(self.store.csv_rows(), [])uu# checks removed with their run (cascade)
79 |
80 | def test_payload_and_static_report(self):
81 |     run_once(self.cfg, self.store, "manual", checker=lambda c: [CheckResult(Check.TASK, Status.DOWN, 1.0, "</script><x>")])
82 |     d = payload(self.cfg, self.store, 24)
83 |     self.assertEqual(d["current"]["status"], "down")
84 |     self.assertEqual(len(d["timeline"]), 96)
85 |     self.assertEqual(d["availability"]["24h"], 0.0)
86 |     self.assertEqual(d["incidents"][0]["end"], None)
87 |     html = static_html(self.cfg, self.store, 24)
88 |     blob = html.split("window.STATIC_DATA=", 1)[1].split("</script>", 1)[0]
89 |     self.assertNotIn("<", blob)
90 |     self.assertEqual(json.loads(blob)["recent"][0]["reason"], "task: </script><x>")
91 |
92 |
93 | class ServerTest(unittest.TestCase):
94 |     def setUp(self):
95 |         self.tmp = tempfile.TemporaryDirectory()
96 |         port = free_port()
97 |         self.cfg = make_config(Path(self.tmp.name), server={"host": "127.0.0.1", "port": port})
98 |         self.store = Store(self.cfg.data_dir)
99 |         self.loop = Loop(self.cfg, self.store)
100 |        self.srv = ThreadingHTTPServer(("127.0.0.1", port), make_handler(self.cfg, self.store, self.loop))
101 |        threading.Thread(target=self.srv.serve_forever, daemon=True).start()
102 |        self.base = f"http://127.0.0.1:{port}"
103 |        self.env = mock.patch.dict(os.environ, {"FAKE_VERO_DELAY": "0.05", "FAKE_VERO_MODE": "ok"})
104 |        self.env.start()
105 |
106 |    def tearDown(self):
107 |        self.env.stop()
108 |        self.srv.shutdown()
109 |        self.srv.server_close()
110 |        self.tmp.cleanup()
111 |
112 |    def status(self, path, method="GET", **headers):
113 |        try:
114 |            return urllib.request.urlopen(urllib.request.Request(self.base + path, method=method, headers=headers)).status
115 |        except urllib.error.HTTPError as e:
116 |            return e.code
117 |
118 |    def get_json(self, path):
119 |        with urllib.request.urlopen(self.base + path) as r:
120 |            return json.load(r)
121 |
122 |    def test_pages_and_api(self):
123 |        with urllib.request.urlopen(self.base + "/") as r:
124 |            self.assertIn(b"Vero status", r.read())
125 |            d = self.get_json("/api/status?hours=6")
126 |            self.assertEqual((d["range_hours"], d["current"]["status"]), (6, "unknown"))
127 |            self.assertEqual(self.get_json("/api/status?hours=abc")["range_hours"], 24)
128 |            self.assertEqual(self.get_json("/api/health")["app"], "vero-availability")
129 |            self.assertEqual(self.status("/nope"), 404)
130 |
131 |    def test_security_checks(self):
132 |        self.assertEqual(self.status("/api/health", Host="evil.example"), 403)
133 |        self.assertEqual(self.status("/api/check", "POST", Origin="http://evil.example"), 403)

```

```

134 |
135 |     def test_check_now_then_csv(self):
136 |         self.assertEqual(self.status("/api/check", "POST", Origin=self.base), 202)
137 |         for _ in range(60):
138 |             if self.store.count() == 1 and not self.loop.running:
139 |                 break
140 |             time.sleep(0.1)
141 |             self.assertEqual(self.store.count(), 1)
142 |             with urllib.request.urlopen(self.base + "/api/export.csv") as r:
143 |                 self.assertEqual(r.read().decode().count("\n"), 3)
144 |         # header + cli + task
145 |
146 | if __name__ == "__main__":
147 |     unittest.main()
=== END FILE: tests/integration/test_runner_server.py ===

```

```
=== FILE: tests/unit/__init__.py (1 lines) ===
  1 | """Unit tests: pure logic, no processes, no network."""
=== END FILE: tests/unit/__init__.py ===
```

```
=== FILE: tests/unit/test_config.py (81 lines) ===
```

```

1 | import json
2 | import os
3 | import tempfile
4 | import unittest
5 | from pathlib import Path
6 | from unittest import mock
7 |
8 | import tests.helpers_ as # noqa: F401
9 | from vero_monitor import config
10 | from vero_monitor.config import ConfigError
11 |
12 |
13 | class ConfigTest(unittest.TestCase):
14 |     def setUp(self):
15 |         self.tmp = tempfile.TemporaryDirectory()
16 |         self.dir = Path(self.tmp.name)
17 |
18 |     def tearDown(self):
19 |         self.tmp.cleanup()
20 |
21 |     def write(self, obj, bom=False):
22 |         p = self.dir / "c.json"
23 |         p.write_bytes((b"\xef\xbb\xbf" if bom else b"") + json.dumps(obj).encode())
24 |         return p
25 |
26 |     def test_example_file_is_valid_and_equals_defaults(self):
27 |         cfg = config.load(config.EXAMPLE_FILE)
28 |         self.assertEqual(config.read_json(config.EXAMPLE_FILE), config.DEFAULTS)
29 |         self.assertEqual((cfg.interval_minutes, cfg.vero.task_timeout_s, cfg.port), (15, 180, 8766))
30 |         self.assertIn("--json", cfg.vero.task_args)
31 |         self.assertNotIn("--yolo", cfg.vero.task_args)
32 |
33 |     def test_notepad_bom(self):
34 |         self.assertEqual(config.load(self.write({}, bom=True)).vero.command, "vero")
35 |
36 |     def test_invalid_values(self):
37 |         bad = [
38 |             {"schedule": {"interval_minutes": "abc"}},
39 |             {"schedule": {"interval_minutes": 0}},
40 |             {"schedule": {"interval_minutes": 2000}},
41 |             {"vero": {"task_args": ["task"]}}, # no {prompt}
42 |             {"vero": {"task_args": "task --json"}}, # not a list
43 |             {"vero": {"command": " "}},
44 |             {"checks": {"cli": False, "task": False, "chat": False}},
45 |             {"checks": {"task": "yes"}},
46 |             {"schedule": {"interval_minutes": 3}}, # 180 s task cannot fit in 3 min
47 |             {"retention_days": -1},
48 |             {"vero": {"env": []}},
49 |         ]
50 |         for b in bad:
51 |             with self.assertRaises(ConfigError, msg=b):
52 |                 config.load(self.write(b))
53 |
54 |     def test_broken_json_and_hint(self):
55 |         p = self.dir / "c.json"
56 |         p.write_text('{"data_dir": "C:\\Tools\\x"}')
57 |         with self.assertRaisesRegex(ConfigError, "Windows paths"):
58 |             config.load(p)
59 |         p.write_text("[1]")
60 |         with self.assertRaisesRegex(ConfigError, "object"):
61 |             config.load(p)
62 |
63 |     def test_missing_file(self):
64 |         with self.assertRaisesRegex(ConfigError, "not found"):
65 |             config.load(self.dir / "nope.json")
66 |

```

```

67 |     def test_data_dir_relative_and_env(self):
68 |         self.assertEqual(config.load(self.write({})).data_dir, config.ROOT / "data")
69 |         var = "%VAM_T%" if os.name == "nt" else "$VAM_T"
70 |         with mock.patch.dict(os.environ, {"VAM_T": str(self.dir)}):
71 |             self.assertEqual(config.load(self.write({"data_dir": f"{var}/d"})).data_dir, self.dir / "d")
72 |
73 |     def test_public_view_has_no_secrets(self):
74 |         cfg = config.load(self.write({"vero": {"env": {"AWS_SECRET": "s3cr3t"}}, "checks": {"chat": True}}))
75 |         text = json.dumps(cfg.public())
76 |         self.assertNotIn("s3cr3t", text)
77 |         self.assertNotIn("token_env", text)
78 |
79 |
80 | if __name__ == "__main__":
81 |     unittest.main()
=== END FILE: tests/unit/test_config.py ===

```

```
=== FILE: tests/unit/test_domain.py (112 lines) ===
```

```

1 | import unittest
2 |
3 | import tests.helpers_ as noqa: F401_ (sets sys.path)
4 | from vero_monitor.domain import (
5 |     Check,
6 |     CheckResult,
7 |     Point,
8 |     Status,
9 |     availability_percent,
10 |     current_since,
11 |     incidents,
12 |     overall_status,
13 |     timeline,
14 |     worst,
15 | )
16 |
17 | U, D, X, N = Status.UP, Status.DEGRADED, Status.DOWN, Status.UNKNOWN
18 |
19 |
20 | def cr(check, status, detail=""):
21 |     return CheckResult(check, status, 1.0, detail)
22 |
23 |
24 | class WorstTest(unittest.TestCase):
25 |     def test_order(self):
26 |         self.assertIs(worst([]), N)
27 |         self.assertIs(worst([U, D]), D)
28 |         self.assertIs(worst([U, X, D]), X)
29 |         self.assertIs(worst([N, U]), U)
30 |
31 |
32 | class OverallStatusTest(unittest.TestCase):
33 |     def test_empty_is_unknown(self):
34 |         self.assertIs(overall_status([]), N)
35 |
36 |     def test_all_up(self):
37 |         self.assertIs(overall_status([cr(Check.CLI, U), cr(Check.TASK, U)]), U)
38 |
39 |     def test_task_down_means_down(self):
40 |         self.assertIs(overall_status([cr(Check.CLI, U), cr(Check.TASK, X)]), X)
41 |
42 |     def test_task_slow_means_degraded(self):
43 |         self.assertIs(overall_status([cr(Check.CLI, U), cr(Check.TASK, D)]), D)
44 |
45 |     def test_secondary_down_only_degrades(self):
46 |         self.assertIs(overall_status([cr(Check.CLI, U), cr(Check.CHAT, X), cr(Check.TASK, U)]), D)
47 |
48 |     def test_cli_is_primary_when_no_task(self):
49 |         self.assertIs(overall_status([cr(Check.CLI, X), cr(Check.CHAT, U)]), X)
50 |         self.assertIs(overall_status([cr(Check.CLI, U), cr(Check.CHAT, X)]), D)
51 |
52 |     def test_chat_only(self):
53 |         self.assertIs(overall_status([cr(Check.CHAT, X)]), X)
54 |
55 |
56 | class AvailabilityTest(unittest.TestCase):
57 |     def test_degraded_counts_as_available(self):
58 |         self.assertEqual(availability_percent([U, D, X, U]), 75.0)
59 |
60 |     def test_unknown_ignored_and_empty(self):
61 |         self.assertIsNone(availability_percent([]))
62 |         self.assertIsNone(availability_percent([N, N]))
63 |         self.assertEqual(availability_percent([U, N]), 100.0)
64 |
65 |
66 | class IncidentsTest(unittest.TestCase):

```

```

67 |     def test_groups_consecutive_bad_runs(self):
68 |         pts = [Point(0, U), Point(10, X, "task: timeout"), Point(20, D, "slow"), Point(30, U), Point(40, D, "slow"), Point(50, U)]
69 |         inc = incidents(pts, min_degraded_runs=1)
70 |         self.assertEqual(len(inc), 2)
71 |         self.assertEqual((inc[0].start, inc[0].end, inc[0].worst, inc[0].runs), (10, 30, X, 2))
72 |         self.assertEqual(inc[0].reason, "task: timeout") # reason of the worst run
73 |         self.assertEqual(inc[0].duration_s(now=99), 20)
74 |
75 |     def test_single_slow_run_is_not_an_incident(self):
76 |         pts = [Point(0, U), Point(10, D, "slow"), Point(20, U), Point(30, D), Point(40, D), Point(50, U)]
77 |         inc = incidents(pts)
78 |         self.assertEqual([(i.start, i.runs) for i in inc], [(30, 2)])
79 |         self.assertEqual(len(incidents(pts, min_degraded_runs=1)), 2)
80 |
81 |     def test_ongoing_incident(self):
82 |         inc = incidents([Point(0, U), Point(10, X, "down")])
83 |         self.assertIsNone(inc[0].end)
84 |         self.assertEqual(inc[0].duration_s(now=70), 60)
85 |
86 |     def test_unsorted_input_and_no_incident(self):
87 |         self.assertEqual(incidents([Point(30, U), Point(10, U)]), [])
88 |         self.assertEqual(len(incidents([Point(20, U), Point(10, X)])), 1)
89 |
90 |
91 | class TimelineTest(unittest.TestCase):
92 |     def test_worst_per_bin_and_empty_bins(self):
93 |         pts = [Point(1, U), Point(2, X), Point(15, U), Point(29.9, D)]
94 |         bins = timeline(pts, 0, 30, 3)
95 |         self.assertEqual([(t, s, n) for t, s, n in bins], [(0, X, 2), (10, U, 1), (20, D, 1)])
96 |         self.assertIs(timeline([], 0, 30, 3)[0][1], N)
97 |
98 |     def test_outside_range_and_bad_args(self):
99 |         self.assertEqual(timeline([Point(-1, X), Point(30, X)], 0, 30, 3)[0][2], 0)
100 |         self.assertEqual(timeline([], 10, 10, 3), [])
101 |         self.assertEqual(timeline([], 0, 10, 0), [])
102 |
103 |
104 | class CurrentSinceTest(unittest.TestCase):
105 |     def test_since_first_of_trailing_streak(self):
106 |         self.assertEqual(current_since([Point(0, U), Point(10, X), Point(20, X)]), (X, 10))
107 |         self.assertEqual(current_since([Point(20, U), Point(0, U)]), (U, 0))
108 |         self.assertEqual(current_since([]), (N, None))
109 |
110 |
111 | if __name__ == "__main__":
112 |     unittest.main()
=== END FILE: tests/unit/test_domain.py ===

```

```
=== FILE: tests/unit/test_parsers.py (80 lines) ===
```

```

1 | import unittest
2 | from pathlib import Path
3 |
4 | from tests.helpers import FIXTURES, make_config
5 | from vero_monitor.checks.vero_chat import parse_reply
6 | from vero_monitor.checks.vero_cli import clean, hard_limit_s, parse_stream, task_command
7 |
8 |
9 | class StreamParserTest(unittest.TestCase):
10 |     def test_real_stream_from_integration_guide(self):
11 |         s = parse_stream((FIXTURES / "vero_task_stream.ndjson").read_text().splitlines())
12 |         self.assertEqual(s.task_id, "1790868292727")
13 |         self.assertEqual(s.completion, "OK")
14 |         self.assertEqual(s.provider_id, "bedrock")
15 |         self.assertEqual(s.model_id, "us.anthropic.claude-haiku-4-5-20251001-v1:0")
16 |         self.assertEqual(s.events, 8)
17 |
18 |     def test_only_completion_result_is_the_answer(self):
19 |         s = parse_stream(['{"type": "say", "say": "text", "text": "OK", "partial": false}'])
20 |         self.assertIsNone(s.completion)
21 |
22 |     def test_partial_completion_ignored(self):
23 |         s = parse_stream(['{"type": "say", "say": "completion_result", "text": "0", "partial": true}'])
24 |         self.assertIsNone(s.completion)
25 |
26 |     def test_garbage_lines_ignored(self):
27 |         s = parse_stream(["", "Vero banner", "{not json", "[1,2]", "'str'", '{"type": "task_started", "taskId": 7}'])
28 |         self.assertEqual((s.task_id, s.events), ("7", 1))
29 |
30 |     def test_error_event_captured_and_cleaned(self):
31 |         s = parse_stream(['{"type": "error", "message": "Unauthorized_Bearer abc.def"}'])
32 |         self.assertEqual(s.error, "Unauthorized [masked]")
33 |
34 |
35 | class CleanTest(unittest.TestCase):
36 |     def test_masks_credentials_and_flattens(self):
37 |         self.assertEqual(clean("a\n_b\tc"), "a b c")
38 |         for secret in ("Bearer eyJabc", "token=abc123", "token: abc", "password=hunter2", "aws_secret_access_key=x"):
39 |             self.assertNotIn(secret.split()[-1].split("=")[-1].split(":")[-1].strip(), clean(f"err {secret} end"))
40 |
41 |     def test_length_limited(self):
42 |         self.assertEqual(len(clean("x" * 1000)), 200)
43 |
44 |
45 | class TaskCommandTest(unittest.TestCase):
46 |     def test_placeholders_filled(self):
47 |         cfg = make_config(
48 |             Path("/tmp/x"),
49 |             vero={
50 |                 "command": "vero",
51 |                 "task_args": [{"task", "--json", "-m", "{model}", "-t", "{timeout}", "-c", "{workdir}", "{prompt}"],
52 |                 "task_timeout_s": 180,
53 |             },
54 |             schedule={"interval_minutes": 15},
55 |         )
56 |         args = task_command(cfg, "C:/vero.cmd", Path("/w"))
57 |         self.assertEqual(
58 |             args,
59 |             ["C:/vero.cmd", "task", "--json", "-m", cfg.vero.model, "-t", "180", "-c", str(Path("/w")), "Reply with exactly: OK"],
60 |         )
61 |         self.assertNotIn("--yolo", args) # never auto-approve (ADR-003)
62 |         self.assertEqual(hard_limit_s(cfg), 210)
63 |
64 |
65 | class McpReplyTest(unittest.TestCase):
66 |     def test_plain_json(self):

```

```

67 |         self.assertEqual(parse_reply('{"jsonrpc":"2.0","id":1,"result":{}}'), {"jsonrpc": "2.0", "id": 1, "result": {}})
68 |
69 |     def test_sse(self):
70 |         body = 'event: message\ndata: {"jsonrpc":"2.0","id":1,"result":{"serverInfo":{"name":"WChat MCP Server"}}}\n\n'
71 |         self.assertEqual(parse_reply(body)["result"]["serverInfo"]["name"], "WChat MCP Server")
72 |
73 |     def test_error_and_garbage(self):
74 |         self.assertIn("error", parse_reply('data: {"jsonrpc":"2.0","id":1,"error":{"message":"bad"}}'))
75 |         self.assertIsNone(parse_reply("<html>login page</html>"))
76 |         self.assertIsNone(parse_reply('{"jsonrpc":"2.0"}'))
77 |
78 |
79 | if __name__ == "__main__":
80 |     unittest.main()
=== END FILE: tests/unit/test_parsers.py ===

```

```
=== FILE: tests/unit/test_scheduler.py (49 lines) ===
```

```

1 | import unittest
2 | import xml.etree.ElementTree as ET
3 |
4 | import tests.helpers_ as noqa: F401
5 | from vero_monitor import scheduler
6 |
7 | NS = "{http://schemas.microsoft.com/windows/2004/02/mit/task}"
8 |
9 |
10 | def parse(xml):
11 |     return ET.fromstring(xml.split("\n", 1)[1])
12 |
13 |
14 | class TaskXmlTest(unittest.TestCase):
15 |     def test_check_task_laptop_safe(self):
16 |         root = parse(scheduler.probe_task_xml(15))
17 |         s = root.find(f"{NS}Settings")
18 |         for tag, val in (
19 |             ("DisallowStartIfOnBatteries", "false"),
20 |             ("StopIfGoingOnBatteries", "false"),
21 |             ("StartWhenAvailable", "true"),
22 |             ("MultipleInstancesPolicy", "IgnoreNew"),
23 |             ("ExecutionTimeLimit", "PT1H"),
24 |         ):
25 |             self.assertEqual(s.find(NS + tag).text, val, tag)
26 |             self.assertEqual(root.find(f".//{NS}Repetition/{NS}Interval").text, "PT15M")
27 |             args = root.find(f".//{NS}Exec/{NS}Arguments").text
28 |             self.assertIn("vam.py", args)
29 |             self.assertIn("check --trigger task", args)
30 |             self.assertEqual(root.find(f".//{NS}RunLevel").text, "LeastPrivilege")
31 |
32 |     def test_dashboard_task(self):
33 |         root = parse(scheduler.dashboard_task_xml())
34 |         self.assertIsNotNone(root.find(f".//{NS}LogonTrigger"))
35 |         self.assertEqual(root.find(f".//{NS}ExecutionTimeLimit").text, "PT0S")
36 |         self.assertIn("serve --no-scheduler", root.find(f".//{NS}Exec/{NS}Arguments").text)
37 |
38 |     def test_names_differ_from_the_old_latency_monitor(self):
39 |         self.assertNotIn("Latency", scheduler.PROBE_TASK + scheduler.DASH_TASK)
40 |
41 |     def test_cron_and_alignment(self):
42 |         self.assertEqual(scheduler._cron_expr(15), "*/15 * * * *")
43 |         self.assertEqual(scheduler._cron_expr(120), "0 */2 * * *")
44 |         self.assertEqual(scheduler.next_aligned(900, 1000), 1800)
45 |         self.assertEqual(scheduler.next_aligned(900, 1800), 2700)
46 |
47 |
48 | if __name__ == "__main__":
49 |     unittest.main()

```

```
=== END FILE: tests/unit/test_scheduler.py ===
```

=== FILE: uninstall.bat (7 lines) ===

```
1 | @echo off
2 | rem Removes the scheduled tasks and the desktop shortcut. Data and config stay.
3 | setlocal
4 | cd /d "%~dp0"
5 | call scripts\_py.bat || exit /b 1
6 | %PY% vam.py uninstall
7 | pause
=== END FILE: uninstall.bat ===
```

```
=== FILE: uv.lock (8 lines) ===
1 | version = 1
2 | revision = 3
3 | requires-python = ">=3.8"
4 |
5 | [[package]]
6 | name = "vero-availability-monitor"
7 | version = "3.0.0"
8 | source = { editable = "." }
=== END FILE: uv.lock ===
```

```
=== FILE: vam.py (12 lines) ===
 1 | #!/usr/bin/env python3
 2 | """Entry point. No install needed: python vam.py <command>."""
 3 |
 4 | import sys
 5 | from pathlib import Path
 6 |
 7 | sys.path.insert(0, str(Path(__file__).resolve().parent / "src"))
 8 |
 9 | from vero_monitor.cli import main
10 |
11 | if __name__ == "__main__":
12 |     sys.exit(main())
=== END FILE: vam.py ===
```

```
=== VERIFY SCRIPT: verify_rebuild.py (80 lines) ===
```

```
1 | import hashlib, pathlib, sys
2 | # usage: py verify_rebuild.py C:\Tools\vero-availability-monitor
3 | root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else "vero-availability-monitor")
4 | CRLF = ['START_HERE.txt', 'check_now.bat', 'scripts/_py.bat', 'setup.bat', 'setup_demo.bat', 'start_dashboard.bat', 'status.bat', 'uninstall.bat']
5 | FILES = {
6 |     '.ci/README.md': '84f4caf43cc1c411273eeab9ba9ded9054a8595c6ef8a85b68bab0d1ee383076',
7 |     '.gitattributes': 'de5647982a8a835e7449c0a776bc039927e23530e9bdc1d588961500bb5af61f',
8 |     '.gitignore': '7756a240e02140de687ed2f99bc4d70c8e1befd9764916064fb9e7eb4aa2d93c',
9 |     'AGENTS.md': '44800f1ef26c843a15ceeeddf9569021a991c36318777ed97ab7e9454e57d8bb',
10 |    'CHANGELOG.md': 'e0c7d175509ba7deea14ba3e4d0055de3d6787c1242210e24b8de0f8c7f9b7d7',
11 |    'CODEOWNERS': '695a47f89d8c85232ee4946424a5f6a87f2fcea7a945bbc6abb6701981b09d44',
12 |    'CONTRIBUTING.md': '0309ecb3676545058b6c326e9f4c90ba88b4e62f484b74c59b774d0c55f0fdef',
13 |    'README.md': '9becd6a527151eeacd2d886d603ab73d2ce8a65d02c1c683ba1d26da25f85de',
14 |    'START_HERE.txt': '2d1c4ef3ea253fcd055ee7882780ecd85636c481e38005f3f69db1bb9c61e871',
15 |    'VERO_CONTEXT.md': 'd3fa978304042979db68092fd43a3788ded2c788dfc4798e49a9eadc2c20d22c',
16 |    'check_now.bat': '4bb4f1d8e7f19ffdc273ca2b4809be61fd0644faba3bd4a1a7d8e5a81581748',
17 |    'config/monitor.example.json': '9e78c6a7a86a94f21d5a62e20e7ec1e3479765a568b3aa554066a50a0f740c35',
18 |    'docs/ARCHITECTURE.md': '44ff1006d66b37dee2156ac7c4425ae2bbd2ea18fcbe0de5f205a2d8797cbb6a',
19 |    'docs/DESIGN_SYSTEM.md': 'fee93a3b440e1dab0f5ab8d7c4e4e1e7ea2090fdf1e5ea5b559f9b959f7c9b22',
20 |    'docs/PRD.md': '884cfd889e035fe1a5d5979df28f26e34f5cb9b545130c102d121e3a87a4fc61',
21 |    'docs/decisions/ADR-001_stdlib-only.md': '0e8be0912e36d2a913c3185ba34e08f57bc39ac457bd6eba1648fe048f4a00e8',
22 |    'docs/decisions/ADR-002_availability-not-latency.md': 'e1eba8243b818e04e5da7d10e81b054b02ca52f7dd7f8c05cd41afefa91c297',
23 |    'docs/decisions/ADR-003_no-yolo-sandbox.md': '8ddb2cab993e4b50ed65cc98427a18348a82bdacc43d7b9bab09398aa76dc72',
24 |    'docs/decisions/ADR-004_vero-chat-handshake.md': 'd2afb7b0b6032cf142e87684d8a8b6946d091dc021d94cfaa8a51f4c772dde6a',
25 |    'pyproject.toml': '536774c15d69d08ec17acd662efa7ace556ac35d713c8a9fa5d383d4831e0ee5',
26 |    'requirements.txt': '97a344f15656255459c548b3281186ad0860a97ae2c8d962911507d6102ec837',
27 |    'scripts/_py.bat': 'ee03008dcfe04647b013bb12a8f75bfaabcl2ebbc3395986b8829fdd9cc96',
28 |    'scripts/build_rebuild_md.py': 'd2630ee8d7d70ea5e6b533de0df3eeef1ac32d115e34c772f4536277000bac0',
29 |    'scripts/build_rebuild_pdf.py': '943c126f1de7b3ad64188014fe7989b1d3a5eb3b87a28e54683c7accd1025109',
30 |    'scripts/fake_vero.py': '8ac166e8e0e0b845a37161c46ed3192ded0a91a35beb10222bfeac86289a6fd7',
31 |    'scripts/seed_demo_data.py': 'de8000130adac8ddad6337cd3c3105120af9247814f2238cb89f535f6afe53f5',
32 |    'setup.bat': '1676ff1a7460c049868868237db86fba6173e107ea78a2f321b56f40b3ae02f6',
33 |    'setup_demo.bat': '4881f05e4c2bfef053ed4d0cc03fe585ef4544b549d7e43a9f1b9b28291de648c',
34 |    'src/vero_monitor/__init__.py': 'c77b5ab0da53e27487203d17afb38f8dc9ba0c68a7962759827afd5c9c27aee6',
35 |    'src/vero_monitor/__main__.py': '13a1a5b340cdcf1902b62be90e508c7c71886000d5bf087e7854aadf09fb35e',
36 |    'src/vero_monitor/checks/__init__.py': '4e22f94dd92edc74066e117897f3a52506ed23460fdcc8b42192aeee28a3fc38',
37 |    'src/vero_monitor/checks/process.py': 'bfeb6ac9be3f9e9a93c0cb0d68f4b1d121aac015d5154b8b7626a021c509fd94',
38 |    'src/vero_monitor/checks/vero_chat.py': 'f08bf1bf4287d3ccb0e20395517fdd5f49481f91c0bedbab4373e4af4e9e9209',
39 |    'src/vero_monitor/checks/vero_cli.py': '3ddceef9910b68e3f54311b3996aff4e53d8175c82761e855231ad712c35efef',
40 |    'src/vero_monitor/cli.py': '4c4986cdc26cc381eff0ac137e195eb8105c238a196b138f2b2dc4f5dc8d5b56',
41 |    'src/vero_monitor/config.py': 'e7d56144b413831fe7acef7313ec9ef7890f4750bdbc9753b4f07c5c636d1aec',
42 |    'src/vero_monitor/domain.py': 'a6baf2d88794a41ffa5a50fcdf60a8c0fe0556afe8e8621a556bb6474b0a2cd',
43 |    'src/vero_monitor/report.py': 'ecf99cc7db72004040d9d44f247243b701d26e431bb30af6d1734221cbf5cbef',
44 |    'src/vero_monitor/runner.py': '9c72b7358f12d5c068731c7560eac9efe744f95008c68e3175235a9b5ca27834',
45 |    'src/vero_monitor/scheduler.py': '758cf1aba70498ae6a2af6c6e603f77839af12855ed453bf3e9cb05a3c462b0b',
46 |    'src/vero_monitor/server.py': '42a1833e37c8f670a622242e5b6d6d8ffb7c685a20b25ab8ee88328256b58da6',
47 |    'src/vero_monitor/store.py': 'b144a73706a236cc8bc60e847995175e4fc644cd8495ff497110845747de4e0a',
48 |    'src/vero_monitor/web/dashboard.html': 'd43abb27805424ff80a82f8b157d7d76edcf9c18c09db758502daa3f1723b679',
49 |    'start_dashboard.bat': 'c9b963844055acd44896d5c42375bf0b84e41edb5190aa65cdfa576dd6fa6b9b',
50 |    'status.bat': '6a754b5b8c1f74cd0842a9641d535ea25baf4dd5349a1819ce56703e3c33d65b',
51 |    'tests/__init__.py': '7e9af23c4d8769e855436c950c52874ef9d5ac0c36f24531913b3ca965d6860f',
52 |    'tests/fixtures/.gitkeep': '2be8225f3a9ad9e9ee49e2dbc7a20ef74b7d127b93047271e68eb3ba7d3992ec',
53 |    'tests/fixtures/vero_task_stream.ndjson': 'e4018110354d711892f0e577d030159c43140f0e405edc88af74ccd2569a220b',
54 |    'tests/helpers.py': '0b35f2b3014752b801454367100e14ba0b2744ee9fdb9ad8a4c6d0957d6a28e2',
55 |    'tests/integration/__init__.py': '629ee0283bc7304250576c851f51e068a0625c70ada33c06f5abb2e3f8dedbc',
56 |    'tests/integration/test_chat.py': '4462c0068b40fc9d56a3800129a92653e849006c84976a65ed6c5345964133d4',
57 |    'tests/integration/test_checks.py': 'd033eb5c33fbbed346aa1700d1ac3aaafc9eafc67c5d67b9d03a9d052809592d6',
58 |    'tests/integration/test_runner_server.py': '3bd30361d1bed25273da1911bf7a96321d2ddbf21b52a123bc44e63d41311b3',
59 |    'tests/unit/__init__.py': '34d8097b45b98a68acd466b69b9balcec7b2d93164ec960b913db0795e38c290',
60 |    'tests/unit/test_config.py': 'b089b300e44f15cafe6b466898610440389f4cc31a61eda5660877c56b1d2a91',
61 |    'tests/unit/test_domain.py': '9401a448b001527ce750da702d1db15a9c27ccea51a2a4523d0b4c1b6c0339c2',
62 |    'tests/unit/test_parsers.py': '83b0765d6362af1683ef5d3d6452e3707bea16cd03deb240b2036bee96e2fb3a',
63 |    'tests/unit/test_scheduler.py': '6d6460ec569ce5a126ab5a706932b6c7b197f61854b171360fe65deb2a15d959',
64 |    'uninstall.bat': 'bce5f932e3942a7e39f3611867464061e55713a513f0526388368abed454e464',
65 |    'uv.lock': 'e01fcb44c5572550cf4f0d84f325540253b782395df574ab4981bbc8d033a9fd',
66 |    'vam.py': '25f30ade67d0e13c78622e8bc991228535d2535de206892795e9da557f5d8f1',
```

```

67 | }
68 | bad = 0
69 | for path, sha in FILES.items():
70 |     p = root / path
71 |     if not p.exists():
72 |         print("MISSING", path); bad += 1; continue
73 |     text = p.read_bytes().decode("utf-8").replace("\r\n", "\n")
74 |     if hashlib.sha256(text.encode("utf-8")).hexdigest() != sha:
75 |         print("DIFFERENT", path); bad += 1; continue
76 |     want = text.replace("\n", "\r\n") if path in CRLF else text
77 |     if p.read_bytes() != want.encode("utf-8"):
78 |         p.write_bytes(want.encode("utf-8")); print("fixed line endings", path)
79 |     print("OK: all files match" if not bad else str(bad) + " file(s) wrong: recreate them from the document")
80 | sys.exit(1 if bad else 0)
=== END VERIFY SCRIPT ===

```